

Contents

Editorial	John Jacob	2
-----------	------------	---

News

BAA Chairman's report 2015	Paul Grosvenor	3
BAA AGM Minutes 2015	Chris Hogan	4
Dyalog	Morten Kromberg	9
Optima Systems Ltd	Paul Grosvenor	15
APL2000		17
4xTra Alliance	Chris Hogan	22
Kx Systems	Abby Gruen	23

General

SwedAPL November 2015	Samuel Gutsell	25
Second childhood	Stephen Taylor	27

APL

A notation for APL array embedding and serialization	Phil Last	36
Reading new code	Dan Baronet	46
Sixteen APL amuse-bouches	Roger K.W. Hui	51
Pi, Pin and Ping	Ray Cannon	63
APL and partitioned data	Jonathon Barman	77

J

J-ottings 58 – Internal Rate of Return and related Norman Thomson topics		93
Using J for Actuarial Applications Part I: The Chain-Ladder Method	Jeremy Smith	100

kdb+ & q

Book review Q Tips: Fast, Scalable and Maintainable Kdb+	Yang Guo	107
Letting data tell a story with kdb+: what makes a good tennis match?	Matthew Doherty & Paul Loughran	112

A graphical sandbox for K

John Earnest

122

Editorial

This issue of *Vector* has been some while coming but brings a range of articles from the APL world covering robots, sport, education and finance; as well as papers that look both forward and back in considering some fundamental programming aspects.

A dramatic tale unfolds in “A second childhood” as Stephen Taylor relates how the Childcare Company, not only got out of trouble with the help of an APL developer, but produced an award winning solution into the bargain.

In “Sixteen APL amuse-bouches” Roger Hui examines sixteen concise phrases that contain some large concepts. He explores their nature and imparts some entertaining history along the way.

There are three very welcome kdb+ and q related papers. A book review of Yanq Guo reviews “Q Tips: Fast, Scalable and Maintainable Kdb+” by Nick Psaris; Matthew Doherty & Paul Loughran have some fun on the tennis court with “Letting data tell a story with kdb+”; John Earnest describes iKe, an interpreter written in JavaScript, in “A graphical sandbox for K”.

Reprinted in this issue is “APL and partitioned data” by Jonathon Barman together with an introduction by Phil Last. This article appeared in the very first issue of *Vector*. Several people have told me how influential they found the original article and I have encouraged the Optima apprentices to read it. I hope you will agree it is still a relevant paper.

John Jacob

Chairman's Report – August 2015

Paul Grosvenor



Paul Grosvenor

Welcome back to Vector I hope you all enjoy the news, views and muse.

So what has been going on? Well in April and November last year we saw two more Swedish APL group meetings held in Malmo and Gothenburg. About 20 participants attended and discussed a wide range of subjects. A good time was had by all. The next meeting will be held on 1st April in Copenhagen and is being hosted by SimCorp.

BAA London continues to meet every month but now at the Hoop and Grapes, Farringdon Street, London and these are now beginning to be transmitted live via a web link when possible. If anyone is interested to listen in or even attend (which is free) then please join the BAA London Google Group where all the relevant information can be found.

On the 29th May we held our AGM in a combined event with BAA London, held in the Albion, London. The minutes of this meeting are published later in this edition of Vector. Our meeting was a little longer than usual (nearly an hour!) but we did get to discuss a couple of important matters and pass some resolutions.

We are now looking to plan our 2016 AGM and maybe make a whole day of it with presentations from a variety of sources. We will keep you informed.

Please keep your articles coming as without them we will have no Vector. For those of you that do provide articles to us, thank you all for your commitment and support. As always your help in the production of this publication is needed. We know that your time is precious but if you can spare us just a little of it please contact one of the committee members (at the back of this edition) and let us know.

As always your comments are always welcome. Please forward anything you have to editor@vector.org.uk

Thank you for reading and all the very best

Paul Grosvenor

British APL Association AGM - Friday 29th May 2015

Venue

The Albion (2/3 New Bridge Street, London EC4V 6AA)
and by Webinar (hosted by Optima Systems).

Agenda

Apologies

- Peter Merrit
- Phil Last

Minutes of AGM 2014

The minutes of the 2014 AGM (as published in *Vector* 26:2 and 3) were approved unanimously

Report from the Chairman (Paul Grosvenor)

- *Vector* takes more time but vector next in June Colour limited by cost
- BAA website on new linux server soon
- thanks to BAA London
- requests for help to keep things going

Report from the Treasurer (Nicholas Small)

(including report from membership secretary)

Income/Receipts	2014/15 (R&P)	2013/14 (R&P)
Subscriptions	2,434.52	2,964.42
Vector advertising	325.00	0.00
Bank interest	34.25	13.38
Total receipts	2,793.77	2,967.80

Expenditure/Payments		
Meetings	0.00	0.00

Administration	17.95	17.25
Vector production and despatch	2,559.36	2,639.01
Conferences and seminars	0.00	113.92
Other	176.57	157.13
Total payments	2,753.88	2,927.31
Assets summary:		
Bank and other balances	12,275.99	12,246.10
Debtors	757.50	1,197.50
Creditors	-3,835.00	-6,425.00
Net assets	9,238.49	7,018.60

- No questions were raised about the financial report, although Gitte requested a short discussion with Nicholas after the meeting.
- The association has new individual members one in Australia and one in the USA
- The total membership is currently: 178 individual members with 327 copies of *Vector* circulated

Election of the committee for 2015-2016

Paul Grosvenor proposed the committee as it is currently composed:

- Phil Last - Events
- Paul Grosvenor - Chairman
- John Jacob - Vector Editor
- Kai Jaeger - Production manager
- Peter Merrit - Secretary
- Nicholas Small - Treasurer and Membership Secretary
- Ian Clark - Projects

Kai suggested the position of Production manager be abolished as it is no longer necessary.

The ensuing discussion also concluded that webmaster not a committee role and as we have no active projects the Projects officer is also no longer required.

As Peter has not been at the AGM recently, several attendees suggested that we have a new secretary. Stephen Taylor volunteered

- Proposed Kai Jager
- Seconded Chris Hogan

As the constitution permits secondment of association members on a temporary basis, a

new committee structure was proposed with the following officers:

- Phil Last - Events
- Paul Grosvenor - Chairman
- John Jacob - Vector Editor
- Stephen Taylor – Secretary
- Nicholas Small - Treasurer and Membership Secretary

A show of hands approved the changes, as did Ray Canon remotely

Appointment of Auditor

Chris Hogan was reappointed as the association's auditor

- Proposed Paul Grosvenor,
- seconded Kai Jaeger

Questions

- *Vector* should be produced a fixed number of times per year not four, two or three.
 - Proposed Kai Jaeger
 - Seconded Stephen Taylor
- The sustaining members should be invoiced on a 12 Month rolling basis, rather than when a certain number of issues of *Vector* have been published.
 - Proposed Gitte Christiansen
 - Seconded John Jacob
- Ordinary members should also be converted to a per annum basis
 - Proposed Kai Jaeger
 - Seconded Graeme Robertson
- The library subscription “membership” should still be per volume and the association has some corporate members – for the time being they no changes were proposed to either of these grades of membership

All of these resolutions were carried by a show of hands, plus approval remotely by Michael Hughes and Ray Canon

Any other business

It was proposed anyone may receive a fee for work required by the committee

- Proposed Kai Jaeger
- Seconded Stephen Taylor

This was passed, subject to a check of the constitution, which forbids payments to committee members simply for being members of the association.

Attendees

- Remote
 - Ray Canon
 - Brian Becker
 - Michael Hughes
- In Person
 - Kai Jaeger
 - Chris Hogan
 - John Jacob
 - Paul Grosvenor
 - Nicholas Small
 - Graeme Robertson
 - Stephen Taylor
 - Morten Kromberg
 - Gitte Christiansen

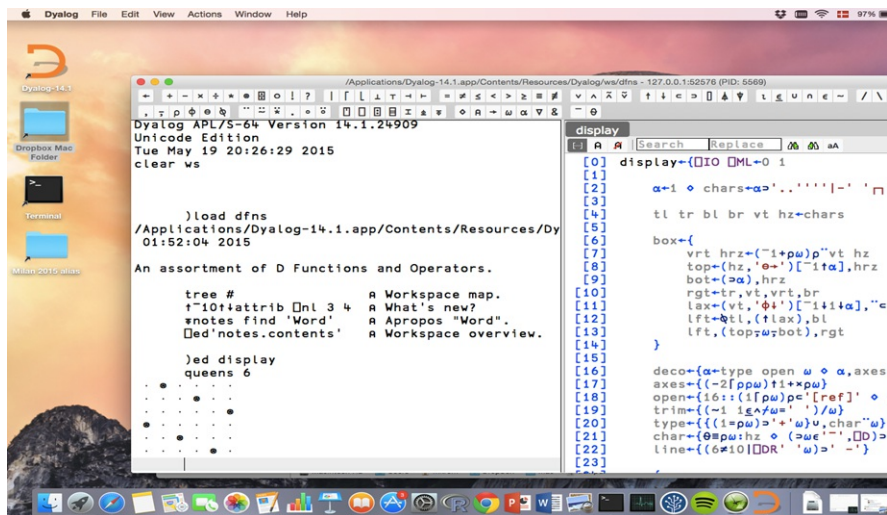
The meeting was followed by presentations by Dyalog

News

Dyalog Ltd industry news

Morten Kromberg

2015 was another extremely busy year! Version 14.1 was released in May with support for Apple OS X and a host of other features. For an introduction to v14.1, see the last part of this news article.



Dyalog version 14.1 under Apple OSX

We were very happy to see that the inaugural two-day Dyalog North America User Meeting [1] in Princeton New Jersey in April managed to gather almost 50 people, and we still had a full house at the global event in Sicily in September[2]. We also got some decent exposure outside our own events: A few days after Dyalog'15 Jay Foad and Morten Kromberg presenting and running a full day workshop on Dyalog APL at FuConf'15[3] in Bangalore, India. In June, Morten presented APL in a Google Talk[6].

It is also very encouraging to see that APL users are slowly learning to consume and contribute to various forms of social media, like our #onlinerwednesday[4] tweets and our developer blog[5], which has a contribution by a member of the development team every week or two. If you want to read about proposals for new APL language extensions or other potential features, follow us, *like* us, sign up for our online newsletter. Contribute to the discussions on line via Facebook, LinkedIn and Twitter – keep an eye on global APL meetings at *dates for your diary*[7]. DYNA'16 will be in Princeton again this year on April 18th & 19th, and Dyalog'16 in Glasgow, October 9th-13th.



Version 14.1

Version 14.1 was packed with features aimed at increasing developer productivity, hardware utilisation, scalability and the security of your applications. Version 14.1 is also available on a new computing platform, Apple OS X.

Dyalog for Apple OS X

From version 14.1, Dyalog is proud to include Apple OS X in the family of fully supported platforms - with a new cross-platform graphical development environment to go with it. The new development front-end, dubbed RIDE 2.0, is available under Windows, Linux and OS X – with a browser-based version in the pipeline. From any of these platforms, RIDE can connect to Dyalog APL running on any one of the growing list of supported execution platforms: Mac OS X, Microsoft Windows, IBM AIX, Intel Linux's - and ARM-based Linux's for the Raspberry Pi and other low power computers.

The Mac OS X version includes many of the same libraries and interfaces as Windows and Linux versions, including User Commands and SALT, TCP support (Conga), the MiServer web server/service framework, and the R interface. ODBC support (SQAPL) will follow; please contact Dyalog if you have ODBC installed on your Mac and would like to help with testing.

The new cross-platform RIDE 2.0 is the desktop development environment of choice under Apple OS X and Intel Linux. Under Windows, the *classical* IDE will probably remain the tool of choice for the next several years. The Windows version of RIDE allows debugging of Windows Services and other processes that are unable to interact with a desktop – and also makes it possible to use Windows as a front end for AIX or Linux servers.

Windows administrator privileges no longer required

Access to administrative privileges are rapidly disappearing in large organisations. In response to this, we have removed the final hurdles that required special privileges, and it will be possible to install version 14.1 (and future versions) of the full Windows development environment without administrative privileges.

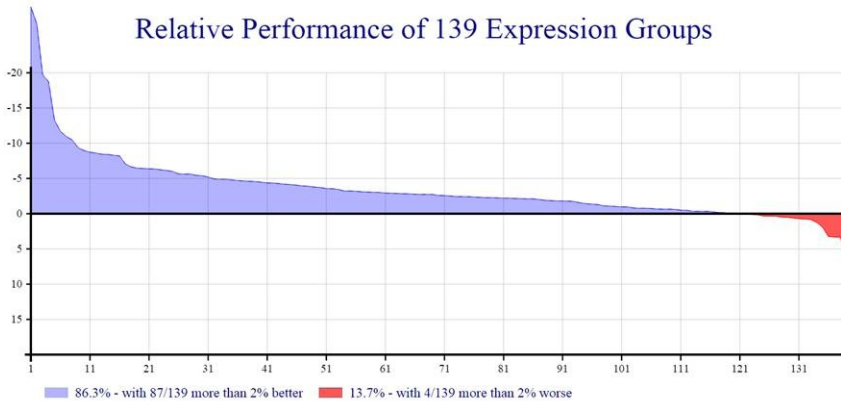
Performance

In recent years, performance has been a major focus of every release. Version 14.1 is no exception, with speed-ups to several primitive functions and operators. A number of idioms have been added or speeded up, including $\#p$, $\neg/$ and $\neg\wedge$, and special code has been added for the $(\geq p)$ train, $\wedge$, $\neg$, $\vee$, $\neq$, $\circ$, $\neg$, $\neg p$ scalar, partitioned enclose, and grade on Booleans.

The following chart is produced by our internal performance test suite, which executes about 14,000 expressions to test all primitive functions and operators with arguments of various types and shapes. The results are grouped into 139 categories, and sorted by performance. Blue means the new version performs better than the old:

Performance Comparison

Between Windows-64 14.1.23745.0 and Windows-64 14.0.21929.0



In addition to the core algorithm improvements, our compiler, which in version 14.0 was only capable of handling a very small portion of a typical application, now supports the most frequently used control structures, and references to global variables and functions. As a result, the v14.1 compiler should be able to compile a significant fraction of functions in many applications. Speed-ups of a factor-of-two for functions with small arguments are likely, but the performance gain will vary depending on language usage. Over the next several releases, the compiler technology should allow further speed-ups as we start applying further optimizations, at the moment it really only removes runtime parsing overheads.

Futures and Isolates

Futures and Isolates are new language features introduced in v14.0, providing a mechanism to parallelise algorithms and take advantage of multi-core CPUs. In version 14.1, isolates make use of the new Conga security features described below to control access to compute servers. Usability is also enhanced through a new feature which allows service threads such as those used by the Isolate mechanism to be designated as un-interruptible; user interrupts will be passed to application threads instead.

Shared code files

Isolates are designed to be able to launch computations in separate processes. An experimental feature known as Shared Code Files can be used to significantly reduce the startup time of isolates in applications with large code bases. An SCF is a workspace that has been saved in a format suitable for memory-mapping. Attaching even a large SCF to the active process takes a split second, after which references to code or data will cause the operating system to page sections of the memory-mapped file in on demand. Once an SCF is memory-mapped, all processes on the same machine, whether they are isolates or processes running on a Citrix farm or other server architectures, share the mapped memory containing code and application constants. This makes it possible to more or less instantaneously start isolate application processes. It can also be used to significantly reduce the start-up time of large legacy applications. For applications with large bodies of

code, memory sharing means that SCF files can significantly reduce overall memory consumption.

Touch-sensitive and high DPI screens

Touch sensitive screens, and screens that have a high number of *dots per inch* are now widespread. User interfaces that use Windows Presentation Foundation, or HTML5/JavaScript via MiServer or other frameworks, have built-in support for touch-driven input events and usually scale automatically to high resolution devices.

However, many existing applications are still based on the Win32-based user interface tools which are built-in to Dyalog (also known as `□WC` GUI), so we have decided to extend these by adding support for modern devices. New events report touch input, and a third co-ordinate system known as scaled pixels allows older applications to look crisp and modern on new devices with little or no rewriting of UI definitions. The classical Dyalog IDE itself is implemented on top of the `□WC` layer, and also has a much more modern look and feel based on the new features.

Improved support for WPF and Microsoft.NET

For the benefit of users of Microsoft's Windows Presentation Foundation, data binding has been extended to provide the ability to bind to a matrix in the APL workspace. This can give significant performance improvements over the existing vector of namespaces approach. A number of other pieces of data binding functionality have also been speeded up.

The new `:Disposable` control structure makes it possible to declare that Dyalog should dispose of Microsoft.NET objects when they are no longer needed: if the referenced objects have a `.Dispose` method, then it will be called in order to release any unmanaged resources that the object was holding, such as file handles.

Security

The TCP tool layer known as Conga has been enhanced with several security-related features. Conga can now make use of certificates saved in the Windows Certificate Store when validating secure connections, making it significantly easier to deploy certificate-based solutions on the Microsoft Windows platform.

It is also possible to configure a Conga-based server to filter incoming connections based on the IP (v4 or v6) address of the peer, or by requiring the client to present a certificate with selected attributes. This means that a server applications written in APL can leave the first level of user validation to the communications framework, leading to higher performance and simpler code.

Clear sensitive memory: Version 14.1 adds a feature which ensures that all free memory in the workspace is overwritten. This allows you to ensure that no trace remains in memory, after your application finishes manipulating sensitive data.

Event Viewer

A number of enhancements have been made to the Event Viewer (many thanks to Kai Jaeger for inspiration):

- User-defined events are now supported
- Selected events can be filtered out from the list of displayed events

- Event information can be copied to the clipboard
- The editor can be opened directly on the call-back function for an event

JSON Parser & Generator

An experimental built-in function is provided to translate JSON to and from several different APL-friendly formats, including namespaces and nested arrays.

Editor and Tracer

- The version 14.1 development environment includes a number of minor enhancements designed to improve productivity. Examples include:
- View dfn arguments in debugger: When suspended in a dfn, `a`, `w`, `aa` and `ww` will appear in hints, and can be viewed in a read-only window.
- Align Comments: The 'Align Comments' feature has been extended to work when editing class and namespace scripts.

References

1. Dyalog North America User Meeting
<http://www.dyalog.com/north-american-user-meetings/2015.htm>
2. Sicily, September 6th to 10th
<http://www.dyalog.com/user-meetings/dyalog15.htm>
3. FuConf'15
<http://functionalconf.com/>
4. #onlinerwednesday
<https://twitter.com/hashtag/onlinerwednesday>
5. developer blog
<http://www.dyalog.com/blog/>
6. Morten presented APL in a Google Talk
<https://www.youtube.com/watch?v=PIM9BXfu7UY>
7. Dates for your diary
<http://dyalog.com/dates-for-your-diary.htm>

Optima Systems Ltd Industry News

Paul Grosvenor – Managing Director



Paul Grosvenor

The last few months have been very busy for us. We have seen a considerable interest shown in our COSMOS product with our support and maintenance customers also being very active. All good news particularly in these times. It seems to me that the long promised 'green shoots' have well and truly started to appear, businesses feel livelier and more willing to do business and the atmosphere somewhat fresher.

Our newest apprentice, Callum, has successfully completed his training and has been taken on by us as a full-time member of staff now as an APL programmer. Callum brings considerable knowledge with him of things other than just APL and is already proving to be a very valuable member of staff. Within the next few days our next APL programming apprentice, James, will be starting with us and we look forward to introducing him to the rest of the APL community. James will bring our apprentice tally up to five.

Our first apprentice intake (they started with us four years ago now) continue to go from strength to strength. Sam concentrating on APL programming and James on IT Support. Both are doing great work and becoming stronger day by day.

The whole apprenticeship exercise has proven to us that bringing in new blood and training to the required standard can be done and is cost effective. We hope to continue this process and will keep you all informed with progress; watch this space ...

We started to design and build our own data analysis and visualisation tool some time ago now and finally things are starting to move. The American markets seem to be moving again and whilst slow there are visible signs If this continues then our operations in America and Sweden are likely to get larger.

Optima works closely with APL companies all around the world and we continue to look for ways in which we might collaborate or just simply share ideas. We would like very much to hear from anyone in the APL community who might like to work with us or simply discuss issues.

If you are from a larger company have you ever considered sharing staff or getting involved in non-confidential strategy meetings to consider potential ways forward? Our view is that the more ways we can help each other the easier it could be to overcome some of the barriers we all face each day. If you have any views or ideas on this then please contact me – paul@optima-systems.co.uk .

Overall we are looking forward to a number of APL events being held this year and being actively involved with projects for the APL community.

All of us here at Optima hope you are experiencing a similar upturn in fortunes and we look forward to hearing your news in the forthcoming conferences.

Best regards to all

Paul Grosvenor

- Proof for author
- 0.1

APL2000 Update

Introduction

APL2000 continues our commitment to developing high quality, feature-rich APL programming products by creating new features and enhancements to improve performance, enhance the experience and expand the reach of APL users.

APL+Win product development focuses on:

- Identification and implementation of performance improvements
- Maintain, to the extent possible, compatibility of customer-developed application software with future versions of APL+Win
- Develop convenient APL+Win interfaces to other environments

What's new in APL+Win V15.0

New and improved system functions

- **Improved `⎕FX`**
The `⎕FX` system function has been improved to allow the argument to be a nested vector of character vectors (or scalars), rather than requiring it to be a matrix.
- **New `⎕CM` - Character Matrix**
The new Character Matrix system function, `⎕CM`, can be used to normalize a character argument into a character matrix result. Like the `⎕FX` system function, `⎕CM` accepts as its argument a nested or heterogeneous array of any rank, gluing all items together into a character matrix.
- **New `⎕CV` - Character Vector**
The `⎕CV` system function can be used to normalize a character vector argument into a character vector result. By default, the character vector result is `⎕TCNL` delimited. However, an optional left argument is available that can specify a different line delimiter.
- **New `⎕GUID` - Global Unique Identifier Generator**
The `⎕GUID` system function is used to obtain a character vector containing a new Global Unique Identifier (GUID). The result may be used as a persistent unique identifier with a high degree of certainty.

New and improved system variables

- **New `⎕CRLF` - Newline + Linefeed**
The `⎕CRLF` system variable returns a two element vector consisting of (`⎕TCNL`, `⎕TCLEF`) and is a convenient left argument to `⎕CV` for producing standard Windows delimited output. In C/C++/C# this is the “`\r\n`” sequence.
- **Improved `⎕R1` - Random Link**
The `⎕R1` system variable has been updated to support several additional pseudo-random number generation algorithms to complement the previously available algorithm.

New primitives

- **Colon primitive (`:`)**
A colon prefix followed by a space (colon-space) at the beginning of a statement in a user defined function will suppress the output on any line of execution including the `:THEN` expression in Inline Control Structures.
- **New Sink primitive (`-`)**
The sink primitive suppresses the output from any line of execution in immediate execution in the session and user defined function when executed monadically.

Additional enhancements

- The APLW.WS ActiveX Server object now supports being instantiated by a 32-bit non-APL+Win client application. Note that this enhancement facilitated renaming the `XStart` method to `XInit`.

- SendInput Windows functions added to APLWADF.INI and APLW*.ADF files.
- The updated aplw.exe.manifest and aplwr.exe.manifest files enable the GetVersionEx Windows function to now correctly detect and report the version number as 6.3 for the Windows 8.1 and Windows Server 2012 R2 operating systems.

APL+Win C# script engine to access the features of the Microsoft .Net Framework directly from APL+Win.

The CSE enables an APL programmer to create multiple instances of the CSE object in memory space separate from that of the initiating APL+Win instance. With the CSE, APL+Win data can be passed to a C# object, where it can be manipulated using the extensive Microsoft .Net Framework and the result can be returned as APL+Win data.

APL+Win C# script engine updates:

- The CSE instance properties 'replstr' and 'usereplstr' properties have been implemented.
- The [CSE] section of the APL+Win .ini file can now be used to set the default values of the CSE 'unicode' and 'usereplstr' properties.
- The CSE system object property 'version' has been implemented so that it is not necessary to create an instance of the CSE to check the currently installed version of the CSE on the target workstation.
- For .Net custom events subscribed by APL+Win the `□carg` system variable is available.
- The values of the APL+Win system variables `□cself` and `□carg` will not be saved with the workspace because if they were so saved when that workspace was loaded these system variable values would not represent existing objects.
- The CSE Script Engine Manual and the associated 'CSE Code Samples' folder has been updated.

APLNext Supervisor for multi-threaded processing in APL+Win applications

The APLNext Supervisor exploits available multiple processors on a workstation to significantly improve APL+Win application system performance. Application systems that are calculation-intensive, and have sections of the processing algorithm that can be asynchronously executed, are best suited to take advantage of multi-core processing.

APLNext Supervisor updates:

New Methods

- 'StartSync' and 'ExecuteAPL' methods for the 'sync' operation mode with potential for improved performance and simplified application system architecture.
- 'GetDebug' and 'SetDebug' to provide verbose log entries for extended debugging information.

New events

- New 'ProcessProgressCallback' Event. The 'kernel' function can trigger this event by using the APL+Win 'Notify' method to provide processing status and feedback to the 'ProcessProgressCallback' event handler in the domain of the 'controlling application.'
- 'ProcessAllDoneCallback' Event has been enhanced.

New error messages

- If the 'Start' method is used after using the 'StartSync', an exception "APLSupervisor started for sync execution already." will be thrown.
- If the 'StartSync' or 'ExecuteAPL' method is used after using the 'Start' method, an exception "APLSupervisor started for async execution already." will be thrown.

Documentation enhancements

- NProcessors property example
- Application system architecture options for using the Supervisor
- Events section improved
- APLNext Supervisor FAQ has been updated
- The 'Debugging the 'Kernel' function' section has been update

APLNext Supervisor deployment tool

- Digitally signed .msi
- Additional options such as silent install.

Sample applications enhancements

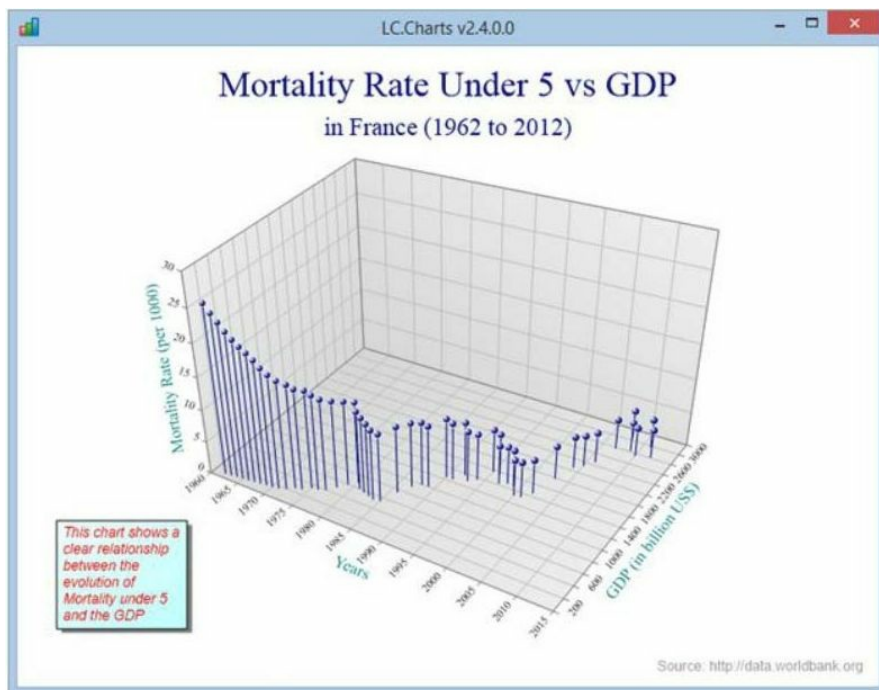
- APL+Win 'supervisortest.w3' for a 'pure' APL sample application
- 'APLNext.SCS' Visual Studio 2013 solution illustrates the use of C# as the 'controlling application' and APL+Win 'kernel' functions performing the processing requested by the 'controlling application.' The 'async' sample in this solution illustrates the 'ProcessProgressCallback' event.
- Set the 'maxpool' configuration property under program control
- Use 'sync' or 'async' mode
- Use events with the 'async' mode
- Sample 'controlling app' and 'kernel function' for each mode

LC.Charts v2.4.0.0: A no-Cost ActiveX component for 2D & 3D charts in APL+Win

Using this component, (implemented and maintained by Lescasse Consulting), APL+Win based application system data may be graphically visualized as two-or-three-dimensional charts.

- Charts may be defined with an APL+Win user command or with the APL+Win `owl` system function.
- Chart objects may be embedded in an APL+Win form or an independent C# form.
- Performance of the component is excellent, e.g. 200,000 points drawn in 100 milliseconds.
- An extensive object model including more than 150 properties, methods and events to facilitate custom annotations, legends, gradients, styles, fonts, formats, titles, etc.
- An included APL+Win user command file provides APL+Win functions illustrating LC.Charts usage.
- LC.Charts may be deployed in your APL+Win-based application system if that application system provides significant additional functionality beyond that of charting.

LC.Charts requires APL+Win v15 and Microsoft .Net Framework 4.0.



Consulting and training

APL2000 provides comprehensive consulting services to address design, development, implementation, conversion, update, enhancement and deployment of APL based application software systems. We also offer all levels of APL training – live seminars, online in a virtual classroom, videos and self-directed tutorials.

For information about APL2000 products and services or to obtain a no-cost trial version of APL+Win contact sales@apl2000.com.

4xtra Alliance - Industry news

Chris Hogan

Not a lot to say in this issue. The separate members of the alliance continue to pursue their own projects, with BAA London being the main activity we perform in common at the moment.

We've continued to organize the regular monthly meetings on APL and related topics and in May we combined with the BAA Annual General Meeting, which will be reported elsewhere in this issue. The meetings can cover a wide range of topics – besides APL specific presentations, we've had talks on general coding principles, test driven development, using GPUs for parallel processing in K, using Dyalog to program the Dyabot robots. Something for everyone.

There may be changes in future as we've been told that our regular venue, the upper room of the Albion pub on New Bridge Street is being redeveloped later this year and it might not be available for hire after that. We'll keep you posted through the Google group.

That's getting a bit off the topic of sustaining member news, so if any of this sounds interesting then keep an eye on the BAA London Google group:

<https://groups.google.com/forum/?hl=en#!forum/baa-london>

As I can't really say what the other members of 4xtra are up to, I'll conclude this short piece, I'll say a little about what HMW has been doing recently and as this is Vector, it will be focused on the APL side of things. Besides continuing to work on DFS for Dyalog, which uses Conga for client/server communications, we've been employing conga in a different context for Web Services.

Freely using Dyalog's SAWS as a model (and raiding Miserver for its JSON to (and from) APL classes, these tools provide an APL client to send notifications via a third party server to smart-phones and an APL service to receive requests from the external server. For pragmatic reasons the incoming service calls use SOAP and the outgoing ones JSON. To co-ordinate the multi-threaded environment these services need, we've included an APL thread management sub-system.

This project will form the basis of a talk at BAA London and maybe a future Vector article.

Kx Systems

Abby Gruen

2015 News

There was a release of kdb+ version 3.3.

Two books for programming in kdb+/q were published. In March 2015 Nick Psaris published “Q Tips: Fast, Scalable and Maintainable Kdb+,” a practical guide to learning the language through building a real life application which is available on Amazon[1]. In November 2015 Jeffry Borror published “Q for Mortals: Version 3, An introduction to Q Programming,” an essential guide for practical information about using q and the kdb+ database. It is also available on Amazon and Amazon UK[2].

In July 2015 Mark Sykes was appointed Global Market Strategist for Kx.

In December 2015 Kx announced that as of January 1, 2016 CEO Janet Lustgarten would become the chairman of the Kx Board and First Derivatives' CEO Brian Conlon would also become CEO of Kx. Mark Sykes was promoted to Chief Operating Officer.

References

1. Psaris, Nick, “Q Tips: Fast, Scalable and Maintainable Kdb+”
Amazon <http://www.amazon.com/Tips-Fast-Scalable-Maintainable-Kdb/dp/9881389909/>
2. Borror, Jeffry, “q For Mortals Version 3: An Introduction to q Programming”
Amazon <https://wwwcreatespace.com/5843336> and
Amazon UK http://www.amazon.co.uk/s/ref=a9_sc_1?rh=i%3Aaps%2Ck%3Aq+for+mortals+version+3&keywords=q+for+mortals+version+3&ie=UTF8&qid=1448215277

General

SwedAPL 2015 Report

Sam Gutsell (sam@optima-systems.co.uk)

Introduction

This year's SwedAPL meeting was held at the Turning Torso in Malmo, Sweden. The turning torso is the tallest building in the Nordic countries, at a height of 190 metres (623 feet). The meeting took place on the 53rd floor, the 2nd highest in the building.



The theme for Autumn's SwedAPL meeting was web services and Gilgamesh Athoraya (Data Analytics) was our host.

Gilgamesh started off proceedings by welcoming the attendees to SwedAPL 2015, he went on to discuss that the SwedAPL name may be changed to something like ScandAPL, to reflect the Scandinavian presence at this meeting and the previous SwedAPL meetings. He then added that there were a variety of nationalities, of which included: Swedish, Danish, German, Serbian and British.

Morning presentations

Gilgamesh then discussed his topic, the implementation of web sockets in MiServer and Rumba. Gilgamesh demonstrated 3 ways in which he had used web sockets; a chat room, a collaborative block building grid and a table shown on a web front end that updated when something changed in the APL server.[1]

Next up was Joakim Hårsman from CompuGroup Medical, who spoke about his experiences of different implementations of SOAP web services, including a C++ solution, a .NET/Dyalog APL solution and a JavaScript solution, but also how they were being maintained.[2]

Just before lunch, we had Stephan Poßberg from Vallourec speak about using web services as a means of validating and distributing information across multiple systems within a company.

Lunch

The lunch took place on the 54th floor, unfortunately the weather wasn't as clear as it could've been but a tremendous view nonetheless and the food was fantastic.







Afternoon presentations

After lunch, Morten Kromberg, CTO of Dyalog gave a talk on RESTful web services in MiServer. He spoke about how Dyalog are trying to make using MiServer to develop applications as similar to how the Windows Create tool set works as possible to make it easier to use. Morten also led a small workshop on RESTful Services in MiServer, spoke about the future of MiServer and about possibly making Rumba a part of MiServer.[3]

Leading on from Morten touching on Rumba, Gilgamesh decided to give a brief overview of Rumba with Paul Mansour (who built Rumba) attending via the web, also chipping in with information about Rumba as Gilgamesh spoke.[4]

Last up was Peter Simonsson from Aplensia, who told us of his and his team's experiences of Web Services at Volvo Cars. He spoke about how the users have local executables which rely on web services to process and provide information that cannot be dealt with locally.[5]

Thank you

Thank you to all of the attendees and a special thanks to all of our speakers, it was a wonderful day and we look forward to seeing you at the next meeting.

1. www.youtube.com/watch?v=NHoa3QaC5SA
2. www.youtube.com/watch?v=NiZffyiYh3A
3. www.youtube.com/watch?v=WvvWphJnX1E
4. www.youtube.com/watch?v=srE7Jmk9tbY
5. www.youtube.com/watch?v=laFU7GnUf_4

Second childhood

Stephen Taylor (editor@vector.org.uk)

Editor's note The story below of what happened to The Childcare Company recounts events which have been the subject of legal proceedings. For this reason, the previous partners of The Childcare Company are not named here. Nor has *Vector* checked this account with them: the subject of our story is not how The Childcare Company got into difficulties, but how it got out.

They say it's never too late to have a happy childhood. But who gets to have their second childhood right after their first? Chris and Sally Eaton of The Childcare Company did, with a little help from an APL developer.

When trouble comes calling, best go out to meet it. But what do you do when the storm you saw on the horizon arrives as a hurricane? You'd been planning how to stay dry when you should have been planning how to save your house.

Sally walked white-lipped out of the conference room for a bathroom break. Outside she saw her IT consultant.

—"Ziggi, leave the building. Go home – we'll talk tomorrow. Do it now. Everybody out – fast!"

She rejoined her husband in the conference room. It was 7 March 2008, their very own Black Friday.

Starting out

Their company had been started to offer a higher standard of training to childcare workers. They both had long experience in this business and had seen that the Web could be used both to make training more easily accessible to workers, many of whom worked irregular hours, and to help their employers and assessors keep in touch with their progress.

The business plan had been straightforward. A services company, (here called 'the service company') had already established an 'e-portfolio' infrastructure for delivering vocational training over the Web. Chris and Sally's company would develop content for vocational training for childcare workers. To do this, and to market the results, they had formed a partnership with a major vendor of nursery software and services, which we shall refer to as 'the parent company', becoming directors of 'The Training Company'.

No one expects even the best-laid plans to run smoothly. By the summer of 2007, product development seemed to the Eatons to have stalled. The service company had the delivery mechanism they needed, but it also had much larger clients; and jobs for larger clients kept

bumping The Training Company's work back down the queue. The service company's delivery framework was well-established and its care divided between numerous specialists. The Training Company's requests for changes and new features seemed to circulate endlessly without result.

Chris & Sally hired Chris 'Ziggi' Paul, an APL developer, to see what he could do to get the lesson content moving forward. He confirmed they had problems.

Yes, development *was* stalled, for a number of reasons. The company providing the framework were asked to develop the site and put more colour into it – we wanted lots of colour, lots of imagery. They had a very traditional-looking system, if you like. Our development there *wasn't* going fast, because they had some very big clients who were asking for things as well, and we were just a little sideshow. So they didn't really have the time, and our requests seemed always to be at the back of a priority list. Things were going very slowly there.

A start-up company can't take three-month delays. But I can understand their point of view. They saw the big client, the big bucks, and initially, when they had taken us on they must have seen us as the icing on the cake. In theory that would work very well. In practice, the big client clicked its fingers, they jumped.

Lisa Sutlieff joined The Training Company about this time, developing content for the new system.

I think that was the point I joined the company. And I just remember – it was such a battle to get anything done, because we would be doing all this work and none of it was actually going live. On the other hand, they had this framework and were expecting us to supply content to a certain formula, and when we said, well, we've done that now, we'd like to do something a bit different, we want to offer a new qualification through your system, then the conversations that we had just went on for weeks. The framework handled one kind of qualification very well, it was just so hard to get anything *else* done.

Ziggi—The parent company's security regime was also a problem for us. They were getting hacked a couple of times a year, so they were naturally concerned. Each of our many, many changes had to pass through their security checks – which still weren't stopping the intrusions. They had all the resources they needed to support our development but they just couldn't get out of their own way.

Ziggi was able to get things moving again. The service company writes its software in VB, not APL, but APL developers commonly handle *all* aspects of an application, and Ziggi's experience enabled him to work effectively between the content creators and the VB programmers, getting logjams unblocked. As an experienced developer he was able to

support the project with an SQL-based content-management system that reduced much custom coding to a collection of templates. He arranged for a development system to be set up on a server at The Training Company, so they could add content without elaborate security checks. By the end of February 2008 the site was already being used by learners and a major nursery chain was about to sign up.

Unfortunately, as the partnership with the service company was beginning to go well, the Eatons were increasingly concerned about their other partnership – with the parent company. In theory its substantial market presence was the ideal complement to the tiny start-up company. In practice, there seemed to be significant incompatibilities. In November 2007 the Eatons proposed an amicable separation, reversing the share swaps and going their separate ways. Negotiations started. Rainy days, perhaps, but the parties might stay dry under separate umbrellas.

Black Friday

That was the agenda for the meeting on 7 March. But Sally had guessed the rainstorm might turn out to be a hurricane. Anticipating an acrimonious meeting and raised voices, she arranged for all staff to be out of the office that afternoon except Ziggi who stayed to take phone calls. She was not mistaken. At the meeting, directors from the parent company announced that the Eatons were suspended from their posts running The Training Company and flourished letters for all their staff, suspending them too and inviting them to the parent company offices to review their contracts. Outside, a van waited to remove all the computers. Sally excused herself from the meeting and secured the computer equipment, all of which belonged to the Eatons personally. None of the employee letters would be delivered that day.

Two weeks later the Eatons sat at the same table with seven of their staff. They had regained some ground. The premises were in their names; the PCs remained. (The separation would later be settled with compensation from the parent company.) But the development server and all their work had disappeared and inquiries at The service company about resuming their project without the parent company did not seem to be making any progress.

The meeting could have been a wake, and perhaps it would have been if the people there had felt they had been treated with greater respect. But the Training Company's employees did not feel like workers laid off from a failed project. They felt mugged; their prize snatched away just before they grasped it. Worse, they feared the system would be poorly supported by the parent company, their work wasted and discredited.

But Ziggi had a proposal.

If you let me – get me APL – and I can write it. And I can write the whole thing, including all the service company parts, and we'll have complete control over it.

We can develop it how we want to develop it, we can adapt very quickly when you want something changed or you don't like something. We won't have to make phone calls or wait for them to get back in a week, we can do it. If we get feedback from our clients we can do it. If any of the nursery chains we start dealing with want different bits we can react at a much faster pace. We can get the product we want – we can do all this.

On the face of it this was an outrageous idea. The service company's infrastructure represented a huge development effort. Even if his colleagues could recreate the training content, how could Ziggi, working alone, possibly create the infrastructure to deliver it over the Web?

Worse, they were out of time. The parent company had its first learners already using the lessons. Web-based training would cause a sea change in training nursery workers. The first firm to bring it to market it would reap a huge first-mover advantage – or, as those around the table feared, wreck the marketplace for it.

But Ziggi wasn't entirely crazy. He was familiar with web applications developed using ASP.Net and VB.Net. He knew how fast he could work using APL. And he had seen the interfaces available from Dyalog. Moreover, he didn't plan to recreate the service company's entire e-portfolio framework, but a subset tailored and enhanced for this training content.

It was a crucial offer. If Ziggi could provide the delivery mechanism then, for the first time, all the skills required to produce the Eatons' new business service were gathered at the same table. They had a chance – however slim – to get to market without partners.

But it was still an outrageous proposal. How did the Eatons come to accept it?

Sally—After our experiences with our partners, the prospect of *total control* – from content to delivery – was very enticing. If it worked, we would be able to react and respond to our customers very fast – crucial for a start-up company.

Chris—It came down to trust. For the previous nine months Ziggi had delivered what he'd said he could deliver, and we had learned to rely on him. If he said he could do it, then he probably could. We're only now starting to appreciate what an extraordinary suggestion it was. So part of it was a character assessment, and part of it was – let's see how far it goes, and if it doesn't work out we can always correct it later.

Starting over

The Childcare Company was formed with fresh memories of work in progress, and a desire to do it better this time around. They started writing their content with

improvements suggested by feedback from their earlier material. And Ziggi got busy with Dyalog APL.

It was his first web application with APL on the server side. After nine months of learning about NVQ training he had a clear view of what was needed and soon had a simple version working.

This produced some interesting feedback. Ziggi knew the application from the conversations and meetings he'd been in, not from studying a formal specification. If the application had important capabilities that had not been discussed since he joined the project, he would know nothing of them. And so it proved. The service company framework provided for internal and external assessors. Established long ago, these had never been on the agenda of any meeting Ziggi attended.

Ziggi—Although I knew the terms I did not really appreciate the complexity. When I first demo'd the Internal Verification tools I was told "This is re-assessing, not IV-ing!" I had to ask what the difference was and then go back and start that part again.

On the face of it, this was a serious flaw in his approach. Without a formal specification, how could he hope to meet the requirements? In practice, once he knew what was wanted he was able to add the extra function quickly.

Ziggi—Bits did come and bite me somewhat. Having knowledge of the lesson aspects of the system, and having seen the service company people demonstrate the learners' upload and download facilities, what I *didn't* know was the complexity on the Assessors' side, or the Internal Verifier or External Verifier roles. I didn't know how they were going to work – I didn't know they were *needed*, actually, when I first agreed to things.

Chris—No, we never said to you that there was an External Verifier page required. We sort of assumed that – we all knew that and it didn't need saying. So we'd never told *you* that. But we came across problems like that and you solved them.

Ziggi—It's one of the other aspects of APL here. I couldn't have changed direction and jumped back and forth so easily if I had had a very rigid SQL file structure and so on. Using APL component file structures and being able to change the programs that use them allowed me to adapt very quickly.

Clearly the Eatons had a judgement call to make. Every programming nerd is happy to be left undisturbed to write code. How long to leave Ziggi? Without technical skills of their own, how to judge whether he was making progress – or stuck in a dead end?

Sally—I knew he was getting somewhere when I began to see ideas and

contributions from other team members turning up in the application.

Ziggi's approach to development depended on close collaboration with the rest of the team to complete his understanding, and to identify and solve problems. It assumed he could write and rewrite code fast enough to expose and fix the inevitable errors and misunderstandings – and that this would produce a usable system faster than formal development methods would. The prompt appearance in the application of ideas from his team mates was a strong signal he was succeeding.

Delivering

As it turned out, the Eatons did not have to wait long. Ziggi had no code when he started redeveloping in April 2008. The service was launched the following month and went live in July with 75 users. By the end of 2008 The Childcare Company was supporting 250 users; a number expected to double by the system's first anniversary. Gratifyingly, the customers include the major prospect they thought they had lost in the rupture of March.



Ziggi's 'direct development' approach had discovered and produced everything required to use the system. In November 2008 the industry awarding body CACHE visited and confirmed that all of the key components needed for assessment and internal and external verification were in place and, in fact, easy to use and view.

Does he have a scaling problem ahead? Having built the system at a gallop will he have to rewrite large portions to accommodate thousands of users? Apparently not. Ziggi:

I can monitor in real time who is on the system. I can see the whole time what is going on. The load on the system, and how it impacts the system, is something that I was very nervous about in the early days. I'm less so now. It can grow quite considerably before it becomes an issue. At the moment, we really... I think we can easily go to two thousand users.

As the eminent application developers 37 Signals write in *Getting Real*, their guide to developing web applications:

In the beginning, make building a solid core product your priority instead of obsessing over scalability and server farms. Create a great app and then worry about what to do once it's wildly successful. Otherwise you may waste energy, time and money fixating on something that never even happens.

Believe it or not, the bigger problem isn't scaling, it's getting to the point where

you have to scale. Without the first problem you won't have the second.



The Eatons have already achieved their goal of setting a new standard of quality in training childcare workers. In November 2008, seven months after Ziggi wrote their first line of code, their industry magazine, Nursery Management Today, awarded The Childcare Company its Supplier Innovator prize for the year. CACHE has given them its highest rating. The Childcare Company is now collaborating with Sir Christopher Ball, author of the 1994 “Start Right” report, and is becoming a channel for content from Teachers TV.

Looking back

The Eatons have had the unusual experience of successfully developing two very similar applications using entirely different methods: on the one hand, conventional IT development, on the other, the Direct Development style more common with the APLs. What lessons do they draw?

Sally—Ignorance is bliss. Thank goodness we had no idea what a radical path we had chosen! But if we had to do it again – or another project – without Ziggi, we'd have to find someone like him, with the character and the people skills to listen and collaborate. He's not the usual sort of person you find working in IT.

Ziggi—The teamwork made it possible. We knew what we wanted to do, we had a huge focus, and a common purpose. The intensity was terrific, and the team just gave me their trust.



Award winning team – the Eatons at left, Ziggi far right

Chris—Compared to working with regular IT, the control – the responsiveness – was unbelievable. Ziggi has completely spoiled us for conventional IT development.

APL

A Notation for APL array Embedding and Serialization

Phil Last (phil.last@4xtra.com)

Most systems include a number of tables or arrays that are referred to frequently but rarely changed. I examine the utility and possibility of making these and other data easily visible, editable and transferable between different systems or parts of a system possibly implemented in different APLs.

Introductory

And I think ... that it's probably time for us to come up with a notation for constants in the language so that ... you can declare matrices and so on in a nice readable fashion. Morten Kromberg, Dyalog'14, Eastbourne, Technical Road Map

My diaeresis hides Morten's emphasis on *scripts*. Certainly *Dyalog* APL's ability to store code in and retrieve it from scripts external to the traditional *workspace* leaves a gap where stored arrays are concerned. But there seems to be no good reason to keep the benefits that would accrue with such an array notation to one limited form of code storage which most APLs don't support. At the same time there is the necessity to transport systems via the internet which requires serialization not only of code, however stored, but of data and in a form that is also independent of data storage.

In what follows all the examples and a model presented use Dyalog APL V14.0 but the proposal is intended to be cross platform within APL. The appendices contain a further proposal to include dictionaries as separate entities in the notation and a short description of a model.

Requirement

The requirement for an array notation has existed since the first implementers of APL omitted to allow for the direct definition of multi-dimensional arrays in the syntax *without function application*.

From very early in my APL career, when writing systems requiring persistent, constant, arrays I was unhappy with the facilities offered for defining and maintaining them and the necessity to save them as global *variables* along with the code. Why not code an easily edited representation of the data into the function and extract it from its own □CR at initialization of the system?

Examples of functions returning their embedded data might be

```

▽ r←fText
  r←2 2⊖CR'fText'
  ⌘ Embedded text array to be
  ⌘ extracted at runtime.
  ⌘ ...
  ▽
    fText

  [-----]
  | Embedded text array to be |
  | extracted at runtime.    |
  | ...                      |
  [-----]

  fNums←{
    ↑×/↑⊖VFI''↑1 2 2⊖CR'fNums'
  ⌘ 01 12 23 34 45 56 67
  ⌘ 78 89 90 01 23 45 67
  ⌘ ...
  }

  fNums''

  [-----]
  | 1 12 23 34 45 56 67 |
  | 78 89 90 1 23 45 67 |
  | 0 0 0 0 0 0 0 |
  [-----]

```

This led very soon to a utility function that would extract all trailing comment lines from the `⊖CR` of its caller. I have tried many other variants over the years such as: that all comment lines starting with a particular string are returned; that all contiguous comment lines immediately following the call are returned; that `⊖VFI` (`⊖VI` and `⊖FI`) is called internally so that in many cases no further processing is required on the returned data; and several that included mark-up for multi-dimensional arrays.

All the above have their drawbacks and inconveniences but I find them vastly more appealing than the repeated assignment and catenation that is currently the only alternative, albeit a marginally more efficient one in terms of actual machine time.

But what I really wanted was a notation, native to APL, that permitted me to code the array directly into the function without the comments; without repeated assignment, catenation and reshape; and without having to extract it with another function. In other words: executable code; an extension to vector notation. But the possibility did not really present itself until 1997 with the release of *Dyalog* 8.1 that included *dfns* for the first time. Here we had a new syntax that permitted a pair of braces to span line-ends within a function rather than being restricted to a single line as were brackets and parentheses.

∇	r_{-f00}	w
	...	
	$f01_{-}$	
	α	$\dots \omega$
	...	
	}	
	$r_{\leftarrow \dots}$	$f01$
	...	
∇		

If we could encompass several lines with a function expression then perhaps we could do the same with a display form of a multi-dimensional array to be evaluated during the tokenization of the containing function. This could make all arrays editable within the function editor and eliminate the need to store global constants along with the code.

A conforming extension

It happens that no APL expression can start with an opening bracket. In other words an opening bracket cannot immediately follow a left arrow, an opening bracket, brace or parenthesis or a line-end.

Also, at least before the advent of *dfns*, it was not possible to have line-ends within matching brackets or parentheses. The ability to code a multi-line *dfn* between parentheses or index or axis brackets partially lifts that restriction. Still, the line-end cannot be directly between them; it must be between braces as well.

These two facts, or the reversal of the one and the relaxation of the other, make possible a syntax that would be a natural and even familiar notation to all APLers.

Dyalog's experimental interpreter, *APLSharp*, permitted line-ends between parentheses, calling what was between them an *expression* whose value was that of the last expression in the list. What follows might appear similar but here the value of a parenthesised or bracketed expression containing line-ends will be the result of evaluating and joining all of them *in some way* so that all play an equal part in the result. An extension of vector notation, if you will.

The two two-dimensional arrays that display as

$\boxed{\text{zero}}$ $\boxed{0 \ 1 \ 2 \ 3 \ 4}$

```
|one | and | 5 6 7 8 9|
|two |    |10 11 12 13 14|
|three|   |15 16 17 18 19|
└───┬───┘
```

could simply be defined in code as

```
|... |
|[2] T-['zero' |
|[3]   'one' | and |[7]   5 6 7 8 9 |
|[4]   'two' |      |[8]  10 11 12 13 14 |
|[5]   'three'] |    |[9]  15 16 17 18 19] |
|... |
└───┬───┘
```

Brackets will do a task analogous to parentheses but where the latter are used to group items adding *depth*, the former will add *rank*, with each new row of the representation indicating a new cell in the data. And there is no reason not to extend this such that between brackets further brackets will introduce another dimension in the data. Thus, where

```
| d-((('these' 'seven' 'words') ('form' 'a text' 'array')) |
└──────────────────────────────────────────────────────────┘
```

gives us a depth-three, two-item list of three-item lists of strings,

```
| r-[['these' |
|     'seven' |
|     'words'] |
| ['form' |
|     'a text' |
|     'array']] |
└────────────────┘
```

gives us a simple, two-plane, three-row, six-column, three-dimensional array.

It is worth mentioning here that there is a significant number of APLers who would happily see index and axis brackets removed from the language. The argument is that a pair of brackets does not denote either a function or an operator but it selects and amends data as if it were one or other of them; it is thus an interloper in the language. The arrival of the *index function* was

welcomed because it dispensed with the need for index brackets but it came with the disappointment that yet another use of axis brackets was needed to make it workable. The subsequent addition of the *rank operator* may finally lay this anomaly to rest. I claim that the introduction of brackets as *notation* is *not* an extension of it but rather restores the bracket to its rightful place along with parentheses, braces and quotes as *punctuation*.

Some use of a bracketed array notation *could* lead to slight if unnecessary confusion with both index and axis specification.

In expression `a[...]`, the bracketed part is unambiguously an index if `a` is an array and an axis if `a` is a function or operator, that is if axis can *ever* be unambiguous.

In expression `a([...])`, the parenthesis is unambiguously an array specification because parentheses are not permitted around index or axis brackets. The whole expression is a function call if `a` is a function and a two item list if `a` is an array.

The notation extends easily to nested data. One particular common type of static array is the table containing columns of numbers and/or strings. They are the devil to edit. Many *Dyalog* users will have seen the array `DRC.ErrorTable` that contains all the error numbers, codes and descriptions for Conga, *Dyalog's* remote communicator. The first few rows and a later one look like this

0	SUCCESS	
100	TIMEOUT	
1000	ERR_LOAD_DLL	
1001	ERR_LENGTH	
1104	ERR_SEND	/* Could not send data*/

display like this

0	SUCCESS	
100	TIMEOUT	
1000	ERR_LOAD_DLL	


```
|xxxx      four hex digits (0-9, A-F, a-f) |
|          #0023 encodes the hash (pound) sign |
|_____
```

Diamonds thus fulfil two roles. At the same level of punctuation-nesting: within brackets they delimit cells; within parentheses they delimit items in a list. Thus `[...◇...]` is an array of two major cells, while `[ (...◇... ) ]` is a list of two items.

Within a list the above definition encompasses the full panoply of vector notation but also the restriction such that a vector of any depth or length can be defined, *perhaps excepting one of a single item or a nested empty list*.

Within a multi-dimensional array the major cells can be further delimited by brackets, the rank of the array being one more than the highest rank of any of the cells to which all are implicitly raised.

```
| a-[ [... ◇ ... ] ◇ [... ◇ ... ] ] |
|_____
```

Note that the definition precludes both function definition and execution. This is deliberate as the notation is intended to be an extension of vector notation which also does not involve function calls. Another reason is the proposed equivalence of the multi-line embedded array definition and its serialized counterpart. Including function calls in serialised data would certainly be considered a security issue.

Within any definition only punctuation `[ ( ◇ ) ]`, numbers and white-space are allowed unquoted while most typeable characters can be included between quotes (with `'` itself doubled) and an escaping protocol is used for non-typeable characters. Any unicode character can be encoded as the *escape character* followed by four hex digits (0-9, A-F, a-f) that encode the character's code-point. I have chosen to use `#` as it is typeable but perhaps uncommon in data; another could be chosen but would have to be standard across all implementations. The escape character *must* be encoded in this way when it represents the character itself. `#` would be `#0023`, carriage return `#000D`, line feed `#000A` and the White Queen ♁ `#2655`.

In most presently implemented APLs a diamond is equivalent to a line-end so, in a reversal of the conceptual leap earlier, where we went from a multi-line approximation to a linear definition, the above syntax permits the array definition to be spread over a number of lines. And as all lines in an APL function can be commented then so can our array definition when embedded over a number of lines in a function or script.

Limitations

Normal vector notation provides no facility to produce an enclosed scalar or a zero or one item vector, enclosed or otherwise. This restriction *could* be extended to array notation but equally it could be avoided. Normal APL permits blank lines and contiguous diamonds in functions. They are not executed and produce no results. Contiguous diamonds in array notation should follow this pattern and produce no part of the output. Nevertheless there is no reason *not* to differentiate between `[0 1 2]` and `[◊0 1 2]`. Although the diamond in the second case is ostensibly redundant it is apparent that whereas the first is intended to be merely a vector the second is clearly expected to produce a two dimensional result. What should its shape be? We have the choice between `1 3` and `3 1`. Again, it is clear that `[0◊1◊2]` is expected to produce a one column array albeit that its items are strictly scalar. Allowing this leaves our `[◊0 1 2]` to represent a one row matrix. Similar arguments can be used to define arrays of other ranks with dimensions of one or zero.

Conclusion

The need for such a notation and the desirability of its being defined to be cross-platform is unquestionable. If a round-trip is desirable, as I believe it is, then the above limitations need to be overcome. But they will require more than one person's imagination. I believe the nested bracket approach could be the simplest and most versatile for multi-dimensional data, that outlined here possibly forming a basis for discussion. Given the power of vector notation APL needs very little enhancement to make it work. Some of the details here might be questionable and could undoubtedly be bettered.

A collaborative effort should be made to come to an agreed design with an eye on extensibility and forward compatibility such that providers could add their own enhancements.

Appendix A - dictionaries

In all the above I have been referring to multi-dimensional and nested data. Dictionaries, variously known as associative arrays, objects, maps, key-value pairs, namespaces &c. might be considered worthy of their own notation. At least one supplier has implemented namespaces that can contain a set of named arrays and several have implemented object oriented features in which an instance of a class with a number of fields or properties could qualify.

Where no special provision is made for them in an implementation then any current use must necessarily be represented as an array so an encoder would naturally encode it as such.

JSON objects use a colon `:` to join and separate the key and value of each pair and a comma `,` to separate the pairs from each other. A natural choice for minor separator in an

APL implementation would be the left arrow $\leftarrow$ while the pairs would be separated from one another by diamonds. But *JSON*'s use of braces to distinguish objects from arrays is almost redundant as the presence of the colon would be sufficient except for the empty object that contains no key-value pair and therefore no colon. An arbitrary decision could be made to include a single left arrow merely to distinguish an empty dictionary $[\leftarrow]$ from any other empty array.

What data structure a *decoder* would generate from the notation would be implementation specific as would the array characteristics which would prompt the *encoder* to recognise candidates for encoding in this way.

The implementing of dictionaries along these lines would require the addition of a few more items to the definition:

dictionary	[$\leftarrow$]
	[pairs]
pairs	pair
	pair $\diamond$...
pair	key $\leftarrow$ value
key	string

and we must add one more character to the list of permitted unquoted characters giving $[\diamond\leftarrow]$, a total of six.

Appendix B - an experimental model

For the purposes of the proposal I have implemented a set of methods that simulate the action that the parser itself would undertake in a native implementation.

In present APLs the proposed array syntax would engender a syntax error so we have to *trick* the parser into allowing us to define the array in code without having to quote or comment it. We wrap the array definition in a *dfn* and pass it as operand to an operator that extracts and analyses the definition and returns the array without running the code. It results in an indented display not quite as indicative as that above but in which only the syntax colouring indicates anything in any way abnormal.

Methods

ArrayToCode is for data embedding.

Given any APL array, *ArrayToCode* returns the derivation of a function call to embed in your own code where it is needed.

```
⍣a←'zero' 'one' 'two',∘°,./1∘3 2

┌──────────┐
│ ┌──┐ ┌──┐ ┌──┐ │
│ │zero│ │0 0│ │0 1│ │
│ └──┐ └──┐ └──┐ │
│   '⋈' '⋈' '⋈' │
│ ┌──┐ ┌──┐ ┌──┐ │
│ │one │ │1 0│ │1 1│ │
│ └──┐ └──┐ └──┐ │
│   '⋈' '⋈' '⋈' │
│ ┌──┐ ┌──┐ ┌──┐ │
│ │two │ │2 0│ │2 1│ │
│ └──┐ └──┐ └──┐ │
│   '⋈' '⋈' '⋈' │
└──────────┘

#.#.naples.ArrayToCode a

┌──────────┐
│ { ⍣ edit indented rows only │
│   ['zero'(0 0)(0 1) │
│   'one'(1 0)(1 1) │
│   'two'(2 0)(2 1)] │
│ }#.#.naples.CodeToArray 0 │
└──────────┘
```

Once there, it can be edited as a part of the function or script while the operator *CodeToArray* will return the edited array the next time you run your code. Perhaps a native implementation, which would contain only the middle three indented lines above, would recreate the array immediately on fixing the edited code.

APLToSerial & *SerialToAPL* are for serialization and de-serialization.

Given any APL array, *APLToSerial* returns a simple text string suitable for transmission and independent of data storage implementation considerations while *SerialToAPL* will reconstitute the original array at the other end.

```
⍣s←#.#.naples.APLToSerial a

┌──────────┐
│ ['zero'(0 0)(0 1)∘'one'(1 0)(1 1)∘'two'(2 0)(2 1)] │
└──────────┘

a = #.#.naples.SerialToAPL s

1
```

In a native implementation *CodeToArray* and *SerialToAPL* would be redundant as the

notation would be a part of APL itself and as such, executable code while the format primitive could be enhanced to return either of the forms produced above by *ArrayToCode* and *APLToSerial*.

Introduction

Case Study

$$\uparrow \{ (0^\circ, +, \circ 0) * \omega, 1 \}^{\circ} \circ 1$$

This expression is a 2-train (called an Atop), a function made of 2 functions juxtaposed.

$\uparrow$	$\{(0^\circ, +, \circ 0) * \omega, 1\}'' \circ \mathbf{t}$
------------	--

A train is a series of functions lined up one after the other IN ISOLATION as in $(+\times!-)$, a train of 5 functions. The functions can be primitives, derived, composed, Dfns, other trains and even user defined traditional functions but they must be in isolation. In isolation means the functions are isolated from data. In $(+\times!-)\ 5$ the 5-function train is isolated from the 5, whereas in $+\times!-\ 5$ it is not.

Let's see why this expression is a train

49

1. The first symbol is *Iota*, the Index generator function.
2. The second symbol is *Jot*, the *Compose* operator. *Compose* accepts two operands, each may be a function or an array.
3. The third symbol is *Dieresis*, another operator named *Each*, which means the left operand to *Compose* is not an array. It is a derived function, made of another function which must be bound to *Each* for *Compose* to work.
4. The fourth symbol is right brace `}` which marks the end of a Dfn. Scanning left until we reach its matching `{` we get the function `{ (0°,+,°0) *ω,1 } ° ° 1`
5. The next symbol is a function, `↑`, and it is not bound to `{` and the entire line, except for the leftmost character *Up Arrow*, is the rightmost function of the Atop.

The only symbol left, *Up Arrow* `↑`, is the other function and this line constitutes the definition of a two-train function, an Atop. This can be seen as

```
P1← ↑
P2← { (0°,+,°0) *ω,1 } ° ° 1
P ← P1 P2
```

OK, but what does it do?

Let's have a better look at `P2`. This is a Dfn bound to *Each* as left operand to *Compose* and with *Iota* as the right operand. `P2`'s argument (`n`) is given to *Iota*, generating a series of `n` integers each of which is fed to the Dfn one by one (because of *Each*) making effectively `n` separate calls to the Dfn.

Let's have a look at that Dfn in more details:

Reading from the right inside the Dfn `' , '` is the first function encountered but the object to its left is bound to the operator *Power* (`*`) so `' , '` is monadic, it is the *Ravel* function. It turns the scalar `1` to its right into a 1-element vector containing the number `1`.

Power only accepts functions as left operand so what's in parentheses to its left better be a function or we'll get a SYNTAX ERROR.

What's within parentheses and `omega` constitute the function applied to `,1`. In the Dfn `{ (0°,+,°0) *ω ,1 }` the function `(0°,+,°0)` is applied *omega* times to `,1`. We could rewrite `P2` like this:

```
F← (0°,+,°0) ° P2← { F*ω ,1 } ° ° 1
```

Let's have a look at `F` more closely. It's a Fork. This is the first function:

```
[ ]
```

$$0^\circ, + \mid ,^\circ 0 \mid$$

it is ‘,’ bound to 0 via the *Compose* operator. It is a function that concatenates 0 **after** its argument.

Operators bind their arguments **before** the functions can make up trains. This is a rule which did not exist before because trains of functions did not exist then.

It is preceded by another function *Plus* which is not bound to anything:

$$0^\circ, \mid + \mid ,^\circ 0$$

This in turn is preceded by another function made of 0 bound to ‘,’ via the *Compose* operator. It is a function that concatenates 0 **before** its argument.

$$\mid 0^\circ, \mid + ,^\circ 0$$

The function is a three-function train. In this case it is a Fork.

$$F \mid 0^\circ, \mid + ,^\circ 0 \mid$$

Applying this function once to , 1

$$(0^\circ, + ,^\circ 0) , 1$$

1 1

Is the same as

$$(0^\circ, , 1) + (,^\circ 0 , 1)$$

1 1

or

$$0 \ 1 + 1 \ 0$$

1 1

Applying this function twice to , 1

$$F \ F , 1 \quad \text{or} \quad F \circ F , 1$$

Is the same as $F \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, or the same as

$$(0^\circ, \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}) + (\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, 0^\circ)$$

or

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} + \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 2 & 1 & 1 \end{pmatrix}$$

Applying this function thrice to $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ is

$$F \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} F \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} F \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \text{ or } F^3 \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

Is the same as $F \begin{pmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \end{pmatrix}$, or the same as

$$(0^\circ, \begin{pmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \end{pmatrix}) + (\begin{pmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \end{pmatrix}, 0^\circ)$$

or

$$\begin{pmatrix} 0 & 1 & 2 & 1 & 1 & 1 & 2 & 1 & 0 \\ 1 & 3 & 3 & 1 & 1 & 3 & 3 & 1 & 0 \end{pmatrix}$$

Going back to the original function:

$$P \leftarrow \uparrow \{ (0^\circ, \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}) \} \circ \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

This is a train of two functions. Which again can be seen as

$$P \leftarrow P_1 P_2$$

P_2 applied to an integer e.g. 4, **in origin 0**, is

$$\{ (0^\circ, \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}) \} \circ \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} 4$$

or

$$\{ (0^\circ, \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}) \} \circ \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} 0 \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} 3$$

or

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \end{pmatrix} \begin{pmatrix} 1 & 3 & 3 & 1 \\ 1 & 3 & 3 & 1 \end{pmatrix}$$

Applying $P_1(\uparrow)$, **with ML 1**, to this array results in

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

```
1 2 1 0
1 3 3 1
```

And going back to the original function:

```
P←↑{(0∘,+,∘0)×ω,1}∘∘t
P 4
1 0 0 0
1 1 0 0
1 2 1 0
1 3 3 1
```

It generates the Pascal Triangle.

Conclusion

This kind of code will appear more and more often in the future. Even if we do not intend to use all the new features yet we need to be able to read them and understand what it means. APL is evolving all the time, just like any other language. English, Russian, Mandarin weren't spoken the same way 300 years ago and they won't be the same 300 years hence. APL wasn't the same 30 years ago; it WILL be different in the future. We need to adapt.

Note

You can "see" this article in a video here: youtu.be/kt4lMZbn-so or search for APLtrainer in YouTube to find it.

Sixteen APL Amuse-Bouches

by Roger K.W. Hui

The amuse-bouche is the best way ... to express big ideas in small bites.

– Jean-Georges Vongerichten, 2002

Such were the vagaries of what may have been the most powerful computer language ever hatched by a Canadian. Yes, I know that James Gosling, an alumnus of the University of Calgary, was the “father of Java.” He and I have talked about it. And yes, there is now a game called “Perl Golf” in which you try to do the most work in that scripting language with the fewest number of (key-) strokes. Java and Perl are good and useful and popular, but APL was ... beautiful.

– Tom Keenan, Business Edge, 2004

0. $(x>0) - (x<0)$

Signum of x for real x , assuming the values 1, 0, or -1 according as x is strictly positive, 0, or strictly negative. The phrase dates from the earliest days of APL, being found in section 1.4 of *A Programming Language* [1]. Three decades later, the idea was adopted by Knuth [2], who wrote that “Iverson’s convention” led to improvements in exposition and technique. Some writers call these “data-driven conditionals” [3]; during a discussion between Phil Last, John Scholes, and myself it was suggested that they be called “array logic” [4].

The usefulness of array logic in APL is due to the following:

- Boolean functions have value 0 and 1 rather than true and false [5, #implementers2].
- Functions apply to entire arrays, as in, for example, $+/\times>100$ to compute the number of elements of vector x greater than 100 [5, #Maple].
- A simple function precedence (“right to left”).

Amuse-bouches 0, 2, 4, D, and E use array logic.

1. $(+\div\neq) \times$

The average of x . The construct $+\div\neq$ is a fork, a 3-train. Using $+\div$ and $\neq$ instead of the more familiar $+/$ and ρ is superior for reasons described in [6].

2. $\times\boxtimes x=x$

A shortest phrase for the average of vector x . Note that $+/ \div \rho \neq$ are not used. The

idea originated with Timo Seppälä at APL82 [7]. To see why it works, start with the definition

$$y \boxtimes x \leftarrow (\boxtimes (+ \boxtimes x) + . \times x) + . \times (+ \boxtimes x) + . \times y$$

which defines the rectangular case in terms of the square case.

$x \boxtimes x = x$	
$x \boxtimes w$	$w \leftarrow x = x$
$(\boxtimes (+ \boxtimes w) + . \times w) + . \times (+ \boxtimes w) + . \times x$	definition of $\boxtimes$
$(\boxtimes (\boxtimes w) + . \times w) + . \times (\boxtimes w) + . \times x$	w is non-complex
$(\div \# x) + . \times w + . \times x$	w is $(\# x) \rho 1$; $w + . \times w$ is $\# x$
$(\div \# x) + . \times \neq x$	$((\# x) \rho 1) + . \times x \leftarrow + \neq x$
$(\div \# x) \times + \neq x$	LHS and RHS of $+ . \times$ are scalars
$(+ \neq x) \div \# x$	

Alternatively, $y \boxtimes x$ computes a linear regression for y wherein the constant term is the mean of y ; that is, the best least square “fit” of a vector by a single number, is its mean.

3. $x \iota x$

Index-of with both arguments being the same, an instance of a “selfie” [8]. $x \iota x$ are like ID numbers; questions of identity on x can often be answered more efficiently on $x \iota x$ than on x itself.

The $x \iota x$ selfie occurs in several important and useful computations:

$((\iota \# x) = x \iota x) \neq x$	nub (unique)
$(x \iota x) \circ . = (x \iota y)$	efficient computation for $x \wedge . = \boxtimes y$ for matrices x and y [9]
$(x \iota x) \circ . = (x \iota y)$	efficient computation for $x \circ . \equiv y$ for non-simple vector x [10]
$(\boxtimes \uparrow x \iota \iota x) \iota (\boxtimes \uparrow x \iota \iota y)$	inverted table index-of [11]
$(x \iota x) \text{ f} \boxplus y$	equivalent to $x \text{ f} \boxplus y$

4. ' . □ ' [$x^\circ . > \iota \uparrow / x$]

A bar chart of non-negative integer vector x .

```
x ← 3 1 4 1 5 9
' . □ ' [x° . > ι ↑ / x]
□ □ □ . . . . .
□ . . . . .
□ □ □ □ . . . .
□ . . . . .
□ □ □ □ □ . . .
□ □ □ □ □ □ □ □
```

I encountered this phrase when I first learned APL in the mid 1970s. By then, it was part of the APL milieu, found in STATPACK [12] and APL in Exposition [13, p.13]. (I believe I saw it in the former.) The phrase brought home to me that indices can be entire arrays.

5. $+\backslash 1 \bar{1} 0 [' () ' i x]$

Depth of parentheses nesting.

```
x←'ω (( ∇ < S ) , = S , ( ∇ > S ) ) ω □ ~ ? # ω '
x , [ ¯ 0.5 ] +\1 1̄ 0 [' () ' i x]
ω ( ( ∇ < S ) , = S , ( ∇ > S ) ) ω □ ~ ? # ω
0 1 2 2 2 2 1 1 1 1 1 2 2 2 2 1 0 0 0 0 0 0
```

The phrase has an ancient and storied history. As recounted by Alan Perlis [14]:

I was at a meeting in Newcastle, England, where I'd been invited to give a talk, as had Don Knuth of Stanford, Ken Iverson from IBM, and a few others as well. I was sitting in the audience sandwiched between two very esteemed people in computer science and computing — Fritz Bauer, who runs computing in Bavaria from his headquarters in Munich, and Edsger Dijkstra, who runs computing all over the world from his headquarters in Holland.

Ken was showing some slides — and one of his slides had something on it that I was later to learn was an APL one-liner. And he tossed this off as an example of the expressiveness of the APL notation. I believe the one-liner was one of the standard ones for indicating the nesting level of the parentheses in an algebraic expression. But the one-liner was very short — ten characters, something like that — and having been involved with programming things like that for a long time and realizing that it took a reasonable amount of code to do, I looked at it and said, “My God, there must be something in this language.” Bauer, on my left, didn't see that. What he saw or heard was Ken's remark that APL is an extremely appropriate language for teaching algebra, and he muttered under his breath to me, in words I will never forget, “As long as I am alive, APL will never be used in Munich.” And Dijkstra, who was sitting on my other side, leaned toward Bauer and said, “Nor in Holland.” The three of us were listening to the same lecture, but we obviously heard different things.

6. $x[\Delta \nabla (p x) p 0 \ 1]$

Perfect shuffle, a special case of a solution to the more general mesh problem described by Ken Iverson [1, §1.9] and solved by Luthor Woodrum [15]. The phrase was found on the back of I.P. Sharp T-shirts in the 1970s and 1980s.

```
x←'ABCDEFGHJKLMNOPQ'
```

```
x[4*(p0-1)]
IAJBKCLDMENFOGPHQ
```

The phrase $4 \cdot (p_0 - 1)$ is a permutation of length n . Its order is the minimum number of times a perfect shuffle has to be applied to yield the original argument [16, 17]. The sequence of these orders is A024222 in the indispensable On-line Encyclopedia of Integer Sequences [18].

7. $Q_{-}\{1 \geq w : w \diamond S_{-}\{a \neq a \ a \ a \ w\} \diamond w((\nabla < S), =S, (\nabla > S)) \diamond w \approx ? \neq w\}$

Quicksort [19].

The “pivot” $w \approx ? \neq w$ is randomly chosen. $((\nabla < S), =S, (\nabla > S))$ is a fork, selecting the parts of w which are less than the pivot, equal to the pivot, and greater than the pivot. The function is recursively applied to the first and the last of these three parts.

```
Q-x--?13p20
2 2 7 10 10 11 3 10 14 5 9 1 16
Q x
1 2 2 3 5 7 9 10 10 10 11 14 16
```

The variant Q_1 obtains by enclosing each of the three parts. Its result exhibits an interesting structure. The middle item of each triplet is the value of the pivot at each recursion. Since the pivot is randomly chosen the result of Q_1 can be different on the same argument.

$Q_1_{-}\{1 \geq w : w \diamond S_{-}\{a \neq a \ a \ a \ w\} \diamond w((C \circ \nabla < S), (C = S), (C \circ \nabla > S)) \diamond w \approx ? \neq w\}$

$Q_1 \ x$

$Q_1 \ x$



8. $P \leftarrow \uparrow \{ (0^\circ, +, \circ 0) * \omega, 1 \} \circ \iota$

Pascal’s triangle [20].

```
P 10
1 0 0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 0 0 0
1 2 1 0 0 0 0 0 0 0
1 3 3 1 0 0 0 0 0 0
1 4 6 4 1 0 0 0 0 0
1 5 10 10 5 1 0 0 0 0
1 6 15 20 15 6 1 0 0 0
1 7 21 35 35 21 7 1 0 0
1 8 28 56 70 56 28 8 1 0
1 9 36 84 126 126 84 36 9 1

10 ↑ (,°.÷110) {+/\ω}⌈,P 10
1 1 2 3 5 8 13 21 34 55
```

The expression $(0, x) + (x, 0)$ or its commute, which generates the next set of binomial coefficients, is present in the document that introduced *APL360* in 1967 [21, Fig.1] and the one that introduced *J* in 1990 [22, Gc&Gd]; in *Elementary Functions: An Algorithmic Treatment* in 1966 [23, p.69], in *APL360 User’s Manual* in 1968 [24, A.5], in *Algebra: An Algorithmic Treatment* in 1972 [25, p.141], in *Introducing APL to Teachers* in 1972 [26, p.22], in *An Introduction to APL for Scientists and Engineers* in 1973 [27, p.19], in *Elementary Analysis* in 1976 [28, ex.1.68], in *Programming Style in APL* in 1978 [29, §6], in *Notation as a Tool of Thought* in 1980 [30, A.3], in *A Dictionary of APL* in 1987 [31, m∇n], and probably others.

$°.!\approx 1n$ also computes Pascal’s triangle of order n . It is shorter, but is less interesting algorithmically and historically.

9. $+^\circ \div \backslash np1$

Convergen^ts to the golden mean by continued fractions [32].

```
+^\div\16p1
```

```

1 2 1.5 1.66667 1.6 1.625 1.61538 1.61905 1.61765 1.61818
      1.61798 1.61806 1.61803 1.61804 1.61803 1.61803
      1  ^  +÷÷\16p1
1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597

```

A. $(2 \div \omega \vdash + \div) \setminus \text{np}x$

Convergenets to the square root of x by Newton's method [33].

The operand of $\setminus$ is a fork, itself a composition of two forks $(2 \div \omega (\vdash \div))$ and equivalent to $\{0.5 \times \omega + a \div \omega\}$.

```

□-t- (2 ÷ ω ⊢ + ÷) \ 7p2
2 1.5 1.41667 1.41422 1.41421 1.41421 1.41421
      (2*0.5) - t
-0.585786 -0.0857864 -0.0024531 -0.0000021239 -1.59472e-12
      2.22045e-16 2.22045e-16

```

Since the phrase uses only rational operations, it yields rational convergenets when applied to rational arguments. The following result is computed in J:

```

2 3r2 17r12 577r408 665857r470832 886731088897r627013566048
      1572584048032918633353217r1111984844349868137938112

```

B. $x (+ \setminus x^{\omega-1}) * 1 \ 2 \vdash y$

Inner product using row-at-a-time operation. For example:

```

x-?3 4p100
y-?4 5p100
(x+.x y) ≡ x (+ \ x^ω-1) * 1 2 ⊢ y
1

```

The main advantages of row-at-a-time over the conventional row-by-column are better cache utilization and ability to exploit sparsity [34] and [35, #IC2013]. Note: The Dyalog implementation of $+. \times$ is already row-at-a-time, so there is no need to resort to the circumlocution.

C. $\text{r-}, \text{t} \vdash \diamond (\text{r G}) \equiv \text{r} \circ . \{a[\omega]\}^{\omega} \downarrow \text{r G}$

The finite case of Cayley's theorem [36, §4.4]: every group G is isomorphic to a subgroup of the group of permutations. For example:

```

□-T-2 2°p" (4p2) °T" 9 6 7 11 13 14

```

1	0	0	1	0	1	0	1	1	1
0	1	1	0	1	1	1	0	1	0
$\square \leftarrow G \circ \circ, (\neq, \wedge) \approx T$									
1	0	0	1	0	1	0	1	1	1
0	1	1	0	1	1	1	0	1	0
0	1	1	0	1	1	1	0	1	0
1	0	0	1	0	1	0	1	1	1
0	1	1	0	1	1	1	0	1	0
1	1	1	1	1	0	0	1	0	1
1	0	0	1	0	1	0	1	1	1
1	1	1	1	1	0	0	1	0	1
1	1	1	1	1	0	0	1	0	1
0	1	1	0	1	1	1	0	1	0
1	1	1	1	1	0	0	1	0	1
1	0	0	1	0	1	0	1	1	1

T are the non-singular 2-by-2 boolean matrices, and G is its group table under boolean matrix multiplication. The function $r \leftarrow, i \vdash$ relabels a group table according to its index in its ravel (cf. $\times i \times$); such relabelling, like displaying in a different font, does not affect group-theoretic properties.

$r \leftarrow, i \vdash$									
$r \ G$									
0	1	2	3	4	5				
1	0	4	5	2	3				
2	3	5	4	1	0				
3	2	1	0	5	4				
4	5	3	2	0	1				
5	4	0	1	3	2				

The rows of $r \ G$ are permutations, and the relabeled version of its group table as permutations is the same as the relabelling of the original group:

$(r \ G) \equiv r \circ \{a[\omega]\} \approx i \ r \ G$

D. Symmetries of the square [36, §4.4] and [37].

A permutation can be represented as an integer vector or as a square boolean matrix with exactly one 1 in each row and each column. Functions `pmv1 ← °. = 1 ° #` and `mp1 ← 1 ° 1 ° 1` transform from one to the other.

$\vdash \Delta \nabla \Phi$ on permutations produce permutation results. They can be identified with $\vdash \boxtimes \boxtimes \boxtimes \boxtimes$ on square matrices.

61

$$\begin{aligned} 1 & \quad (\& \text{ pm } p) \equiv \text{ pm } \& p \\ 1 & \quad (\& \text{ pm } p) \equiv \text{ pm } \nabla p \\ 1 & \quad (\ominus \text{ pm } p) \equiv \text{ pm } \phi p \\ 1 & \end{aligned}$$

Since $\vdash_{\mathbb{Q}} \ominus \ominus$ on matrices are transpositions of the square, then so are $\vdash_{\mathbb{A}} \Psi \Phi$ on permutations. The group table $\mathbb{D8}$ is a compact presentation of numerous identities involving $\vdash_{\mathbb{A}} \Psi \Phi$ on permutations — $\mathbb{D8}[i;0]$ composed with $\mathbb{D8}[0;j]$ is $\mathbb{D8}[i;j]$. For example:

i	j	D8[i;0]	D8[0;j]	D8[i;j]
5	5	Δ	Δ	$\vdash$
2	2	$\nabla \nabla$	$\nabla \nabla$	$\vdash$
1	2	∇	$\nabla \nabla$	$\Delta \Phi$
1	5	∇	Δ	Φ
1	6	∇	$\Delta \nabla$	Δ

That is, the 2 2 entry asserts that $\forall \forall \forall \forall p \leftrightarrow \vdash p$; the 1 5 entry asserts that $\forall \forall p \leftrightarrow \phi p$; and so on. The veracity of these assertions can be checked by $(\underline{\phi} \text{ "D8, " 'p' }) \equiv \underline{\phi} \text{ " (} \circ, \sim \text{ "D8) , " 'p' } \vdash p \text{ - ?n .}$

Finally, per the previous section, $(r \text{ D8}) \equiv r \circ \{ \alpha[\omega] \} \sim \downarrow r \text{ D8}$.

E. $\mathbf{I} \leftarrow \{ (\rho \omega) \rho (\neq \alpha) \downarrow \mathbf{i} \vdash \mathbf{i} [\mathbf{i}] \leftarrow + \setminus (\neq \alpha) > \mathbf{i} \leftarrow \nabla \alpha, \omega \}$

Interval index: α is a sorted (strictly ascending) vector; $\alpha \perp \omega[i]$ is the least j such that $\alpha[j]$ follows $\omega[i]$ in the ordering, or $\# \alpha$ if $\omega[i]$ follows $\perp_1 \alpha$ in the ordering or if α is empty. The function is named `classify` in [36, §1.2] and is a limited version of the $\perp$ -primitive in J [38].

t , [0.5] 1.5 1 4 I t-2+18
 -2 1 0 1 2 3 4 5
 0 1 1 2 2 2 3 3
 1.5 1 4 I 2 2p5 3.1 4 -20
 3 2
 3 0
 'aeiou' I 'deipnosophist'
 1 2 3 4 3 4 4 4 4 2 3 4 4

The function can be expressed more concisely and more clearly by use of the `merge` operator, discussed in [36, 38, 39] and whimsically denoted by $\rightarrow$:

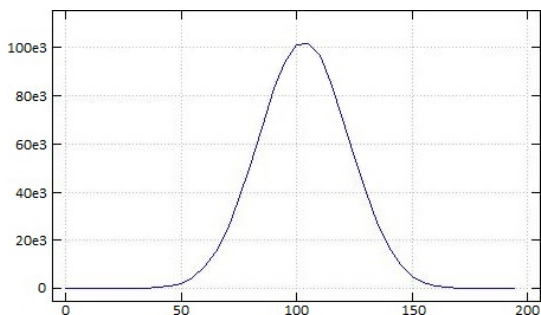
```
{ (p0) p (#a) ↓ (+\i<#a) (i→) i←4a, w }
```

F. `⌈1+{#w}⌈(t41), (5×t40) I+÷?10 1e6p21`

An illustration of the central limit theorem, that the sum of independent random variables converges to the normal distribution [38, I.].

```
t←⌈1+{#w}⌈(t41), (5×t40) I+÷?10 1e6p21
p t
41
5 8p t
0 0 0 0 0 13 28 90
317 894 2095 4574 8671 15001 24338 36728
51254 66804 82787 93943 101045 101752 96510 85281
70418 54506 39802 27267 16964 9764 5031 2467
1059 422 136 32 6 1 0 0
```

`t` counts the number of occurrences in the intervals with endpoints `5×t40`, of `1e6` samples from the sum of ten repetitions of `?21`. A plot of `t` :



The derived function `{#w}⌈x` counts the number of occurrences of each unique cell of `x`. On 1-byte integers the computation takes only 1.4 times as long as required for the apparently simpler problem `⌈/x`. Thinking about an obvious C implementation for `{#w}⌈x` leads to a non-obvious implementation for `⌈/x` [8, §16] faster than

```
max=*x++; for (i=1; i<n; ++i) { if (max<*x) max=*x; ++x; }
```

The fine print. The exposition requires: Dyalog APL Version 14.0 or later; `Ⓜio←0`; `Ⓜml←1`; `]box on`.

References

1. Iverson, K.E., “A Programming Language”, Wiley, 1962-05:
www.jsoftware.com/papers/APL.htm
2. Knuth, Donald E., “Two Notes on Notation”, American Mathematical Monthly, Volume 99, Number 5, 1992-05-01:
arxiv.org/PS_cache/math/pdf/9205/9205211v1.pdf
3. Scholes, John, “Data-driven Conditionals”, Dyalog Blog, 2014-10-13:
www.dyalog.com/blog/2014/10/data-driven-conditionals-2/
4. Hui, Roger K.W., Phil Last, and John Scholes, e-mail discussion, 2014-10-18 to -20.
5. Hui, Roger K.W., editor, “Ken Iverson Quotations and Anecdotes”, 2014-10-10:
<http://keiapl.org/anec>
6. Hui, Roger K.W., “On Average”, Vector, Volume 24, Number 2&3, 2010-08:
archive.vector.org.uk/art10500270
7. Azzarello, Arlene, editor, “APL QUOTE-QUAD: The Early Years”, APL Press, Palo Alto, 1982-11:
www.jsoftware.com/papers/QQ_Early_Years.htm
8. Hui, Roger K.W., “Index-Of, A 30-Year Quest”, J Conference 2014, 2014-07-25:
www.jsoftware.com/papers/indexof/indexofscript.htm
9. Hui, Roger K.W., “Dyalog Potential v14.0 Language Features”, Dyalog User Conference, 2012-10-15:
www.dyalog.com/uploads/conference/dyalog12/presentations/D04_14.0/v14.0_Roger.zip
10. Hui, Roger K.W., “A Speed-Up Story”, Dyalog Blog, 2014-11-05:
www.dyalog.com/blog/2014/11/a-speed-up-story-2/
11. Hui, Roger K.W., “Rank & Friends”, Dyalog User Conference, 2013-10-22:
www.dyalog.com/uploads/conference/dyalog13/presentations/D08_Rank_and_Friends/
12. Smillie, Keith W., “STATPACK: An APL Statistical Package”, University of Alberta, 1968.
13. Iverson, K.E., “APL in Exposition”, IBM Corporation, 1972-01.
14. Perlis, Alan J., “Almost Perfect Artifacts Improve only in Small Ways: APL is more French than English”, APL78, 1978-03-29:
www.jsoftware.com/papers/perlis78.htm
15. Hui, Roger K.W., “An Amuse-Bouche from APL History”, 2014-10-25:
www.jsoftware.com/papers/mesh.htm
16. Thomson, Norman, “Jottings 43: A Rippling Good Yarn!”, Vector, Volume 21,

- Number 3, 2005-05:
archive.vector.org.uk/art10010570
17. Scholes, John, “Ripple”, D-Function Workspaces, 2007-03-05:
dfns.dyalog.com/n_ripple.htm
 18. OEIS, “Sequence A024222”, On-line Encyclopedia of Integer Sequences, 2014:
<https://oeis.org/A024222>
 19. Hui, Roger K.W., “Quicksort”, J Wiki Essay, 2005-09-28:
www.jsoftware.com/jwiki/Essays/Quicksort
 20. Hui, Roger K.W., “Pascal’s Triangle”, J Wiki Essay, 2005-09-28:
www.jsoftware.com/jwiki/Essays/Pascal's_Triangle
 21. Falkoff, A.D., and K.E. Iverson, “The APL360 Terminal System”, Research Report RC-1922, IBM 1967-10-16:
www.jsoftware.com/papers/APL360TerminalSystem.htm
 22. Hui, R.K.W., K.E. Iverson, E.E. McDonnell, and A.T. Whitney, “APL?”, APL90, APL Quote Quad, Volume 20, Number 4, 1990-07:
www.jsoftware.com/papers/J1990.htm
 23. Iverson, K.E., “Elementary Functions: An Algorithmic Treatment”, Science Research Associates, Inc., 1966.
 24. Falkoff, A.D. and K.E. Iverson, “APL360 User’s Manual”, IBM Corporation, 1968-08.
 25. Iverson, K.E., Algebra: “An Algorithmic Treatment”, Addison-Wesley, 1972.
 26. Iverson, K.E., “Introducing APL to Teachers”, IBM Corporation, 1972-07.
 27. Iverson, K.E., “An Introduction to APL for Scientists and Engineers”, IBM Corporation, 1973-03.
 28. Iverson, K.E., “Elementary Analysis”, APL Press, 1976.
 29. Iverson, K.E., “Programming Style in APL”, 1978 APL Users Meeting Proceedings, 1978-09-18:
www.jsoftware.com/papers/APLStyle.htm
 30. Iverson, K.E., “Notation as a Tool of Thought”, Communications of the ACM, Volume 23, Number 8, 1980-08:
www.jsoftware.com/papers/tot.htm
 31. Iverson, K.E., “A Dictionary of APL”, APL Quote Quad, Volume 18, Number 1, 1987-09:
www.jsoftware.com/papers/APLDictionary.htm
 32. Hui, Roger K.W., “Fibonacci Sequence”, J Wiki Essay, 2005-09-26.
www.jsoftware.com/jwiki/Essays/Fibonacci_Sequence
 33. Hui, Roger K.W., “Newton’s Method”, J Wiki Essay, 2005-10-15:
www.jsoftware.com/jwiki/Essays/Newton's_Method
 34. Hui, Roger K.W., “Inner Product — An Old/New Problem”, 2009-06-07:
www.jsoftware.com/papers/innerproduct/ip.htm
 35. Hui, Roger K.W., editor, “APL Quotations and Anecdotes”, 2014:
www.jsoftware.com/papers/APLQA.htm

36. Hui, Roger K.W., “Some Uses of { and }”, APL87, APL Quote Quad, Volume 17, Number 4, 1987-05:
www.jsoftware.com/papers/from.htm
37. Hui, Roger K.W., “Symmetries of the Square”, J Wiki Essay, 2005-11-07:
www.jsoftware.com/jwiki/Essays/Symmetries%20of%20the%20Square
38. Hui, Roger K.W., and K.E. Iverson, “J Introduction and Dictionary”, 2011-05-03:
www.jsoftware.com/help/dictionary/vocabul.htm
39. Scholes, John, “Merge, D-Function Workspaces”, 2014:
dfns.dyalog.com/n_merge.htm

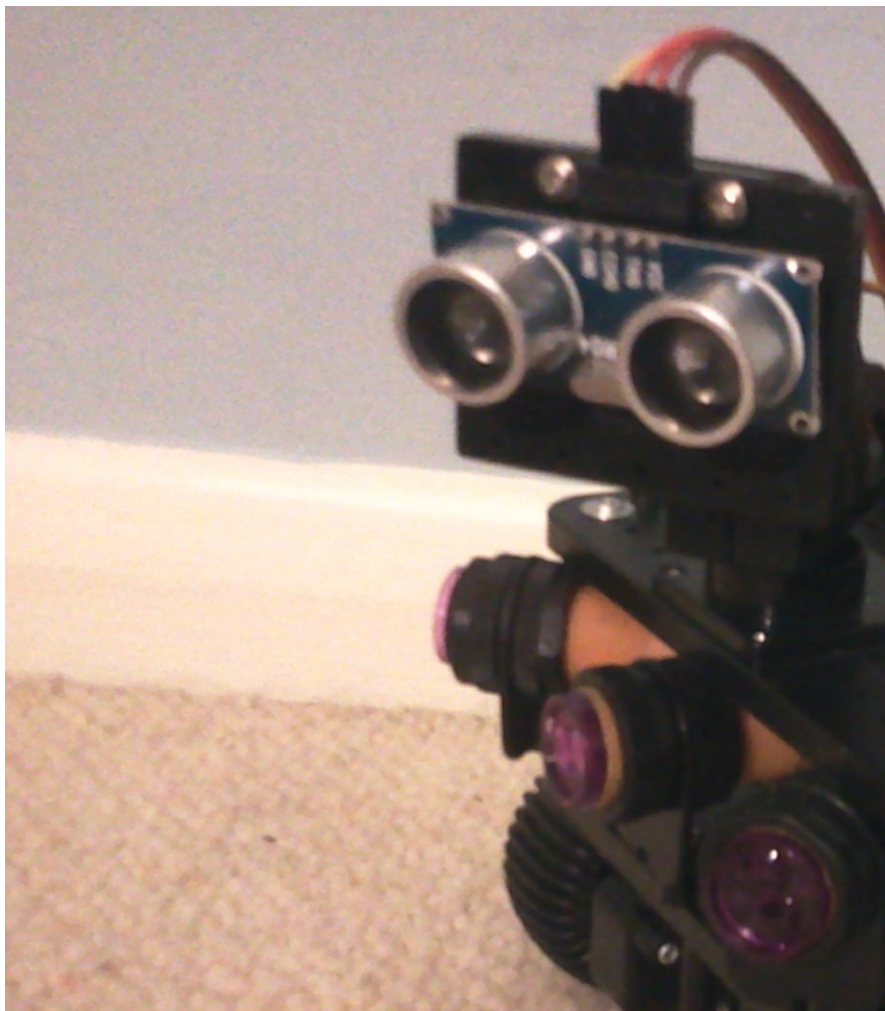
Pi, Pin and Ping

Ray Cannon

(Or How to control a Raspberry Pi robot from Dyalog APL)

Part One – The Hardware

As you properly know by now, Dyalog APL is available on the Raspberry Pi computer (running Linux), and several APLers have built *robots* controlled from Dyalog (including James and Shaquil, two of Optima's "Three blind mice", not to mention Morten). As I often work in the Optima office, I have watched James and Shaquil building their robots that they demonstrated at the Deerfield Dyalog Conference last year, and felt very jealous!





Assembled Initio Robot Kit

For my last birthday (after some not so subtle hints), I was given a 4tronics Initio[1] robot kit with all the trimmings.

So here are my thoughts and experiences with it.

Background.

I started out with no LINUX experience, and have only ever really used Dyalog under Windows. So every command needed to even copy, rename or move a file, has been a hard learning curve. As to using a computer for real time control, other than programming an Analogue Computer on my Chemical Engineering course back in the early 1970's, again, I have no experience.

To interact with the 'outside world' the Model B Raspberry Pi has many hardware connection points:

- One HDMI port;
- One Composite Video port;
- Two USB ports;
- A network port;
- An audio output (earphone) socket.
- A SD card reader
- A connector for the Raspberry Pi camera.
- And a micro-usb port for its 5V power supply.

In addition it has several *interfaces* for low level peripherals such as the General Purpose Input Outputs (GPIO).

The BCM2835 chip (the ARM chip at the heart of the Pi) has 52 GPIO ports and the Raspberry Pi has exposes 26 of them via its *GPIO header pins* (marked P1 on the Pi's circuit board). The Pi circuit board exposes other GPIOs, (via P5 and P6) but these do not have *pins* attached and require some soldering if you wish to make use of them.

These General Purpose Input Output pins connect directly onto the BCM2835 ARM chip

at the heart of the Pi. These *pins* can be at *ground* 0 Volts (LOW binary 0) or at +3.3 Volts (3V3 HIGH binary 1), and can either be set to *read* (input) or *write* (output) by software.

There are a large number of very cheap +5 Volts *logic* devices such as sensors, LEDs and relays on the market. The GPIO pins allow one to connect these devices to the Raspberry Pi, allowing the Pi to control them or gather information from them.

Who ordered the Pi and fried chips?

Directly connecting these 5V0 devices to the 3V3 pins will kill the Raspberry Pi dead.

So, one must provide an interface of some sort between the pins and the devices. At its simplest, two resistors of the right value, acting as a voltage divider, will do the trick. There are also many interface boards designed to safely connect the 5V0 devices to the 3V3 GPIO pins.

As an alternative to using the pins, many people have used an Arduino, (which is a micro-controller with number of 5v0 logic pins of its own, controlled via programmes written in C). This is the route that Morten took, and the “mice” followed when building their Robots.

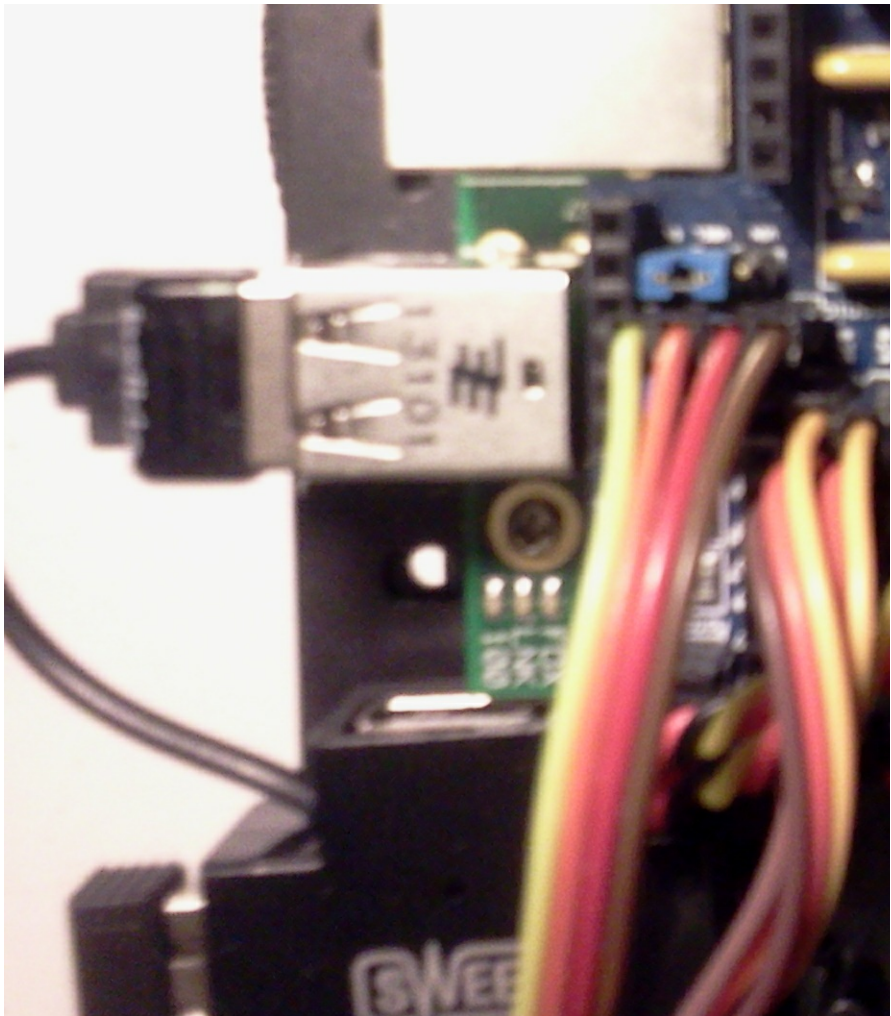
Old kit bag

In addition to the mechanical Robot parts (wheels, electric motors, gears, chassis, ‘pan and tilt’ mount with servos, battery compartment, an on-off switch), a Raspberry Pi, a 4GB SD card pre-loaded with software, 4 different types of sensors, and a WiFi dongle, my kit contained a PiRoCon (Pi Robot Controller) board that plugged directly into the Raspberry Pi’s P1 GPIO pins.

The PiRoCon board, supplies the Pi with all the power it needs, as well as *exposing* the P1 GPIO pins safely to 5V0 logic. (A bank of eight of Pi’s twenty six GPIO pins are directly available, each with its own Ground and 5V0 supply.)

It also contains a dual h-bridge *L239D* integrated circuit. This can control two electric motors (powered by the PiRoCon board or via a separate supply). It uses four pins:





PiRoCon board (blue) in-situ

- pin 7 controls forward motion on the port side
- pin 8 controls forward motion on the starboard side
- pin 9 controls backwards motion on the port side
- pin 10 controls backward motion on the starboard side

The sensors included in the kit were:

- Two Infrared Obstacle sensors:
- Two light detecting sensors (for following white lines on the floor):
- Two sensors built into the axles to provide accurate feedback of wheel movements
- One Ultrasonic Distance sensor.

Is there anybody out there?

The two InfraRed IR Detector 5V DC Switches return a logic *High* signal when an obstacle is detected within range. (The range between 3cm to 80cm is adjustable.) They have an angular beam of about 15 degrees. I later added a third putting one in the centre front pointing forward and the other two on either side, angled to give a 45 degree forward coverage. Each of these each required a GPIO logic pin (plus 5V0 power and ground) so are connected to three of the “bank of eight” mentioned above.

The Ultrasonic Distance sensor consisted of an ultrasonic transmitter and receiver. When activated, it sends out a short set of eight *pings* at 40Kz. The distance can be calculated knowing the speed of sound and measuring the time taken for any echo to be received. Currently this is mounted on a *turret* controlled by servos which allows it to be moved in two dimensions (*Pan* and *Tilt*).

In the early days of radio controlled model aeroplanes (1960’s) lightweight servos were developed for use over radio channels. To keep costs down, multiple servos were controlled on a single channel by using Pulse Width Modulation (PWM).

PMW is a *square* wave (or should that be *rectangular* wave) where the ratio of the duration of HIGH and LOW values vary. A value of On for 100% of the pulse or 0% or any value in between can be set. Thus a *digital* signal, (on or off) can be used to send an analogue *value*. On receiving a 75% PWM signal, the servo would move to its 75% position. My understanding of this, is that this can all be achieved through simple *electronic circuits* requiring no software or micro-computers.

The descendants of these servos are now smaller, lighter and much cheaper than their forbearers, but are still PWM based. My kit contained two PWM controlled servos that are used in the Pan and Tilt mechanism of the turret.

At the current time I am not using either the line following sensors nor the wheel motion sensors.

Putting it all together.

Mechanical construction of the kit was straight forward, if a little fiddly. The only tools required were cross-headed screwdriver (and a pair of scissors to open some of the plastic packs).

The Raspberry Pi was screwed onto the top deck, and the PiRoCon board simply plugged into the top of the Pi.

The (colour coded-red and black) motor wires were screwed into terminal blocks on the PiRoCon board.

The Servo control wires (brown orange yellow) were plugged on to the 5V0 GPIO pins on the PiRoCon board, as were the IR sensors wires (red black yellow).

I plugged the WiFi USB dongle (supplied) into the Raspberry Pi, and wireless keyboard and mouse USB dongle (not supplied).

Finally I went out and bought two sets of six AA rechargeable batteries.

Once the six batteries were charged, I installed them in the battery compartment, plugged the power lead into the PiRoCon, and connected a monitor up to the PI, I was ready to

turn it on.

I almost forgot to place the robot on blocks, as I had been instructed to.

First Power On

As I powered it up for the first time, all four wheels span for a second or so and stopped. (So that's why I needed the blocks.) One red LED on the PI, and a red and green LED on the PiRoCon board, and a blank screen.

I quickly discovered that I had forgotten to install the supplied SD card (with the Raspberry Pi's operating system on it) into the PI!

Power off, install the SD card, and power on again.

Again all four wheels span for a second or so and stopped, the LEDs lit up, then the green LED on the Pi lit up, then the monitor came to life, and the Raspberry Pi booted up.

The Wheels on the bus go round and round.

As supplied, the software on the SD card logged me on and straight into X. (X the GUI supplied with Linux). I started SCRATCH 4GPIO, selected the MOTOR TEST project from the menu, and ran the test. It worked, the wheels reacted to key strokes, forward back, left and right.

Next I tested the Pan and Tilt. That worked too, but juddered a lot. I saw that it was programmed to move in 5% jumps.

The ultrasonic distance sensor and IR obstacle sensors also worked.

I then reloaded the motor test, disconnected the monitor, put the robot on the floor and tried controlling it via the wireless keyboard. It all worked.

Scratch to Dyalog

Ok, Scratch (like Turtle graphics in its time) is a great introduction to programming, but you can't teach an old dog new tricks or an old APL programmer like me Scratch (not because I can't learn to use it, but because I do not want to).

What I do want to do is to write my robot brain in APL.

My first point: How was I to pass instructions from APL to the Pi's GPIO pins?

My first idea was to use the Python *Shared Objects* (Window users read Dynamic Link Libraries) that accessed the GPIO pins from APL via `ctypes` calls. The principle was sound, but the practice failed, due to un-resolved dependency in the Python Shared Objects. (I spent a day following a Google trail that ended up in a flame war between one side saying it was a *bug*, and the other a *feature*.)

I then searched for a C Shared Object and found the `BCM3825` library[2]. It is an excellent piece of work written (in C) by Mike McCauley.

This is a C library for Raspberry Pi (RPi). It provides access to GPIO and other IO functions on the Broadcom BCM 2835 chip, allowing access to the GPIO pins on the 26 pin IDE plug on the RPi board so you can control and interface with various external devices.

I had some early success calling this software from Dyalog (running as *root*) via `□NA` calls, but in the end ran into too many problems with 64bit integers. (Dyalog APL dose not support 64bit integers as a native type.)

(Do I hear Windows users asking what *root* is? It is the user account with full administrator access. Under LINUX anyone can run a command with root privileges by pre-positioning `sudo` in front on the command on the command line. So entering the command `sudo dyalog control.dws` would run the workspace `control.dws` under Dyalog APL with administrator access rights.)

On Linux (as Unix) hardware devices appear to the operating system as files. The BCM2835 GPIO pins are in the `/sys/class/gpio/` folder which require *root* access to read and write to it.

Blinking Lights

I next discovered that I could from the command line, access the pins directly!

With a LED attached to pin 17 the following will make the LED light up and go out twice:

```
sudo echo 17 > /sys/class/gpio/expand
sudo echo out > /sys/class/gpio/gpio17/direction
sudo echo 1 > /sys/class/gpio/gpio17/value
sudo echo 0 > /sys/class/gpio/gpio17/value
sudo echo 1 > /sys/class/gpio/gpio17/value
sudo echo 0 > /sys/class/gpio/gpio17/value
sudo echo 17 > /sys/class/gpio/unexpand
```

So from Dyalog APL, I should be able type:

```
□SH 'sudo echo 17 > /sys/class/gpio/expand'
□SH 'sudo echo out > /sys/class/gpio/gpio17/direction'
□SH 'sudo echo 1 > /sys/class/gpio/gpio17/value'
□SH 'sudo echo 0 > /sys/class/gpio/gpio17/value'
□SH 'sudo echo 1 > /sys/class/gpio/gpio17/value'
□SH 'sudo echo 0 > /sys/class/gpio/gpio17/value'
□SH 'sudo echo 17 > /sys/class/gpio/unexpand'
```

And if Dyalog was started with `sudo` then all I would have to type would be

```
□SH 'echo 17 > /sys/class/gpio/expand'
...
□SH 'sudo echo 17 > /sys/class/gpio/unexpand'
```

And it works, but it was not very fast

My next thought was to try `□ARBOUT`, and with a little help from the guys at Dyalog (via the forum) I worked out by trial and error that what I needed was:

```
'/sys/class/gpio/expand' □ARBOUT □UCS '17',□TC[2]
'/sys/class/gpio/gpio17/direction' □ARBOUT □UCS 'out',□TC[2]
'/sys/class/gpio/gpio17/value' □ARBOUT □UCS '1',□TC[2]
'/sys/class/gpio/gpio17/value' □ARBOUT □UCS '0',□TC[2]
Etc
```

Finally, I ended up with

```
'/sys/class/gpio/expand' □ABOUT 49 45 10
'/sys/class/gpio/gpio17/direction' □ABOUT 111 117 116 10
'/sys/class/gpio/gpio17/value' □ABOUT 49 10
'/sys/class/gpio/gpio17/value' □ABOUT 48 10
'/sys/class/gpio/gpio17/value' □ABOUT 49 10
'/sys/class/gpio/gpio17/value' □ABOUT 48 10
'/sys/class/gpio/unexpand' □ABOUT 49 45 10
```

And this is FAST!

Look left, right and left again

Now I can control the motors directly from Dyalog.

Question, how do I produce a PWM signal required for the pan and tilt servos?

Searching the internet, I discovered *ServoBlaster*. This is a well known (in Raspberry Pi circles) piece of software that allows simple command to activate the servos.

Following instructions on the internet, I installed *ServoBlaster* on my machine.

At Pi boot-up time I automatically start *ServoBlaster*. I archived that by placing in */etc/rc.local* file the following line of code:

```
sudo /home/pi/PiBits/ServoBlaster/user/servod -idle-time=50 -p1pins=18,22 &
```

This command (running as root) tells *ServoBlaster* to run using, pins number 18 and 22 on the Raspberry Pi GPIO header. *ServoBlaster* *exposes* these as pins 0 and 1. The *idle-time=50* turns the power to the servos off 50 milliseconds after use to save power. The *&* at the end of the line, tells the programme running the *rc.local* script NOT to wait for this command to finish. THIS IS IMPORTANT as the code runs as an endless loop, or it is told to turn itself off.

Since I boot up with an auto-login, but no monitor, I also added into *rc.local* the commands

```
echo 0=50% > /dev/servoblaster
echo 1=50% > /dev/servoblaster
```

These commands move the 'Pan-and-Tilt' servos into their central position.

When there is no monitor attached, this gives me audio/visual warning that the Raspberry Pi has loaded its OS and is now ready to run.

Within APL, I use the code:

```
'/dev/servoblaster'□ABOUT <pin>,61,(DUCS <pcent>),37 10
```

Where *<pin>* is 48 for servo 0 and 49 for servo 1, and *<pcent>* is the percentage (as characters – eg '50') to move.

I have a test function that moves the *pan* servo from 50% down to about 10% back to about 90% and back to 50% in intervals of 0.5%. Then it repeats the process using the *tilt* servo. The 0.5% steps creates a very smooth motion.

There's magic in my eyes, I can see for miles and miles

My kit came with the HC-SR04 a very cheap (but quite good) ultrasonic distance sensor. The blurb says: “Ultrasonic ranging module HC - SR04 provides 2cm - 400cm non-contact measurement function, the ranging accuracy can reach to 3mm”

Unfortunately, the accuracy is only as good as your *stop-watch*. The 3mm accuracy relates to a time difference of a few microseconds. In a multi-tasking environment such as Linux running multiple threads under Dyalog, starting and stopping a clock cannot hope to achieve anything better than a centimetre or two. And this assumes you have a clock that measures in micro-seconds. `QTS` only goes down to milliseconds.

After hunting around, I found I could use the `gettimeofday` and `usleep` function in the `libc.so.6` library that Dyalog uses on my Raspberry Pi.

The `QNA` calls that I use are:

```
QNA'libc.so.6|gettimeofday={U U} =P'
QNA'U libc.so.6|usleep U'
```

To get a better reading, I make several calls, throw away the *outsiders* and then take the smallest. Taking the average might at first seem better, but the *clock* should never show a shorter time than it took, but will often show a longer time.

Departing from the script.

The instructions I got with my first Raspberry Pi, did not tell me how to stop it, so I simply ‘pulled the plug’. However this frequently causes file corruption and is not recommended. It is far better to issue a `sudo poweroff` command. With no monitor or keyboard attached to the robot, this can often be quite tricky. Then I came across a web page that described how to first add a *RESET* button to the Pi, and then a safe *POWERDOWN* button.

The reset switch was the easy one. The unpopulated *Header P6* (on Model B Raspberry Pis only) exposes two points that connected forces the BCM2835 chip to RESET. So adding a *push to make* micro switch across them give one a hardware RESET button. Included in the Initio kit were 40 *header pins strip*. Following instructions on a U-Tube video, I very carefully, soldered two of the header pins into P6 on the Raspberry Pi. I then attached a reset switch recovered from an old PC. This hardware reset required no programming.

Pressing the reset button, will reboot the Raspberry Pi. But unless done after a power down, it is lightly to create file damage.

The web page I had found described exactly how to force a power-down via button to a GPIO pin.

The python script at flipdot.org/blog/uploads/shutdown.py.txt does this trick. This bit of code waits until GPIO 31 (exposed as pin 6 on header P5) to be grounded (eg connected to pin 8 on header P5). So installing this script as `/home/pi/shutdown.py`, and adding the line to `crontab -e`

```
@reboot sudo python /home/pi/shutdown.py
```

So out with the soldering iron again, this time adding two pin on to header P5, pins 6 and

8, attaching another *press to make* switch between them, and we are away.

Pressing the power-down button, issues ‘sudo powerdown’ request, and the Raspberry Pi closes down without file damage. Actually this can be controlled by software without a switch being installed. Once set up, to power down, one can simply issuing the command in APL:

```
'/sys/class/gpio/gpio31/value' □ARBOUR 49 10
```

Pressing the reset button, then boots it up again. See *Part Two* for a description of the software.

Note for would be Roboteers.

The choice between Pivoted wheels vs Tank tracks.

One of the most fundamental choices one must make is (having decided on a wheeled vehicle, as against say a quad-copter, boat, or articulated legged spider) is the wheel and steering arrangement.

The simplest is a three wheel (or more usually, two wheels on one axle and a *ball*) arrangement. There are two independent motors, one for each of the two drive wheels, and these provide both propulsion and steering. The third wheel or *ball* is free running and able to rotate about a vertical axis. The robot is able to turn (spin) on the spot.

The next simplest is the tracked arrangement (as in a tank) where the left and right hand tracks (each with its own motor) provide both propulsion and by varying the relative speed of the two tracks, steering. It too is able to turn on the spot. A two axle four wheel drive, with the front and back wheels on each side synchronised (where the two axles are parallel and fixed) can be regarded as a tracked vehicle. My Initio has this arrangement.

More complex is two axle four wheel arrangements where one of the axles can rotate about a vertical axis. With steering being provide by a servo via the vertical axis, and propulsion via a single motor but distributed thought a differential gear. This is how most cars work. It is unable to turn (spin) on the spot, and may require significant space to turn though 180 degrees.

The big advantage of the tracked robot is more stable and rugged and better able to transverse uneven ground.

The disadvantage of a tracked robot over a ‘two wheels and a ball’ robot is the motors driving the tracked robot have to be a LOT stronger when turning in a carpet. Turning a tracked vehicle involves some sliding, so the motors must be able to overcome the friction between track and ground. Tracked vehicles tend to tear up the ground. If the motors are not strong enough, the vehicle simply does not turn.

Part Two – The APL Software

From a control point of view the three wheels and tracked robots can use the same software controlling the two independent motors. The four wheeled *car* requires completely different controls, one program for the steering servos, the other for the propulsion speed and direction.

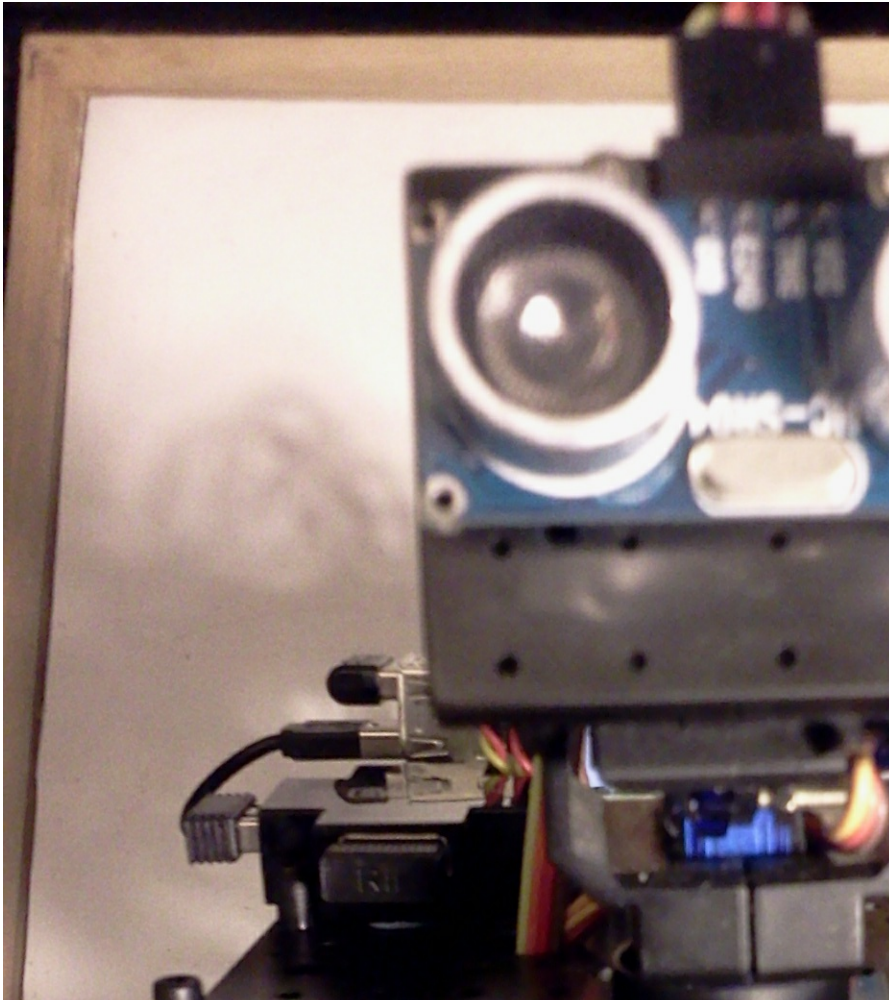
My Initio can be considered as a tracked vehicle, and requires separate orders to be sent to the left and right hand sides. By varying the relative speed and direction of the two motors,

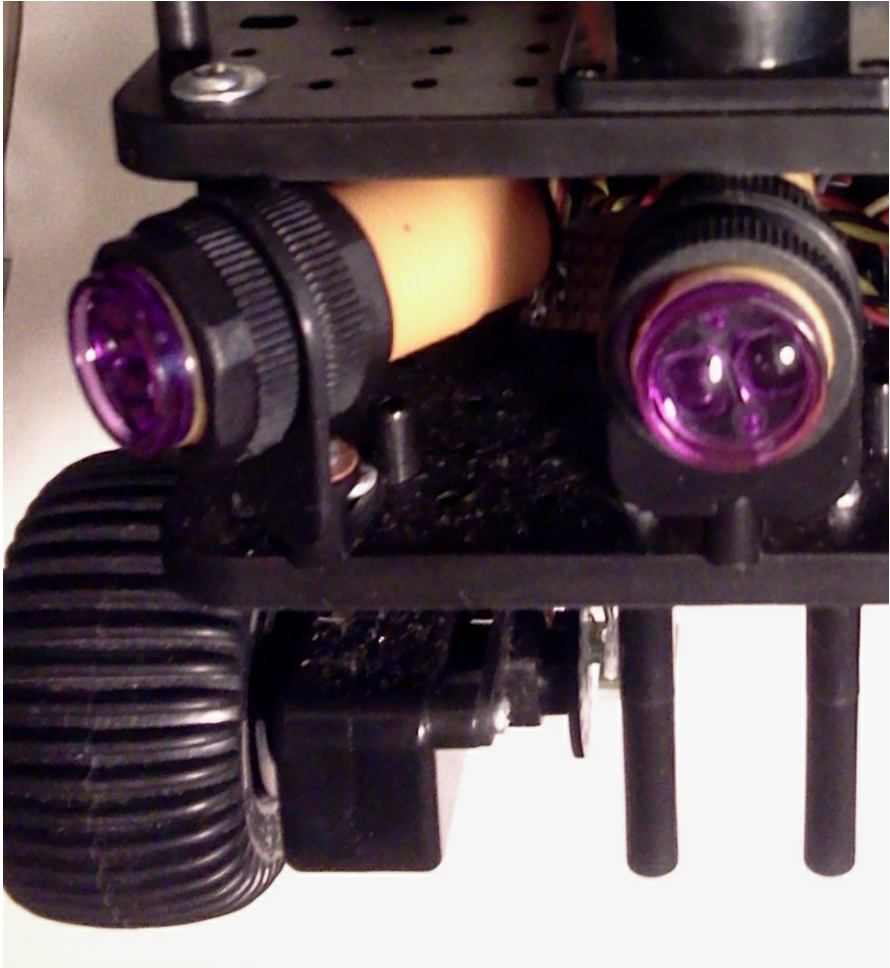
the position of the robot can be fully controlled.

My current code for the robot has evolved over two month, and I am sure will continue to evolve. Initially I was concerned on how would simply control the motors and servos, but later it was about multi-tasking, using the feedback from the sensors. Finally it was about strategy and tactics, and giving it the name 'PiRat Bot'.

Along the way I become aware of the problems of controlling a Raspberry Pi without a monitor attached in an alien environment without recourse of a known wireless network.

At Home, I can simply attach a monitor, keyboard and mouse to PiRat Bot, set it on *jacks* so it does not run away as I power it up, and then test the software in the comfort of my office. At this point I am able to connect PiRat Bot to my local network via WiFi, and take note of the IP address, which the Pi displays on screen each time it boots up.





Face to face with PiRat Bot

Then to test PiRat Bot while running free, I can use PuTTY. Running PuTTY on my desktop PC (or laptop) I can connect via WiFi to the Raspberry Pi and logon using my desktop's keyboard Mouse and monitor. All PuTTY running on my desktop needs to know is the IP address that the WiFi adaptor is using to connect to my local network.

However, running PiRat Bot away from my home network, I cannot use the local WiFi network, without first connecting to it which would normally require running software and providing network passwords etc. This in turn requires a screen, keyboard and mouse attached to the Pi.

To get round the alien WiFi , I now bring along a WiFi hotspot (supplied by my phone) that the Pi has already been *introduced* to. However, that did not always supply the PI with the same IP address.

My solution to that was to install the free software package `eSpeak`. `eSpeak` converts text to sound which can then be played via a speaker, or headphone, and get it to *say* my IP address.

The final implementation was to add a Bluetooth USB dongle, set up as a A2DP sink and send the spoken IP address to a Bluetooth headset. (“A2DP - The Advanced Audio Distribution Profile is a Bluetooth profile that allows for the wireless transmission of stereo audio from an A2DP source.”)

So now I can, using PuTTY connect to my Raspberry Pi, from my laptop using my phone as a WiFi hotspot. This was done in *anger* for the BAA London meeting on 28th Feb 2014, where Dan Baronet demonstrated how to install Dyalog APL and MiServer on the Raspberry Pi.

Once `eSpeak` was installed, I integrated it with the Dyalog’s `□SM` (the character based *screen map* interface), to provide a *spoken menu system*. This allows me to start up PiRat Bot without a screen, but with the audio feedback of the spoken menus, navigate my way around, running applications, and powering down Linux safely.

So now, if I just want to take the robot for a run, I can travel *light*, with just BlueTooth speaker and keyboard needed to accompany the PiRatBot with its BlueTooth adaptor. (I no longer need the laptop, phone, WiFi hotspot, or WiFi adaptor, unless I want to debug the APL.)

PiRat Bot the Amazing Rodent

Initially I tried running an APL thread for each process (eg one for each motor, servo and sensor) with the main thread co-ordinating whole. Named tokens were used (via `TGET` and `TPUT`) to pass information between the threads. I used the analogy of a steam ship, and the Engine Order Telegraph (*chadburn* – Full Ahead, Dead Slow Astern, Stop all Engines) , and quickly replace Left with Port and Right with Starboard (and not forgetting Leslie Phillips “left hand down a bit” in the Navy Lark).

The main thread was the Captain, the sensors threads were port and starboard lookouts, and the motors threads the engine room.

However, this was found to be too erratic, often leading to conflicting orders being sent as obstacles (Icebergs and Sheet Ice) were reported by the lookout while trying to navigate an *inside* right angle bend in a maze.

Eventually after many tests, I went with the much simpler ‘polling in a loop’ method. Once the Tactics on how the input was collected and how to respond to emergency conditions (not bumping into anything) was decided, I went on to the Strategy.

The problem with a strategy is one needs an objective. ‘Not bumping into anything’ is boring, simply ‘Stop all engines’ achieves that. ‘Not bumping into anything, while moving’ is a lot better, but leads to a *semi-drunken* walk (A random walk avoiding obstacles). I eventually ended up with ‘traversing a maze’ as my objective.

And the strategy I adopted was:

1. Find the nearest wall and move to it.
2. Turn so the wall is on one side (left or right, depending which is nearer).
3. Follow that wall, always keeping it to the same side.
4. Stopping when the task was completed.

To find the nearest wall, PiRate Bot sets the servos to their mid positions, pointing straight ahead, takes some ultra-sound distance readings, notes them down, rotates through a small angle and repeats the process until a full circle has been completed.

Then turning back to face the direction where the shortest distance was measured, and finally driving forward till the obstacle sensor could sense the wall.

If during the *turning* the (short range) obstacle sensors registered an obstruction, PiRat Bot assumes it is already next to the wall, so abandons (1) and proceeds to (2)

The main difficulty I have with (1) is knowing how far the *small angle* actually is.

What I ended up doing was turning for a 'fixed short time' for a 'fixed number of turns'. The actual length of time spent turning, and the number of turns determined by trial and error. However, depending on the floor surface, and the strength of the batteries, turning for identical periods of time rotated PiRat through different angles. So this is 'work in progress'.

Wall of death

The process of following a wall, while always keeping it to one side, is much easier to describe if I adopt one of the sides (say left) and describe following the left hand wall with my left hand always touching it. So while my code follows the 'selected side wall' I will describe following the left hand side wall.

While following (running alongside) a wall with an *on/off* obstacle sensor (rather than a distance sensor) one end up turning towards the wall until it is sensed, then turning away from it (till not sensed), then moving a little forward and either 'reacquiring the wall in the forward movement', or having to turn back towards the wall to find it again, before repeating the process.

I use the term *over-run* for the act of moving forward for a short distance without left hand *contact*.

In a maze, while following the left hand wall, a right angle turn in the wall is on the *inside* curve when the turn is to the RIGHT, and the *outside* curve when it is to the LEFT.

So when turning to the right, (inside) the turn must start BEFORE the apex is reached.

When turning to the left, (outside) the turn must not start until the apex has been passed.

The Bot's not for turning

For PiRat Bot with three forward facing obstacle sensors (one facing dead ahead, and the others pointing to the left and right), an inside curve is simple to detect, as the forward sensors are able to warn of the obstacle ahead.

In the real world with uneven walls, moment to moment feedback from the 3 forward sensors initially resulted in PiRat Bot 'changing its mind' repeatedly. Extra code had to be inserted so that it 'would remember' which way it was turning. This was especially true when negotiating a *U-Turn* at the bottom of a cul-de-sac.

However, for the outside curve, contact with the left hand has to be lost while the corner is being negotiated, and then has to be re-established before continuing. (Actually this is the same *over-run* process as already described in following the wall.)

In my code, the number of times round the main loop while in the *over-run* phase (derived by trail and error) is currently set at 800. However, this has recently been revised down from 1000, when I moved my *testing grounds* from my study's carpet onto my bath-room's quarry tiles.

PiRat Bot's will need feed-back from the real world if turning ability the *local* surface is to be automatically controlled, rather than using a pre-defined profile as at present. However, I am unsure how best to get this feedback. A three axis accelerometer is looking like the most useful way forward, but that decision is for another day.

References

1. 4tronics Initio
www.amazon.co.uk/Ultimate-4tronix-initio-Complete-Raspberry/dp/B00GEXGW4K
2. BCM3825 library
<http://www.airspayce.com/mikem/bcm2835/>

APL and partitioned data

by Jonathan Barman

Introduction by Phil Last: If I remember anything from the early editions of *Vector* it's Jonathan's advice in this article. The first thing that springs to mind is always to catenate the identity to the vector before dropping the redundant element to perform a difference operation. Enough detail; it's all here. I must just mention that thirty years on, despite the hopes expressed in Jonathan's closing paragraph, the simple techniques elucidated here can still out-perform their modern counterparts using partitioned enclose, each and join ($\rhd$, $\diagup$).

Introduction

APL arrays provide a natural way of partitioning data. A matrix can be viewed as a set of vectors; each row of a matrix of numbers could be vectors of costs incurred by each department in a business. Adding up the total costs incurred by each department is then a simple matter of applying plus reduction along the last dimension. Reduction and Scan operators allow the application of any scalar function along any axis of an array, and provide powerful tools for creating functions which work on partitioned data. There are, however, limitations in some applications. In the example of department costs, some departments may incur many cost items, and have long vectors, while others may have only one or two cost items. APL arrays have to be rectangular, so holding the costs as a matrix means that all the rows have to have the same length; short vectors have to be padded out with zeros to match the largest number of cost items. The amount of padding required can be so large that it becomes difficult to manipulate the matrix without workspace full messages, although the amount of data is relatively small. If one department out of 100 departments had 5000 cost items and all the other departments averaged 5 items apiece, then the matrix has to be 100 rows by 5000 columns taking up 2,000,000 bytes, of which only 40,000 bytes is data.

Another difficulty is where the data is normally manipulated as a vector and it is inconvenient to form it into a matrix so that reductions and scans can be applied, and then reformat it as a vector. For example, text typed by the user of a system may need to be manipulated and redisplayed, and it is convenient to keep the text as a vector throughout the processing.

This article explores the ways in which partitioned data can be processed in a more natural way, without looping. The techniques are well known and have been in use for many years. The Working Memorandum on Boolean Techniques by Robert A. Smith was published by STSC in 1975, and sets out the fundamental ideas and lists an extensive set of functions. The FinnAPL Idiom list contains examples of manipulating partitioned data. The APL*Plus and Sharp timesharing services both provide work spaces of partition functions.

Before plunging into detail, the general principles will be illustrated with a simple example. The principles will then be analysed in more detail and illustrated with more examples. Taking the department cost example, assume that each department has a unique code and that the costs and codes are held in two numeric vectors COSTS and DEPTS. The costs and department codes were entered from invoices, so that each cost has a corresponding department code.

There are three basic ways of adding up the costs for each department; by looping through each department code, by forming the data into a matrix and using plus reduction, or by using partition techniques. The looping method could be implemented as follows:

```

▽R←DEPTS ADDUP COSTS;A;B;ΠIO
[1]  ⌘ ADD UP <COSTS>FOR EACH CODE IN <DEPTS>
[2]  ⌘ <R[;1]>IS DEPT CODES, <R[;2] IS TOTAL COSTS.
[3]  ⌘ LOOPING METHOD.
[4]  ΠIO←1
[5]  R←O 2pO
[6]  L1:→(0∈pDEPTS)/O
[7]  A←1↑DEPTS
[8]  B←A=DEPTS
[9]  R←R, [1]A, +/B/COSTS
[10] B←~B
[11] DEPTS←B/DEPTS
[12] COSTS←B/COSTS
[13] →L1
▽

```

```

1 4 4 3 1 ADDUP 10 20 30 40 50
1 60
4 50
3 40

```

This method is inefficient if large amounts of data are involved. Lines 9 to 12 reassign the variables, so data is being moved in memory for every unique department found.

Forming the data into a matrix is more efficient than the looping method, but, as explained above, there may be workspace full problems:

```

▽R←DEPTS ADDUP COSTS;A;P;ΠIO
[1]  ⌘ ADD UP <COSTS>FOR EACH CODE IN <DEPTS>
[2]  ⌘ <R[;1]>IS DEPT CODES, <R[;2] IS TOTAL COSTS.
[3]  ⌘ MATRIX METHOD.
[4]  ΠIO←1
[5]  ⌘ SORT INTO DEPT CODE SEQUENCE

```

```

[6] A←⊖DEPTS
[7] DEPTS←DEPTS[A]
[8] COSTS←COSTS[A]
[9] ⌘ FIND WHERE CODES CHANGE
[10] P←DEPTS≠1⊥DEPTS,0
[11] P[(0≠ρP)/ρP]←1
[12] ⌘ FIND NUMBER OF CODES FOR EACH DEPT.
[13] A←P/⊥ρP
[14] A←A-⊥1⊥0,A
[15] ⌘ FORM EXPANSION VECTOR.
[16] A←A°,≥⊥⊥/0,A
[17] ⌘ MAKE COSTS INTO A MATRIX.
[18] R←(ρA) ρ(,A) \COSTS
[19] ⌘ ADD UP, AND APPEND DEPT CODES.
[20] R←(P/DEPTS),[1.5]+/R
    ▽

1 4 4 3 1 ADDUP 10 20 30 40 50
1 60
3 40
4 50

```

Forming the data into a matrix requires a technique which is constantly being used when dealing with partitioned data. Line 10 is a ‘not-equals positive difference operation’, and line 14 is a ‘minus negative difference operation’.

Lines 10 and 11 generate a ‘partition vector’. Line 11 is necessary because it cannot be guaranteed that a department code of zero does not exist. If the rotation method is used:

```
P←DEPTS≠1⊖DEPTS
```

then line 11 is required in case there is only one department code in the data.

Care has been taken that empty arguments do not cause an error. Line 11 checks for an empty vector. Line 14 could have been written as:

```
A←A-0,⊥1⊥A
```

which would have caused a length error if A was empty. Line 16 has a 0 catenated to A in case it is empty. It is good practice to ensure that all code will work properly on empty vectors, but it is sometimes simpler to branch out on empty at the beginning of the function rather than having to include special processing as in line 11.

The partitioned data approach is as follows:

```

▽R-DEPTS ADDUP COSTS;A;P;PIO
[1] A ADD UP <COSTS>FOR EACH CODE IN <DEPTS>
[2] A <R[;1]> IS DEPT CODES. R[;2] IS TOTAL COSTS.
[3] A PARTITION METHOD.
[4] PIO-1
[5] A SORT INTO DEPT CODE SEQUENCE
[6] A-ADEPTS
[7] DEPTS-DEPTS[A]
[8] COSTS-COSTS[A]
[9] A FIND WHERE CODES CHANGE
[10] P-DEPT#1;DEPTS,0
[11] P[(O≠PP)/PP]-1
[12] A CUMULATIVE SUM FOR EACH DEPT.
[13] R-P/+COSTS
[14] A CONVERT TO INDIVIDUAL SUMS.
[15] R-R-1+0,R
[16] A APPEND DEPT CODES
[17] R-(P/DEPTS),[1.5]R
▽

```

```

1 4 4 3 1 ADDUP 10 20 30 40 50
1 60
3 40
4 50

```

The steps down to line 11 are identical to the matrix method. Line 13 gets the overall cumulative sum for each department, and line 15 does the ‘minus negative difference operation’ which converts the cumulative sums back to individual sums. This relationship between scan and negative difference operation is another important technique which will be explored more fully later.

The ADDUP function is really carrying out three processes: sort the data, set up a partition vector, and carry out a partitioned plus reduction. The processes are needed in many varied circumstances, so it is convenient, and better programming practice, to have separate functions. Lines 10 and 11 generated a trailing partition vector as it was needed in this form on line 13. A trailing partition vector is one where a 1 flags the end of each partition:

```

A-2 2 2 6 6 7 7 7
A#1;A,0
0 0 1 0 1 0 0 1

```

A leading partition vector is one where a 1 flags the start of each partition:

```

A#1;0,A

```

```
1 0 0 1 0 1 0 0
```

All partition functions need a partition vector as an argument, and it is necessary to standardise on either leading or trailing partitions. As the literature on partition functions always uses leading partitions, we will do likewise. The first function to be defined is one to create a partition vector:

```
▽R←CREATEΔPARTITION A
[1] ⌵ <R> IS A LEADING PARTITION VECTOR WITH 1'S WHERE
[2] ⌵ <A> CHANGES.
[3] R←A≠1⊕A
[4] →(0∈pR)/0
[5] R[⊠IO]←1
▽
```

```
CREATEΔPARTITION 1 1 1 1 8 8 50 50 50
```

```
1 0 0 0 1 0 1 0 0
```

Line 3 uses the rotation method to allow for the data being either character or numeric, line 4 branches on empty, and line 5 guarantees the first element is a 1.

```
▽R←P PΔPLUSΔRED A
[1] ⌵ <P> IS A LEADING PARTITION VECTOR,
[2] ⌵ <A> IS A NUMERIC ARRAY.
[3] ⌵ <R> IS A PARTITIONED PLUS REDUCTION ON THE
[4] ⌵ FIRST DIMENSION OF <A>.
[5] R←(1⊕P)⊕+A
[6] R←R-(pR)p0, [⊠IO]R
▽
```

Lines 5 and 6 of the partitioned plus reduction function are generalisations of lines 13 and 15 of the last ADDUP example. The rotate of the leading partitions on line 5 changes them into trailing partitions, and the plus scan is carried out along the first dimension so that the data can be a matrix. Line 6 carries out the ‘minus negative difference operation’ along the first dimension of the array.

```
P
1 0 0 0 1 0 1 0 0

A
6 9 1
1 6 7
1 4 1
```

```

5  7  6
10 9  6
1  7  5
8 10  8
3  1  8
4  7  8

P PΔPLUSΔRED A
13 26 15
11 15 11
15 18 24

```

A more general accumulation function can be written in place of the ADDUP function:

```

▽R~ACCUMULATE A;P;010

[1] A <A[;1]> IS A SET OF CODES. REMAINING COLUMNS
[2] A OF <A> IS DATA. <R> IS THE UNIQUE SET OF
[3] A CODES IN COUMN 1. WITH THE TOTALS OF THE DATA
[4] A IN THE REMAINING COLUMNS.
[5] ΠI0←1
[6] →(0∈pR-A)/0
[7] R←R[ΔR[;1];]
[8] P←CREATEΔPARTITION R[;1]
[9] R←(P/R[;1]),P PΔPLUSΔRED 0 1;R

▽

```

```

A
5 37 25 99
4 76 66  8
4 89 28 44
4 48 24 28
2 17 49 90
5  7 91 51

ACCUMULATE A
2 17 49 90
4 213 118 80
5 44 116 150

```

Difference Operations

A list of the boolean difference operations are given in the appendix. At first sight they tend to look similar, and it is difficult to appreciate which ones are going to be useful. Rather than go through them all, the following functions show how the more popular

difference operations are used in practice.

The greater-than negative difference operation keeps the first one in each series of ones, and sets the remaining elements to zero:

```
A-0 1 1 0 0 1 1 1 1 0
A>~1;0,A
0 1 0 0 0 1 0 0 0 0
```

This difference operation is very useful when analysing text typed by the user. For example, when checking full screen data input it is usually necessary to count how many words or numbers have been entered in each screen field:

```
▽R←WORDCOUNT A

[1] ⌘ <A> IS A CHARACTER MATRIX. <R> IS THE NUMBER
[2] ⌘ OF WORDS OR NUMBERS ON EACH ROW OF <A>
[3] R←A≠' '
[4] R←R>(pR)↑0,R
[5] R←+/R
▽

A
ONE TWO
12 34 6
589
WORDCOUNT A
231
```

When errors are found in the text typed by the user, an error message has to be displayed describing what has gone wrong. It is nice to be able to point out the exact location of the trouble:

```
▽R←ERR REPORTaERROR TEXT;A;CR

[1] ⌘ <ERR>IS A BOOLEAN ERROR INDICATOR WITH AN ELEMENT
[2] ⌘ FOR EACH WORD IN THE CHARACTER VECTOR <TEXT>.
[3] ⌘ <R> IS AN ERROR MESSAGE.
[4] A←TEXT≠' '
[5] A←A>~1;0,A
[6] ⌘ VSAPL CARRIAGE RETURN CHARACTER.
[7] CR←0 1 0/⌘TC
```

```
[8] R←'INVALID ITEM:',CR,TEXT,CR,A\ERR\ 'A'
      ▽

      0 0 1 0 REPORT△ERROR' ONE TWO THRE FOUR'
INVALID ITEM:
      ONE TWO THRE FOUR
      ^
```

Lines 4 and 5 flag the first letter of each word in the TEXT, which is used to position the caret on line 8. Two algorithms for checking numbers were published in Quote Quad, and they both exhibit the use of difference operations. Algorithm 139 by Gerald Bamberger in the March 1980 issue of Quote Quad (Vol 10 No 3) verifies numeric input, and is similar to the Quad function available on Sharp APL and APL*Plus APL

```
▽R←VI A
[1] ⌘ VERIFY NUMERIC INPUT.
[2] ⌘ <A> IS A CHARACTER VECTOR CONTAINING GROUPS OF
[3] ⌘ CHARACTERS DELIMITED BY ONE OR MORE SPACES.
[4] ⌘ <R> IS A BOOLEAN VECTOR WITH A 1 WHERE THE
[5] ⌘ CHARACTER GROUP IS A VALID NUMBER. NUMBERS
[6] ⌘ OUTSIDE RANGE (⌈/10)<A<⌊/10 COUNTED AS VALID.
[7] R←' 1111111112345'[( ' 0123456789.¯E't'0 ',A]
[8] R←1⊥((R≠'234')∇R≠¯1⊥' ',R)/R
[9] R←R≠(8 3p0 41 431)+1 12 121 21 31 312 3121 3210.×1 100 1000
      ▽

      VI'123 1.3 3¯4 3.4 3E4 ¯3E¯34 E3'
      1 1 0 1 1 1 0
```

A slightly simpler version, excluding E notation values, may be preferred:

```
▽R←ΔVI A
[1] ⌘ VERIFY NUMERIC INPUT.
[2] ⌘ <A> IS A CHARACTER VECTOR CONTAINING GROUPS OF
[3] ⌘ CHARACTERS DELIMITED BY ONE OR MORE SPACES.
[4] ⌘ <R> IS A BOOLEAN VECTOR WITH A 1 WHERE THE
[5] ⌘ CHARACTER GROUP IS A VALID NUMBER. NUMBERS
[6] ⌘ OUTSIDE RANGE (⌈/10)<A<⌊/10 COUNTED AS VALID.
[7] R←' 111111111234'[( ' 0123456789.¯t'0 ',A]
[8] R←1⊥((R≠'23')∇R≠¯1⊥' ',R)/R
[9] R←R≠1 12 121 21 31 312 3121 321
      ▽
```

Line 8 of the function uses the not-equal negative difference operation to remove

duplicates.

Algorithm Number 146 by Jeffrey Multack, in the September 1980 issue of Quote Quad (Vol 11 No 1) converts numeric input:

```

      ▽R←FI A;M
[1]  ⌘ VERIFY AND CONVERT NUMERIC INPUT.
[2]  ⌘ <A> IS A VECTOR CONTAINING GROUPS OF CHARACTERS
[3]  ⌘ DELIMITED BY ONE OR MORE SPACES, <R> IS A
[4]  ⌘ NUMERIC VECTOR WITH VALID NUMBERS IN <A> OR
[5]  ⌘ ZERO FOR ANY GROUP WHICH IS NOT A NUMBER.
[6]  R←VI ' ',A
[7]  →(v/R)+O
[8]  ⌘ FORM MASK FOR VALID CHARACTER GROUPS.
[9]  M←A≠' '
[10] M←M>~1⊥0,M
[11] M←≠\M\R≠~1⊥0,R
[12] R[R/⊥ρR]←M/A
      ▽

      FI'123 1.3 3~4 3.4 3E4 ~3E~34 E3'
123 1.3 0 3.4 30000 ~3E~34 0

```

Lines 10 and 11 have been altered slightly so that they are in the same form as the difference operations already given. Line 10 is the greater-than difference operation which flags the first character in each group. Line 11 then extends the ones for each character group that is valid.

```

A←'1234 1..23 460'
⊥M←A≠' '
1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 1 1 1
⊥M←M>~1⊥0,M
1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0
⊥R←VI A
1 0 1
⊥M←≠\M\R≠~1⊥0,R
1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 1 1 1

```

Line 11 can be broken down into 3 stages, a not-equal difference operations:

```

R≠~1⊥0,R
1 1 1

```

an expansion:

```
M\R#~1i0,R
1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0
```

and a not-equal scan:

```
#\M\R#~1i0,R
1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1
```

Not equals scan has the property of switching from 1 to 0 and from 0 to 1 every time a 1 is encountered in the vector. The not-equals difference operation does the opposite, so the three stages are: apply a transformation, expand, then put it back to what it was. The partitioned plus reduction does a similar task:

```
R-R~1i0,R-P/+A
```

Apply a transformation (plus scan), compress, then put it back using the minus negative difference operation. The conversion back to the original form is possible because the minus negative difference operation is the inverse of plus scan.

```
□-A-+2 4 1 5
2 6 7 12
A~1i0,A
2 4 1 5
```

Also, the not-equals difference operation is the inverse of not-equals scan:

```
□-A-#\1 1 0 0 1
1 0 0 0 1
A#\1i0,A
1 1 0 0 1
```

Line 11 of FI is so useful that it should be defined as a function:

```
▽R-P PAMASK A

[1] @ <P> IS A LEADING PARTITION VECTOR. <A> IS A BOOLEAN
[2] @ VECTOR WHERE pA IS IDENTICAL TO +/P.
[3] @ <R>IS A BOOLEAN VECTOR WITH 1 IN EACH PARTITION
[4] @ WHERE <A> IS 1.
[5] R-#\P\A#~1i0,A
▽

1 0 0 0 1 0 1 0 0 PAMASK 1 0 1
1 1 1 0 0 1 1 1
```

Having seen that the basic process is difference, expand, scan, with not-equals, an equivalent function can be written using the same basic process, but with minus and plus:

```

▽R-P PAREPLICATE A
[1] ⌘ <R> IS A LEADING PARTITION VECTOR. <A> IS A
[2] ⌘ NUMERIC VECTOR WHERE pA IS IDENTICAL TO +/P.
[3] ⌘ <R> IS A NUMERIC VECTOR WITH EACH ELEMENT OF
[4] ⌘ <A> REPLICATED IN EACH PARTITION.
[5] R←+P\A-1 0,A
      ▽

      1 0 0 0 1 0 1 0 0 PAREPLICATE 2 6 3.2
2 2 2 2 6 6 3.2 3.2 3.2

```

Using the partitioned plus reduction technique of difference, compress, scan, but with not-equals in place of minus and plus, yields a partitioned not-equals reduction function:

```

▽R-P PANEARED A
[1] ⌘ <R> IS A LEADING PARTITION VECTOR. <A> IS A
[2] ⌘ BOOLEAN VECTOR <R> IS ≠/ FOR EACH PARTITION.
[3] R←(1⊖P)/≠\A
[4] R←R≠1 0,R
      ▽

```

This function can be used to flag partitions with an uneven number of occurrences.

Another ‘tool-box’ function that illustrates a difference operation is one for removing surplus spaces:

```

▽R-SQUEEZE A;B
[1] ⌘ REMOVE LEADING, TRAILING AND DUPLICATE SPACES
[2] ⌘ FROM CHARACTER VECTOR <A>
[3] R←A, ' '
[4] B←' '≠R
[5] B←B▽1 0,B
[6] R←1 B/R
      ▽

```

Line 5 has an ‘or’ negative difference operation which adds a 1 after each group of ones. Line 3 guarantees a trailing space which is then removed on line 6.

The following function is one of a set of functions to help formatting numeric data in VS APL:

```
▽R-BRACKETS A;B;C;D

[1] ⍺ <A> IS A CHARACTER ARRAY OF FORMATTED NUMBERS
[2] ⍺ WITH AT LEAST ONE SPACE BEFORE EACH NUMBER.
[3] ⍺ <R> HAS THE NUMBERS MOVED ONE SPACE TO THE LEFT
[4] ⍺ AND HAS BRACKETS IN PLACE OF NEGATIVE SIGNS.
[5] R←1∘,A
[6] B←R≠' '
[7] C←B>~1∩0,B
[8] D←'-'=C/R
[9] R[(C\D)/1∘R]←' ('
[10] C←B<~1∩0,B
[11] R[(C\D)/1∘R]←') '
[12] R←(∘A)∘R
▽

NUMS
.00 368.55 - .35 -40.10 - .34
.00 .00 6.43 536.58 - .04
.00 .00 .00 -761.24 .42

BRACKETS NUMS
.00 368.55 (.3) (40.10) (.34)
.00 .00 6.43 536.58 (.04)
.00 .00 .00 (761.24) .42
```

Line 7 is a greater-than negative difference which flags the beginning of each group of numbers. Line 10 is a less-than negative difference which flags the beginning of each group of spaces. The negative sign is therefore only replaced by both a left and right parenthesis.

The function was created to help develop a generalised formatter for VS APL. In practice, a formatting function would have the format specification available to indicate where the right parenthesis should be placed.

Partition Functions

Partition functions have been given for plus reduction and not-equal reduction, but partition functions are needed to carry out the equivalent of all the reduction and scan operations. The working Memorandum on Boolean Techniques gives a very comprehensive list, but here are two that are most frequently used:

```
▽R-P PAORARED A

[1] ⍺ <R> IS A LEADING PARTITION VECTOR. <A> IS A BOOLEAN VECTOR.
[2] ⍺ <R> IS ∨/ FOR EACH PARTITION OF <A>.
```

```

[3] R←(P∨A)/P
[4] R←(P/A)≥R/1∅R
    ∇

    ∇R←P PΔANDΔRED A
[1] ∅ <P> IS A LEADING PARTITION VECTOR. <A> IS A BOOLEAN VECTOR.
[2] ∅ <R> IS AI FOR EACH PARTITION OF <A>.
[3] R←(P≥A)/P
[4] R←(P/A)∧R/1∅R
    ∇

```

These functions are also published in the FinnAPL Idiom Library numbers 491 and 492.

An example of their use is taken from a set of functions to carry out formatting under VS APL:

```

    ∇R←BLANKΔIFΔZERO A:B:C:P
[1] A <A> IS A CHARACTER ARRAY OF FORMATTED NUMBERS.
[2] A <R> HAS ALL ZERO NUMBERS SET TO SPACES.
[3] R←A
[4] →(0∈B←ppR)/0
[5] R←,R
[6] P←R=' '
[7] P←P>⌈1,0,P
[8] P[∅IO]←1
[9] C←P PΔMASK~P PΔANDΔRED R∈' 0.'
[10]R←BpC\C/R
    ∇

```

```

NUMS
.00 368.55    - .35    -40.10    - .34
.00    .00    6.43    536.58    .04
.00    .00    .00    761.24    .42

BLANKΔIFΔZERO NUMS
    368.55    - .35    -40.10    - .34
                6.43    536.58    .04
                    -761.24    .42

```

Lines 6 to 8 set up a partition vector, and line 9 creates a mask for those partitions that do not have a space, zero or decimal point. Of course, if the format specification is available the partition vector can be set up without searching the data.

Finally, an example of using partition functions to eliminate looping. In the last issue of the APL User Group News Letter Dick Bowman gave a very interesting problem of calculating geometric means of sets of data.

The solution published calculated the geometric mean of each set of data in a loop, which would be inefficient if large amounts of data were involved and Dick ends his article with ‘There surely must be a better way’. The partitioned data approach would be to create a partitioned geometric mean function.

```

▽R←P PAGEOM A;B;C;D

[1] ⌵ <R> IS A PARTITION VECTOR, <A> IS A NUMERIC VECTOR.
[2] ⌵ <R> IS THE GEOMETRIC MEAN IN EACH PARTITION WHERE
[3] ⌵ ALL NUMBERS ARE GREATER THAN ZERO, OTHERWISE ⌵1.
[4] B←P PANDARED A>0
[5] C←P PAMASK B
[6] D←C/P
[7] R←*(D PPLUSARED⊙C/A)÷PASHAPE D
[8] R←(B\R)~B
  ▽

  ▽R←SHAPE P

[1] ⌵ <R> IS A LEADING PARTITION VECTOR.
[2] ⌵ <R> IS THE NUMBER OF ELEMENTS IN EACH PARTITION.
[3] R←(1⊙P)/⌵P
[4] R←R--1+0,R
  ▽

  1 0 0 0 1 0 1 0 0 PAGEOM 3 6 8 1 0 2 3 2 1
  3.464101615 ⌵1 1.817120593

```

Line 4 finds the partitions that need to be processed, and line 5 creates a mask. Line 7 then calculates the geometric mean for each valid partition, and line 8 sets invalid partitions to negative one. This function can then be used to replace the loop in the original function, after having created a partition vector.

Conclusion

Difference operation and partition functions provide a useful set of tools which help to solve the programming problems where the data does not fit in with rectangular nature of APL arrays.

The generalised array facilities in APL2, NARS and Sharp APL developments provide much more powerful tools for manipulating non rectangular data. The partitioned data approach is much easier and more direct with generalised arrays; the partitioned plus

reduction is merely a plus reduction for each element of a vector of vectors, and a proper notation is provided for its application. Roll on generalised arrays - but in the meantime we can make do quite successfully with partitioned functions!

APPENDIX

Boolean Difference Operations

Less-than Negative Difference. The first of each group of zeros is set to one, all other elements are set to zero.

```

□-A-0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A<~1,0,A
0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 1 0

```

Less-than Positive Difference. The last of each group of zeros is set to one, all other elements are set to zero.

```

A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A<1,A,0
0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0

```

Less-than-or-equal Negative Difference. The first of each group of ones is set to zero, all other elements are set to one.

```

A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A<~1,1,A
1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0

```

Less-than-or-equal Positive Difference. The last of each group of ones is set to zero, all other elements are set to one.

```

A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A<1,A,1
1 1 1 1 1 0 1 1 1 1 0 1 1 1 0 1 1

```

Equal Negative Difference. The first of each group of zeros and the element to the right of each group of zeros is set to zero, all other elements are set to one.

```

      A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
      A= $\neg$ 1,1,A
0 1 0 1 1 1 0 1 0 1 1 0 1 1 0 0 0

```

Equal Positive Difference. The element to the left of each group of zeros and the last of each group of zeros is set to zero, all other elements are set to one.

```

      A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
      A=1,1,A,1
1 0 1 1 1 0 1 0 1 1 0 1 1 0 0 0 1

```

Greater-than-or-equal Negative Difference. The first of each group of zeros is set to zero, all other elements are set to one.

```

      A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
      A $\geq$  $\neg$ 1,1,A
0 1 1 1 1 1 0 1 1 1 1 0 1 1 1 0 1

```

Greater-than-or-equal Positive Difference. The last of each group of zeros is set to zero, all other elements are set to one.

```

      A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
      A $\geq$ 1,1,A,1
1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1

```

Greater-than Negative Difference. The first of each group of one is set to one, all other elements are set to zero.

```

      A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
      A>> $\neg$ 1,0,A
0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 1

```

Greater-than Positive Difference. The last of each group of ones is set to one, all other elements are set to zeros.

```

      A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
      A>1,1,A,0
0 0 0 0 0 1 0 0 0 0 1 0 0 0 1 0 1

```

Not-equal Negative Difference. The first of each group of ones and the element to the right of each group of ones is set to one, all other elements are set to zero.

```
A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A≠¬1,0,A
0 0 1 0 0 0 1 0 1 0 0 1 0 0 1 1 1
```

Not-equal Positive Difference. The element to the left of each group of ones and the last of each group of ones is set to one, all other elements are set to zero.

```
A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A≠1,A,0
0 1 0 0 0 1 0 1 0 0 1 0 0 1 1 1 1
```

Or Negative Difference. The element to the right of each group of ones is set to one, all other elements are unaltered.

```
A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A∨¬1,0,A
0 0 1 1 1 1 1 0 1 1 1 1 0 0 1 1 1
```

Or Positive Difference. The element to the left of each group of ones is set to one, all other elements are unaltered.

```
A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A∨1,A,0
0 1 1 1 1 1 0 1 1 1 1 0 0 1 1 1 1
```

And Negative Difference. The element to the right of each group of zeros is set to zero, all other elements are unaltered.

```
A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
A∧¬1,1,A
0 0 0 1 1 1 0 0 0 1 1 0 0 0 0 0 0
```

And Positive Difference. The element to the left of each group of zeros is set to zero, all other elements are unaltered.

```
A
0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1
```

$A \wedge_{1,1} A, 1$

0 0 1 1 1 0 0 0 1 1 0 0 0 0 0 1

J

J-ottings 58

Internal Rate of Return and related topics

Norman Thomson

A common way of expressing the desirability or otherwise of an investment is through a rate of return. There are several ways in which 'rate of return' can be calculated. Comparisons between numbers are only meaningful if the *processes* producing the numbers are clearly and unambiguously understood. This is where J can be of great help in sorting pathways through a terminological jungle.

Internal Rate of Return (IRR) is a commonly used measure. To explain how it is obtained, consider that say 5% growth in any commodity can be calculated by applying a multiplier 1.05, and compound growth by successive applications of this multiplier. In a similar way a list of monetary values can be adjusted for 5% inflation by successive *divisions* by this multiplier. The *opposite* of 5% growth is 5% *decay* (that is growth of -5%), and successive multiplications of 0.95 result in *exponential decay*. However the *inverse* of growth is *not* exponential decay, but rather the process determined by the multiplier $(1.05)^{-1} = 0.9524$, meaning that a post-growth value is 95.24% of its original. In the case of decay of, say 20%, the corresponding inverse multiplier is $(0.8)^{-1} = 1.25$ corresponding to 25%. For example under a 20% tax regime, a pre-tax value is obtained from a net value by adding 25%, a process frequently referred to as 'grossing up'.

The above discussion is consolidated in the following two verbs :

```
ptom=%.>:@(0.01&*) NB. %age growth to inverse multiplier
mtop=.100&*@<:@% NB. inverse multiplier to %age growth
ptom 5 _20
0.952381 1.25
mtop 0.952381 1.25
4.99999 _20
```

In everyday terms this says that under an assumption of 5% inflation a rational person would be indifferent regarding offers of £95.24 today and a promise of £100 next year. £95.24 is said to be the *net present value* (often abbreviated to NPV) of £100 delayed until next year.

An *income stream* is a list such as

```
is=._100 20 50 70 80
```

which describes, say, an initial investment of 100 which produces subsequent returns of

20, 50, 70 and 80. The verb `npv` provides compounded net present values for a decay rate (*discount*) given by the left argument :

```
npv=.]* (ptom@[ ]^(i.@#@[ ]) NB. net present value
5 npv is      NB. 5% decay
_100 19.0476 45.3515 60.4686 65.8162
10 npv is     NB. 10% decay
_100 18.1818 41.3223 52.592 54.6411
```

The total of such lists is called the *net benefit*, and the absolute sum of the negative items is the *cost*. The net benefit of is before discounting is thus 120, and the cost 100. After discounting by 5% and 10% the net benefits reduce to

```
each=.&>
+/"(1) 5 10 npv each <is
90.6839 66.7372
```

Because exponential decay is not the inverse of growth the NPV discounted streams of `is` are different from the result of repeated compound reduction :

```
is*0.95^i.5
_100 19 45.125 60.0162 65.1605
is*0.9^i.5
_100 18 40.5 51.03 52.488
```

Internal Rate of Return answers the question what NPV discount rate would reduce net benefit to zero. In algebraic terms, this amounts to finding a positive real-valued multiplier which satisfies the equation

$$0 = -100 + 20m + 50m^2 + 70m^3 + 80m^4$$

This can be obtained using the root finding verb `p.` which solves polynomials whose argument is coefficients in ascending power order. In the verb `irr` this is followed by a conversion to a percentage:

```
irr=.mtop@pos@real@roots NB. internal rate of return
roots=.>@{:p.          NB. list of roots from result of p.
real=.#~ (= +)         NB. select real values
pos=.#~ >&0             NB. select positive values

irr is
32.3121
is%1.323121^i.5
_100 15.1158 28.5608 30.2203 26.1031
+/_100 15.1158 28.5608 30.2203 26.1031
```

```
0 NB. confirms net benefit is zero
```

In practical terms this means that under an inflation rate of 32.3% the underlying project would finish with no *residual* financial worth.

(Practical note: If `irr` fails through a limit error in `p`, use a standard root finding technique such as the following:

```
Newton=.adverb ':' ]-x%x D.1' (^:_ ) ("0 NB. Newton-Raphson
poly=|.is NB. define a polynomial
mtop (#.&poly) Newton 0.9
32.3121 )
```

Applying NPV before computing `irr` allows prior assumptions to be made about inflation, e.g.

```
irr 10 npv is NB. IRR assuming 10% inflation
20.28
irr 25 npv is NB. IRR assuming 25% inflation
5.85
```

For the reason given earlier these values are not identical with the `irr` values which arise on an *a priori* exponential decay assumption:

```
irr is*0.9^i.5 NB. assuming 10% exponential decay
19.08
irr is*0.75^i.5 NB. assuming 25% exponential decay
_0.7659
```

If all the inflows are delayed until the final period, the `irr` value is simply the rate of compound interest which for `is` would turn 100 into 220 in the course of four periods:

```
irr _100 0 0 0 220
21.7883
100*1.217883^4
220
```

Average Growth Rate

A reasonable question to ask is why bother with the relative complexity of IRR when a rate of return can be found by simply calculating $(\text{inflows} - \text{outflows}) \div \text{outflows}$ and averaging over the number of periods. To address this, the numerator (inflows–outflows) is just another way of expressing net benefit, while the positive sum of the outflows are given by the verb `costs`. This is illustrated with two input streams which differ only in the

order of their items:

```
costs=./(0&>.@-) NB. outflows (i.e. sum of -ve values)

is1=._100 20 45 60 _10 55 70 20 10 10
is2=._100 20 45 60 _10 10 10 20 70 55

costs each is1;is2

110 110
```

The *benefit-cost* ratios associated with these streams are

```
(+/ % costs)each is1;is2

1.636 1.636
```

However the stream in which high returns come later has a lower IRR :

```
,irr each is1;is2

30.19 25.22
```

showing that averaging loses some of the information implicit in ordering.

Where the term ‘benefit-cost ratio’ is used it is worth checking that the numerator reflects *net* benefits – effectively that it is inherently a profit ratio. If only inflows appear in the numerator it is a ‘selling price/cost price’ ratio and thus related to mark-up rather than return. The verb `bcr` first incorporates net present value discounting into the averaging process with the discount level set as the left argument:

```
bcr=.(+/%costs):@npv NB. benefit/cost ratio with discounting

10 bcr each is1;is2

0.80926 0.67133
```

Since the lengths of `is1` and `is2` are both 10 these ratios can be converted to an average compound growth rate by taking the 9th root of 1.80926 1.67133

```
9%:1+0.80926 0.67133

1.0681 1.0587
```

that is the average growth rates are roughly 7% and 6% given a discount rate of 10%. This procedure can be generalised as

```
ktop=.(100&*@<:) NB. transform (1+k) to 100k%

root=.<:@#

agr=.(ktop@((root@])%:(>:@bcr))

10 agr each is1;is2
```

```
6.81 5.873
```

If discounting is applied at around the `irr` value then the net average growth rate is more or less zero:

```
30 agr isl
0.0527249 NB. growth rate is virtually zero (0.05%)
```

Using IRR

Where values are money, choices between alternative projects can be guided by IRRs. IRRs can be recalculated dynamically, although they should not be used as a sole criterion since, provided inflows are positive, IRRs carry on increasing as the input stream lengthens, and so *increases* in the IRR tailing off can be a signal suggesting that other statistics should be investigated as well. In the case of `is` successive income streams are given by

```
}.<\is
+-----+-----+-----+-----+
|_100 20|_100 20 50|_100 20 50 70|_100 20 50 70 80|
+-----+-----+-----+-----+
```

and successive IRRs by

```
,irr each }.<\is
_80 _18.5857 15.6152 32.3121
```

leading to

```
irrseq=.,@:(irr each)@}.@(<\)
irrseq is
_80 _18.5857 15.6152 32.3121
```

Post-initial negative items in an income stream as in `isl` might reflect costs incurred in developing a second version of a product. Since polynomials of even order must have an even number of positive roots, using `p.` means that spurious values of IRR will necessarily arise, as for example in

```
irr _100 20 45 60 _10
_85.04 6.67385
```

where the massive compound 85% decay is clearly inadmissible. To deal with such circumstances it makes sense to insert a filter at the `mtop` level to exclude multipliers of greater than, say 3, or, at the other end of the scale, less than 0.5, hence

```

filter=#~ (<&3)*.(>&0.5)
irr=.mtop@filter@:pos@real@roots
irrseq isl
0 _22.18 10.22 6.674 19.96 27.96 29.33 29.82 30.19

```

The initial value of `_80` following the first positive flow has been filtered to 0. Following the third inflow of 60 the IRR becomes positive at 10.2%, following which there is a dip to 6.67 due to the negative inflow of 10. Towards the end of the series subsequent increases in IRR tail off as a result of smaller inflows, a signal that benefit is tailing off.

Annuities and Personal Finance

An annuity is a special case of an input stream where the inflows are regular. Annuities in general are a large field within actuarial science, although in the simplest case the calculation has a closed form in which the multiplier $\{1-(1+p)^{-n}\}/p$ converts repayments as a percentage of the initial cost into a multiple of present value assuming repayments continue for n years. In May 2014 this factor for a single whole-of-life level annuity at age 65 was around 17. Compare this with an annuity paid for 21 periods at 2.6% whose multiplier is given by

```

ptok=>:@*&0.01 NB. convert %age to (1+p/100)
am=.4 : '100*(1-(ptok x)^(~y))%x' NB. annuity multiplier
2.6 am 21 NB multiplier for ..
16.026 NB. .. 21 repayments at 2.6%
irr _16.026,21#1 NB. confirmed by IRR
2.6001

```

Apply a 5% inflation rate and the annuity doesn't look such a great bargain!

```

irr _16.026,5 npv 21#1
_1.8065

```

In the case of personal investment, IRR can give a rough guide to the true worth of an investment *on completion*. Consider a share purchase for £33.50 which attracts nine annual dividends of £2, £2, £2, £2.50, £3, £4, £4.45, £5.25 and £4.55, after the last of which it is sold a year later for £65.88.

```

is3=_33.5 2 2 2 2.5 3 4 4.45 5.25 4.55 65.88
irr is3
13.46

```

This indicates that over the period the investor has experienced a discount rate of about 13.5% but has no final asset – the inflows account for all benefit. However the average growth rates for 0 and 5% inflation are:

```
0 5 agr each <is3
11.06 6.508
```

indicating that under 5% inflation `is3` has an average profit-growth based return of 6.5%.

There are streams for which growth can be consistently negative as with

```
is4=_57.4 1.5 1.5 1.6 1.7 1.8 1.8 1.95 2 27.9
; (irr;5&agr)is4
_4.051 _7.289
```

Nevertheless because only the first item is negative, IRR is still an increasing quantity:

```
;irrseq is4
_65.84 _52.32 _41.93 _34.25 _28.03 _23.22 _4.051
```

Future Value Analysis

Future Value Analysis (FVA) is a variation of net present value which, for an income stream with a single initial outflow, expresses benefits valued at *today's* money :

```
ratio={[:@] % {:@npv      NB. find the scale-up factor which ...
nfv=.ratio*]@npv        NB. ...leaves final npv value unchanged
5 nfv is
_121.6 23.15 55.13 73.5 80
```

Since ratios of successive terms are unchanged there is no change either to IRRs :

```
,irr each (5 nfv is);5 npv is
26.01 26.01
```

For `is` and `is3` the net benefits valued at the end of the investment period assuming 5% inflation are

```
(+/5 nfv is), (+/5 nfv is3)
110.2 47.94
```

to be compared with values at time of investment of `+/each is;is3` which are 120 and 62.13 respectively. For `is3`, the 47.94 is made up of 36.63 in dividends and 11.31 profit on sale, all at today's values, representing a gain of 47.94/54.57 or 87.85%. The geometrical average of this is $9\%:1.8785$ which equals 7.3%. Three values for growth of this series have been obtained, 13.6% from `irr`, 6.5% from `agr`, and now 7.3% from `nfv`.

The need to understand measurement criteria should be clear!

In conclusion, the techniques described here barely begin to scrape the surface both of practical detail and of jargon. Legitimate further questions might concern how the initial outflow was acquired – if by borrowing this leads to further necessary refinement of the *irr* process. The important message is the usefulness of J in penetrating terminology and in understanding precisely the meanings of quantities which may be subtler than their names suggest.

Using J for Actuarial Applications Part 1: The Chain-Ladder Method

Jeremy Smith
(actuaryjeremysmith@gmail.com)

1. Introduction

This article is the first in a planned series. My goal is to illustrate how J can be used in an actuarial setting. Although I am a J enthusiast, I do not consider myself to be a highly skilled J programmer. I hope that by introducing these actuarial methods to the J community, my programs can be refined and simplified. All feedback is welcome!

All of the methods in this series will be related to property / casualty insurance (e.g. auto, home, workers compensation). I will attempt to explain the methods in an intuitive, informal manner. Readers wishing to follow a rigorous formal treatment are encouraged to consult “Stochastic Claims Reserving Methods in Insurance”[1]. For those readers interested in a “middle ground” textbook (comprehensive, detailed, yet not a formal mathematical treatment with proofs and theorems), “Estimating Unpaid Claims Using Basic Techniques” [2] should be valuable. Not only is [2] freely available online in PDF form, it is also a standard textbook used by the Casualty Actuarial Society for examination purposes.

2. General Description of the Chain Ladder Method

Suppose we have the 10 x 10 array displayed in Figure 1.

357848	1124788	1735330	2218270	2745596	3319994	3466336	3606286	3833515	3901463
352118	1236139	2170033	3353322	3799067	4120063	4647867	4914039	5339085	0
290507	1292306	2218525	3235179	3985995	4132918	4628910	4909315	0	0
310608	1418858	2195047	3757447	4029929	4381982	4588268	0	0	0
443160	1136350	2128333	2897821	3402672	3873311	0	0	0	0
396132	1333217	2180715	2985752	3691712	0	0	0	0	0
440832	1288463	2419861	3483130	0	0	0	0	0	0
359480	1421128	2864498	0	0	0	0	0	0	0
376686	1363294	0	0	0	0	0	0	0	0
344014	0	0	0	0	0	0	0	0	0

Figure 1

Actuaries call this type of array a triangle, for obvious reasons.

This triangle can also be found in Table 1 of “Distribution-free Calculation of the Standard Error of Chain-Ladder Reserve Estimates”[3] (p. 221). Each row can be thought

of as an accident year, e.g. the numbers in the top row all relate to auto accidents occurring in the first year. Let's say the first year is 2004 (10 years ago, as of this writing).

For accidents occurring in 2004, the insurance company paid out \$357,848 during 2004. The company paid \$1,124,788 in total during 2004-2005, and so on. By the end of 2014, the company paid \$3,901,463. Assume, for the sake of argument, that nothing ever gets paid after year ten. In that case, we know that \$3,901,463 is the final value for 2004. In actuarial parlance, \$3,901,463 is called the ultimate loss for 2004.

What's the ultimate loss for 2005? We don't know. The latest information we have is that the company paid \$5,339,085 between 2005-2014 for accidents which occurred in 2005. There's one more year to go before the payments "reach ultimate", as actuaries say. The ultimate loss must be estimated; we must guess a value for the blank cell to the right of \$5,339,085 in the second row. What to do?

You can do all kinds of things. But one simple thing you can do is consider the previous year, 2004. How much was paid out in the tenth year, as a percentage growth? Answer: $\$3,901,463 / \$3,833,515 = 1.018$. Using this information, we can estimate that the ultimate loss for 2005 is $\$5,339,085 \times (1.018) = \$5,433,719$. (Note that I am using the exact value $\$3,901,463 / \$3,833,515$ in the previous computation; rounding differences will result if you use the rounded value 1.018).

So now we have an ultimate value for 2005: \$5,433,719. Ultimately, this is the amount that the company will pay. Since the company has already paid \$5,339,085, the amount remaining unpaid is $\$5,433,719 - \$5,339,085 = \$94,634$. This amount is called the "reserve". It's how much the insurance company should set aside to pay future claims for year 2005, assuming it believes the method result.

This is what a reserving actuary does all day: estimates how much the company needs to set aside as a provision for individual accident years, and in aggregate across all years. It's a big challenge; many companies have failed after underestimating the required reserves. Many insurance companies have experienced stock price plunges after announcing reserve increases. It's a challenging task, and even the best possible reserve estimate may prove to be way off the mark as circumstances unfold. The actuary's responsibility is to calculate a reasonable estimate based on the information known at the time.

Let's continue our example to build our intuition of this method. What about 2006? \$4,909,315 has been paid between 2006-2014 on claims occurring in 2006. We have two years to go! But we know that cumulative payments grow by 1.018 in the final, tenth, year. So we just need to worry about the ninth year. What data do we have?

- For 2004 accidents, payments grew from \$3,606,286 to \$3,833,515 during the ninth year.
- For 2005 accidents, payments grew from \$4,914,039 to \$5,339,085 during the ninth

year.

We can estimate the ninth year growth factor by calculating a weighted average: $(\$3,833,515 + \$5,339,085) / (\$3,606,286 + \$4,914,039) = 1.077$ (rounded; actual value 1.07655...). Since payments will grow an additional 1.018 in year ten, in years nine and ten they'll cumulatively grow $1.077 \times 1.018 = 1.096$. Thus, the ultimate loss for year 2006 is $\$4,909,315 \times 1.096 = \$5,378,826$. (See the previous warning about rounding.)

Ultimate
3,901,463
5,433,719
5,378,826
5,297,906
4,858,200
5,111,171
5,660,771
6,784,799
5,642,266
4,969,825

Figure 2

By now you're probably getting the idea. This process can be carried out iteratively to calculate an ultimate loss for each year. It's pretty easy to set this up in Excel, which is what most actuaries do. If you use Excel regularly, you might want to try setting up the calculation as an exercise. (The data above can be found in an Excel spreadsheet at the link given in "Mack-Method-handout.xls" [4]). If you do it right, you should get the values displayed in Figure 2.

Two more vocabulary terms. Factors like 1.018 and 1.0177 (i.e. incremental, year-to-year factors) are called Age-to-age or ATA factors. Cumulative factors like 1.096 are called Age to ultimate or ATU factors. Other names are used as well, e.g. link ratios, cumulative development factors (CDF), loss development factors (LDF). They're all acceptable but some are ambiguous (e.g. link ratio might refer to ATA or ATU factors).

3. Why J?

The Excel approach is easy enough and gets the job done, so who needs J? First, an Excel implementation of the Chain Ladder method can be a hassle to modify if the triangle changes size (say, if you suddenly get twenty years of history instead of ten). What if you have a lot of triangles, of various sizes, and need to quickly calculate the Chain-Ladder ultimate losses for all of them? This is where J shines and Excel falters.

The J code

In general I find it convenient to export triangles into boxes and remove the zero values, because they cause all sorts of headaches. I use the following two functions extensively:

```
exportboxes=<"I@({~ i.,@#)@|:
```

```
removezeros=(#~(::~&0)) (&.>)
```

See Figure 3 for an illustration. I call our triangle `Tri` in this example.

```
,. exportboxes Tri
```

```
|357848 352118 290507 310608 443160 396132 440832 359480 376686 344014
```

```
|1124788 1236139 1292306 1418858 1136350 1333217 1288463 1421128 1363294 0
```

```
|1735330 2170033 2218525 2195047 2128333 2180715 2419861 2864498 0 0
```

```
|2218270 3353322 3235179 3757447 2897821 2985752 3483130 0 0 0
```

```
|2745596 3799067 3985995 4029929 3402672 3691712 0 0 0 0
```

```
|3319994 4120063 4132918 4381982 3873311 0 0 0 0 0
```

```
|3466336 4647867 4628910 4588268 0 0 0 0 0 0
```

```
|3606286 4914039 4909315 0 0 0 0 0 0 0
```

```
|3833515 5339085 0 0 0 0 0 0 0 0
```

```
|3901463 0 0 0 0 0 0 0 0 0
```

```
,. removezeros @: exportboxes Tri
```

```
|357848 352118 290507 310608 443160 396132 440832 359480 376686 344014
```

```
|1124788 1236139 1292306 1418858 1136350 1333217 1288463 1421128 1363294
```

```
|1735330 2170033 2218525 2195047 2128333 2180715 2419861 2864498
```

```
|2218270 3353322 3235179 3757447 2897821 2985752 3483130
```

```
|2745596 3799067 3985995 4029929 3402672 3691712
```

```
|3319994 4120063 4132918 4381982 3873311
```

```
|3466336 4647867 4628910 4588268
```

3606286 4914039 4909315
3833515 5339085
3901463

Figure 3

Now we’re going to use J’s power to calculate all the ATA factors are once.

```
NB. numerator of the ATA factor calculation
CLnum=: >@:(+/ each @:).@:removezeros@:exportboxes)

NB. denominator of the ATA factor calculation
CLden=: >@:(+/ each @:():@:(): each @:(removezeros @: exportboxes)))

CLATA=: CLnum % CLden

CLATU=: |.@:(*/\@:(|.@:CLATA))
```

Figure 4 displays the results of these functions. The ATU factors can be checked against the spreadsheet example given in “Mack-Method-handout.xls” [4], column N (where they are labeled LDF; see section 2 above regarding varied nomenclature).

CLnum Tri
11614543 17912342 21930921 21654971 19828268 17331381 13429640 9172600 3901463
CLden Tri
3327371 10251249 15047844 18447791 17963259 15954957 12743113 8520325 3833515
CLATA Tri
3.49061 1.74733 1.45741 1.17385 1.10382 1.08627 1.05387 1.07656 1.01772
CLATU Tri
14.4466 4.1387 2.36858 1.6252 1.3845 1.25428 1.15466 1.09564 1.01772

Figure 4

Now we’ll calculate the ultimate losses. First we define a simple utility function which pulls the diagonal off the triangle, except for the top value (which we’re not going to develop, because it has already reached year ten).

```
diagonal=: > @: ((): each @: removezeros @: exportboxes)

diagonal Tri

344014 1363294 2864498 3483130 3691712 3873311 4588268 4909315 5339085 3901463
```

Figure 5

To calculate the ultimates, we multiply the diagonal by the ATU factors, and append the fully-developed 2004 value.

```
CLUltimates=: diagonal * (1,~CLATU)
```

The reserves are the ultimates minus the diagonal.

```
CLReserves=: CLUltimates - diagonal

CLUltimates Tri
4.96982e6 5.64227e6 6.7848e6 5.66877e6 5.11117e6 4.8582e6 5.29791e6 5.37883e6
5.43372e6 3.90146e6
(CLUltimates Tri)
5.30389e7

CLReserves Tri
4.62581e6 4.27897e6 3.9283e6 2.17764e6 1.41946e6 984889 789638 469511 94633.8 0
(CLReserves Tri)
1.86889e7
```

Figure 6

The values for the ultimates and reserves can be crosschecked against the spreadsheet example in “Mack-Method-handout.xls”[4] (columns P and O, respectively). The reserves can also be found in the first column of Table 2 of [3], page 221.

4. Conclusion

We have coded J functions, `CLUltimates` and `CLReserves`, that allow us to compute the ultimate losses and reserves, according to the Chain Ladder Method, for each year of a loss development triangle. J’s versatility allows us to compute these values for a large number of triangles at once, of various dimensions. A straightforward way to do this would be to organize the triangles into a master array of boxes, one triangle per box, and run the command `CLUltimates each triangles` (where `triangles` is the array of boxes).

The Chain Ladder method is one of the most basic methods available to Property/ Casualty actuaries. Over the years, many more sophisticated methods have been developed. In future articles I hope to illustrate more of these methods in J.

I welcome all comments regarding this article, and am especially eager to receive suggestions on how the code could be improved.

5. References

1. Wüthrich, Mario V., and Michael Merz, “Stochastic Claims Reserving Methods in Insurance,” Wiley Finance, 2008
2. Friedland, Jacqueline, “Estimating Unpaid Claims Using Basic Techniques”, Casualty Actuarial Society, 2010
3. Mack, Thomas, “Distribution-free Calculation of the Standard Error of Chain-Ladder Reserve Estimates”, ASTIN Bulletin 23/2, 213-225
4. <https://www.casact.org/education/spring/2006/handouts/Mack-Method-handout.xls>

kdb+ & q

Book review

Q Tips: Fast, Scalable and Maintainable Kdb+

by Yang Guo (yang_guo@mfe.berkeley.edu)

For most people who approach a new language, the first question asked usually isn't about syntax, features and functionality, but rather "what can it do". Q, being as terse as it is, has relied on snippets, anecdotes and reference guides to try to answer this question, however there so far has not been a very satisfying answer. Whereas other languages have tutorials that showcase its strengths, q has not had a comprehensive, clear answer to "what can it do". Nick Psaris's book *Q Tips* aims to be one answer for this question, with a very obvious focus towards quantitative analysts. The overall premise of the book is already grand in its nature – the user should have a fully working complex event processor (CEP) by the end of the chapters, however the most impressive aspect is the seamless interweaving of usable code snippets, introductions to basic language features and building the codebase up in a well-paced and disciplined fashion (not to mention a github full of examples from the book).

Another noteworthy aspect of the book is that it doesn't treat q purely as a database language but rather a data analysis language. This is a big shift from common perception, as kdb+/q is often compared to SQL and NonSQL databases rather than data processing languages like R and Python. *Q Tips* tries to show this often-neglected side of the language, giving examples that belong both in a library for data and quantitative analysts in addition to database administrators and developers performing operational support. Not only will you learn how to create and manage various database types available in kdb, you will also know how to create frameworks for aggregating data, multiple ways of summarizing results and a very sophisticated method of creating simulated time series.

Getting started and generating data

Even from the first steps, *Q Tips* makes no assumption that the reader has expertise in running Q and diligently walks through every aspect of starting an instance. From the detailed explanation of reserved keywords to an explanation of the banner itself, the book leaves no stones unturned. Nick goes through common shortcuts seasoned users of the language take (such as creating aliases), and even shows some tips that even seasoned q users may not know, such as piping input into the Q instance on startup. The basics of the language are introduced clearly and concisely; Nick clearly observes the "show not tell" philosophy as after each brief description is an example that clearly shows each language feature.

Even in this section, before the language has been formally introduced, Nick is already presenting usable examples. For example, in showing the difference between q and k, he

shows how easy it is to write a percentile function, something that will be used later on in developing the CEP. Although the example is simple, it clearly showcases the difference between `q` and `k`, as well as giving the user a taste of the power of the language.

This method of concise explanations with extremely helpful code examples carries on to the introduction of basic data types of the language. Again, with the quant emphasis in mind, Nick's examples go beyond toys and introduce practical, usable code. A particularly interesting example in this chapter deals with random selection with and without replacement – in two lines of code we see how easily it is to define a deck of cards as well as deal it out. The concept itself is useful (e.g. a poker simulator), and can be extended to any problem that requires creating a set of unique items and dealing them.

Functions, code organization

With the basics of the language out of the way, Nick shifts gears and starts delivering on very common functions needed in almost any type of simulation based setting. While introducing `q` as a functional language, he uses two methods of generating normal random variables (Box-Muller and Beasley-Springer-Moro) from uniform random variables as an example of `q` functions. In this chapter, we also start to see core programming concepts such as error handling, vital in creating robust production code.

This is carried on into the chapter on code organization, where Nick goes into detail about code organization, libraries and how `q` interacts with its outside environment. These details are very important when it comes to deploying and maintaining code, and until now I have not really found a resource that describes it so comprehensively. From this chapter it's very apparent that the book is written by someone with experience deploying production `q` code. One particularly clever example for namespace scoping is the `.util.use` function, which allows the user to move a function from a namespace into global scope.

After these chapters, the user has already seen ample examples of the basics of the language, and has multiple working statistics functions and utilities in their toolbox!

A Random walk and tables

One of the powers of `q` is treating data sets as tables, and allowing for easy creation, updating and manipulation of these tables, far beyond what traditional SQL can offer. In these chapters, Nick goes through the most important aspects of creating lists of data and converting them into tables. `Q Tips` take an extremely practical approach again with these chapters, showing off the language using examples that seasoned `q` users have used on the job often. More importantly, he focuses on small but critical parts of data manipulation, often overlooked but significant pain points in data analysis. From reversing a table to

rearranging columns, these examples are explained intuitively and builds upon the previous chapters.

The chapter on large tables is probably the most ‘traditional’ chapter in the book, dealing with the bread and butter of q. In this chapter Nick goes through two core concepts of the language – caches and attributes. While these might not matter too much to a new user, they are absolutely critical when writing optimized q code for large tables, where the difference in algorithmic efficiency is clearly visible.

Trades and quotes and CEP engine components

Even though we have already learned a massive amount, and have been exposed to some very critical code pieces in the previous chapters, this is technically the first non-introduction chapter – which is a feat in itself as we already have enough working knowledge of the language to go do something useful. This chapter is extremely valuable for anyone who has ever wanted to run sensitivity analysis or a strategy backtest, as we literally build a full quote and trade time series using the knowledge we acquired in the previous chapters. The chapter describes generating a very realistic quote and trade table, and shows off some very cool (and fairly unknown) language features. Two particularly interesting features of the tables Nick creates are varied tick sizing depending on security price, using a clever language feature that overloads the sorted attribute, and randomly delaying simulated trades based on quotes that is more in line with what is actually observed in the market. As with the rest of the book, these features are outlined clearly and the code to take the function from an idea to full implementation is laid out step by step. By the end this chapter the reader has a fully simulated and extremely extendible quote and trade table, with timestamps down to the nanosecond, something that is extremely hard to do in most other data processing language!

The next chapter switches back to more of the developer operational pieces by defining a very flexible timer, adding in logging and finally parsing command line arguments. Again, these are absolutely critical in a production environment, and Q Tips is really showing the user how a production level product could and should be built. By the end of this chapter we have all the raw components for running a CEP engine.

Running a CEP engine, security and debugging

The culmination of the previous chapters all lead up to this – combining the skills we have gained in starting/stopping a q process, creating modular functions and creating simulated high frequency data, we can now build our own CEP. As with many other learning experiences, I feel that the journey is more important than the destination, but it is very satisfying to see all the previous chapters combined into one very tight and robust system. This chapter also includes information on loading data from files and the various ways to do so, which is useful for any practical data analysis.

Q-SQL

As I have mentioned before, it is very obvious that Nick sees q as much more than just a query language – the first mention of Q-SQL being in chapter 14 of the book is testament to this. That being said, this is probably one of the best treatments of Q-SQL I have seen so far as well. As with the rest of the book, Nick takes the practical route with the introduction of the syntax and examples, and identifies pain points that many new users have as they approach the language. This goes beyond the normal treatment of Q-SQL, as exemplified in the section about `exec` as well as pivot tables. In here, Nick really digs into how `exec` works, as well as the relationship between a `select` statement and an `exec` statement. His examples on creating a simple `ohlc` table using `exec` versus `select` truly shows off the language's power, as it's easy to see how easy it is to generalize summaries across arbitrary data sets and with any arbitrary metrics in an extensible fashion.

In addition, his section on pivots is probably my favorite section in this book. For many areas in data analysis, a pivot table is one great way to generate a fast set of results to ingest before doing more complex analysis. Many languages have built-in pivot functions, however since they tend to be fairly rigid in functionality you are locked into whatever functionality they provide. Nick shows off the flexibility of q by going through multiple iterations of building a pivot function, and then finally creating one that is simple to use and easily extensible, giving the user far more control than any stock function allows.

The final section on joins shows off the various ways that kdb allows you to join data. While this chapter is as well written as the previous ones, I feel this is the most covered topic for kdb+/q documentation. One item to note is the emphasis on time-based as of (`aj`) and window (`wj`) joins as opposed to the more regular ones (union, left and each join). For time series analysis I find myself using `aj` more so than any other type of join, and a detailed coverage of this is very welcomed.

Big data and beyond

The rest of the chapters talk about more domain specific problems such as partitioning, compression, remote access and custom aggregation. These chapters round off the knowledge needed to create something in q, and give the background knowledge for making an informed decision of how to build out a database as the need for larger data sets arises

Conclusion

By the end of the book you will have taken a giant leap towards being a seasoned q expert. Not only will you be now well equipped to deciding how to process and store data, but you will also know many intricate details of the language such as how to best use `exec` statements (giving you far more flexibility than SQL), how to combine data to make

analysis efficient and how to write modular statistical functions to help in statistical analysis and simulations. Most importantly, however, you will know how to think in the ‘q way’ – with transparent, terse code, modularity and flexibility through projections and each statements as well as very purposeful decisions in terms of data attributes. You will find that when you ask yourself “what can it do” when it comes to q, you can answer this question not just once, but in multiple ways. Just remember to keep it terse.

Q Tips: Fast, Scalable and Maintainable Kdb+

Published by: Vector Sigma, Hong Kong SAR (2015)

ISBN: 978-988-13899-0-9

<http://q-tips.net>

Letting data tell a story with kdb+: what makes a good tennis match?

Matthew Doherty & Paul Loughran

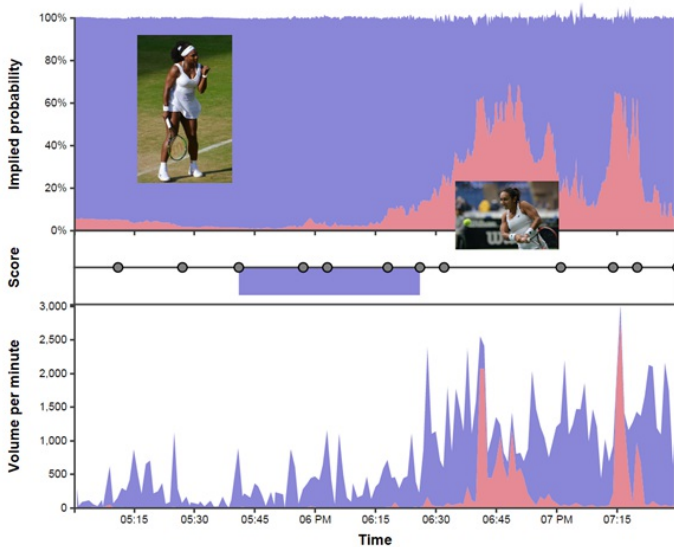


Figure 1 – Using

data to tell the story of a good game of tennis.

Introduction

In tennis the addict moves about a hard rectangle and seeks to ambush a fuzzy ball with a modified snow shoe. In q the addict moves algorithms from their mind to the central processing unit in as few ASCII characters as possible and seeks to puzzle both fellow programmers and their future selves.

kdb+ and q sometimes have a reputation as a difficult language to learn, in part due to the terse and sometimes intimidating syntax. So why do so many trading rooms across the world rely on this technology to handle huge real-time flows of financial data? There are two main reasons:

- Speed: its column oriented nature makes it very fast at the kind of calculations necessary in trading applications.
- Scope: kdb+ is both a programming language and a database, so the majority of an application – from data capture and storage through to real-time and historical analysis – can all be done in the same technology.

Like many APL heritage languages, q makes the barrier between thinking of a solution to a

problem and then implementing it very narrow. The terse syntax represents a trade-off: in exchange for a slightly steeper learning curve we get an incredibly expressive high-level tool for solving problems.

So, what was all that stuff about tennis then? `kdb+` is a full database system built on top of the `q` language, and its main application is as a system for the capture, storage and analysis of timeseries data. In this article we're going to describe an interesting and slightly out of the ordinary application of `kdb+`: the collection and analysis of sports exchange data. Specifically, we're going to look at in-play betting data captured from `betfair.com`[2] during the oldest tennis tournament in the world, the 2015 lawn tennis championships at Wimbledon.

For the uninitiated, sports exchanges are similar to traditional bookmakers in that they offer odds for the outcomes of events, but with one fundamental difference: they act like a marketplace – matching one user with another – rather than taking any positions themselves. They essentially operate in the same way as financial exchanges such as the LSE and NASDAQ, but trading bets instead of securities. Data from sports exchanges are a deeply fascinating treasure trove, rich with information, so here we're going to demonstrate a way to capture this data and use the analytical power of `kdb+` to glean some insights (and hopefully have a little fun).

Data capture & TorQ

The `kdb+` language is very small: the download from `kx.com` of the most recent version comes in at less than 300KB. In addition to the core language, since we want to run a complete data capture and analysis system without reinventing the wheel, we have used a framework. We chose to use the open-source TorQ from AquaQ Analytics which gives us the basis of a production `kdb+` system by adding some core functionality and utilities on top of `kdb+`, allowing us to focus on application specific data capture and analysis logic. The basic TorQ architecture is shown in Figure 2, with the additional data connector (or feedhandler) for `betfair.com`[2] which we've added. The feedhandler we've written for this application retrieves live trade and quote data from `betfair.com` via the JSON API-NG. It handles authentication, parses the returned JSON data, joins the appropriate metadata and pushes data to the tickerplant. We can gather data for multiple “markets” – in this case tennis matches – at set intervals of our choosing.

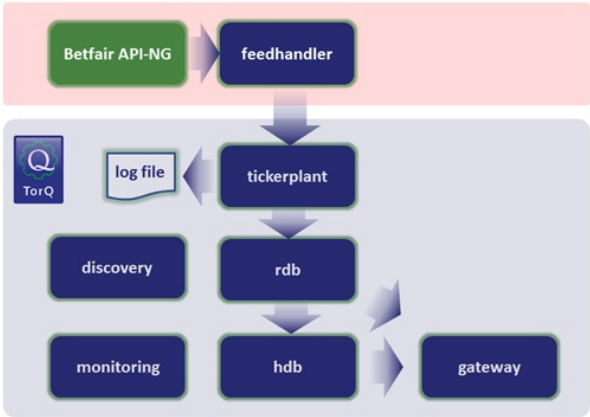


Figure 2 – TorQ architecture for capturing data from betfair.com. Our custom feedhandler gathered data from the API-NG, which was then stored and queried in the TorQ architecture.

The data we capture is stored in four main tables, with the schema and some sample data shown in Figure 3. So let’s explain what we have here a little. The columns sym and selectionId are identifiers for the market and outcome i.e. the tennis match and the winning player. In the marketstatus table we have updates on the status of particular matches; when betting starts, when the market is in-play and when the match finishes and the market closes. The trade table contains information on matched trades and their size, while the quotes table contains market depth. In betting parlance back means you’re betting on a particular outcome happening, while lay means that you’re betting against that outcome. On a sports exchange backs are matched to lays in the same way bids are matched to offers in a traditional financial exchange. In the case of tennis where there are only two outcomes backing one is the same as laying the other. Finally the activeDates table contains information on which markets were traded on which dates. This is important as kdb+ stores data to disk on a daily basis, so by knowing which date partitions to look at for a particular match we can improve performance significantly.

marketstatus							
date	time	sym	status	inplay			
2015.07.07	2015.07.07D00:00:02.365599000	1.117087478	OPEN	0			
2015.07.07	2015.07.07D00:00:02.365599000	1.119377980	OPEN	1			
2015.07.07	2015.07.07D14:19:38.780988000	1.119377980	SUSPENDED	1			
2015.07.07	2015.07.07D14:22:53.833878000	1.119377980	CLOSED	1			

trade						
date	time	sym	selectionId	price	size	
2015.07.07	2015.07.07D00:25:14.796876000	1.117087478	1108296	2.7	20.51	
2015.07.07	2015.07.07D00:25:14.796876000	1.117087478	1108296	2.72	16.66	
2015.07.07	2015.07.07D00:25:26.173638000	1.117087478	1108296	2.68	17.8	
2015.07.07	2015.07.07D01:18:58.777020000	1.117087478	1108296	2.68	1.81	

quote							
date	time	sym	selectionId	backs	lays	b sizes	l sizes
2015.07.07	2015.07.07D00:00:11.968120000	1.117087478	1108296	2.7 2.68...	2.8 2.86...	10251625...	1.99 23.18...
2015.07.07	2015.07.07D00:00:23.358820000	1.117087478	1108296	2.7 2.68...	2.8 2.86...	10251625...	1.99 23.18...
2015.07.07	2015.07.07D00:00:34.739127000	1.117087478	1108296	2.7 2.68...	2.8 2.86...	10251625...	1.99 23.18...
2015.07.07	2015.07.07D00:00:46.103207000	1.117087478	1108296	2.7 2.68...	2.8 2.86...	10251625...	1.99 23.18...

activeDates	
date	marketids
2015.07.04	'1.117087478' '1.119324528' '1.119344738' '1.119345150...
2015.07.05	'1.117087478' '1.119222011' '1.119222015
2015.07.06	'1.117087478' '1.119366605' '1.119377463' '1.119377546...
2015.07.07	'1.117087478' '1.119377980' '1.119390975' '1.119390992...

Figure 3 –

schema of our tables with a few rows of sample data

So using our system we are able to capture the complete state of the betting market for multiple matches at set intervals, and have the live data available for querying along with the complete history which is persisted to disk at the end of each day. This data is particularly interesting as the state of the market at any given point tells us what the market thinks the chance of all given outcomes are at that time. We can look into the collective mind of all sports bettors! We pointed this system at each game in the Wimbledon championships where match odds were available and collected market snapshots every fifteen seconds. This is a tremendously rich dataset, so the question is what are we going to do with all this data?

What makes a good game of tennis?

One thing we thought could be interesting to figure out – purely from this live betting data – is which games were the best. So what makes a good game of tennis (or any sport)? What makes a game exciting to watch? The question is almost philosophical in nature, and

of course we can't fully answer it, but we can make an attempt! Obviously if you have a personal stake in the match – if it's your country/team/favourite player – you're going to enjoying watching more. Betting data can't tell us anything about this. What it can do however, is give some insight into which games are the most interesting for neutral watchers. Fellow blogger Todd Schneider[1] asked the same question, and proposed the following formula to answer it:

$$hotness = \frac{\sum_{t=2}^T (IP(t) - IP(t-2))^2}{T}$$

In q, once we have a timeseries of implied probability for a given event, this calculation can be written concisely as:

```
select hotness:(avg xexp[;2] IP - xprev[8] IP)%`time$max[time]-min[time]
from odds
```

We have a timeseries with a frequency of fifteen seconds so we can use `xprev[8]` to easily compare with odds two minutes in the past. So what does all this mean? This equation assumes that if the odds move a lot during the match then the match is more exciting. So a closely fought back-and-forth game where the outcome is uncertain gets a higher “hotness” score than a match where a favourite walks to an easy straight sets victory. From our quotes table we can calculate the mid-price at any given point i.e. the market agreed fair odds for an outcome. From these odds we can calculate the implied probability (IP) of this outcome i.e. according to the odds what is the chance of this player winning? Now, to calculate “hotness” we take the IP of a given outcome at some point during the match and the IP of the same outcome two minutes later, square it, and sum this number for every point throughout the match. This gives us a good measure of how much the odds were moving during the game. We add constant T to adjust for the length of the match and we have our hotness rating! Obviously hotness is only an estimate of how good a match was and doesn't include many important factors, but it will at least give us some idea of how interesting a game is, and it provides a nice demo of using `kdb+` to ask a question of a dataset!

```
getHotness:{[id]
  dates: exec date from activeDates where any each marketids in\: id;

  / find the volume traded for each market (outcome) of each game
  sizesbyselectionandmkt:sum {[x;y]
    select sum size by sym, selectionId from
    ajoin `sym`time;
    select time, sym, selectionId, size
    from trade
    where date = x, sym in y;
```

```

        select from marketstatus where date = x]
        where inplay, not status = `CLOSED
    }[,id] peach dates;

/ which market had the biggest volume in each game?
highestvolselection:select sym,selectionId
    from sizesbyselectionandmkt
    where size = (max:size) fby ([ sym];

/ find the live odds for that market
odds:raze {[x;y]
    select from
    aj['sym`time;
        select time,sym,back:backs[,0],lay:lays[,0]
        from quote
        where date = x, ([ sym;selectionId) in y;
        select from marketstatus where date =x]
    where inplay, not status = `CLOSED
    }[,highestvolselection] peach dates;

/ find the mid and implied probability from the quotes data
odds:select time, sym, IP: 100 * 1 % mid
    from select time, sym, mid: avg each flip (back;lay) from odds;

/ and calculate the hotness!
hotness:select hotness:(avg xexp[,2]IP-xprev[8]IP)%`time$max[time]-min[time]
    by sym from odds;
:hotness;
};

/ sample function execution
getHotness['1.117087478`1.119324528`1.119344738 ...]

```

Figure 4 – kdb+ code to query our tables and return the hotness for each event. The id arguments represent individual events and have the form R.ID, where R is a region identifier and ID is an event identifier.

Before we can apply the formula above we must first process our raw odds data. The code in Figure 4 demonstrates the full procedure to derive the hotness score of a given list of distinct markets from our data tables. It might look somewhat dense at first glance, but it's doing quite a bit of analysis for us. For each match:

- first we determine which market was the most active

- retrieve the odds from this market while this game was being played
- calculate the implied probability from these odds
- calculate the hotness for this match

We take advantage of some of the unique features of `kdb+`, one of which is the `asof` join (or `aj`). This is a special type of timeseries join which is used to join on event data as of a given point in time. A typical example of this in financial data is joining on quote data to trade data, an `aj` can be used to join the prevailing quotes as of the time a trade has been executed. In our case we have used an `aj` to quickly filter out only trades and quotes that were published whilst the market was in-play i.e. we joined on the market status as of the time of our odds data. Another useful function utilised in the code is `fby`, which is short for function by. This has been used to identify the outcome on each market which had the largest volume of trades executed, the most traded outcome should make for the best quality (least noisy) data set from which to derive the hotness score. The `fby` makes it possible to identify this within a single select statement as opposed to using one select statement to find the total traded for each market and outcome and then a further select statement to determine the highest traded outcome within each market. The calculation of the traded volumes and implied probability makes use of the multithreaded capabilities of `kdb+`. Queries which run over a number of partitions in a historical database (HDB) are ideal candidates to be executed in parallel (even more so if the database has been segmented over multiple disks with separate I/O controllers). This is done by writing a `lambda` which is executed for a list of dates using the `peach` adverb. Once we have extracted our timeseries of quotes data for the most highly traded outcome for each market, we calculate the mid-price and from this the implied probabilities. Finally hotness is calculated in a single line (as shown previously) by taking the implied probabilities for each market and then applying Schneider's formula.

So we collected live odds data for 427 matches played at Wimbledon; a total of well over ten million data points. The algorithm shown above can calculate the hotness for all these games in just over one second. For many of these games the markets were not very active – not many people were betting – so the live odds may be noisy due to the low volumes. If we filter out matches where less than £1 million was matched, we're left with 121 matches. For reference the most popular game of this year's championships – the final between Novak Djokovic and Roger Federer – saw over £30 million matched on in-game betting. So you could say betting on tennis is fairly popular! Ranking these high-volume games by "hotness", we can see what the best and worst matches of this year's Wimbledon were! The top and bottom games are shown in Figure 5.

Rank	Match
1	Bethanie Mattek-Sands vs. Ana Ivanovic
2	Karolina Pliskova vs. Coco Vandeweghe
3	Sloane Stephens vs. Lucie Safarova
4	Benoit Paire vs. Roberto Bautista Agut
5	Sara Errani vs. Aleksandra Krunic
...	
117	Novak Djokovic vs. Marin Cilic
118	Sam Groth vs. Roger Federer
119	Roberto Bautista Agut vs. Roger Federer
120	Novak Djokovic vs. Bernard Tomic
121	Andy Murray vs. Vasek Pospisil

Figure 5 – the best and worst games according to their ‘hotness’ at the 2015 Wimbledon lawn tennis championships

According to our algorithm the best tennis match at Wimbledon this year was between Ana Ivanovic and Bethanie Mattek-Sands in the second round, where the world number 175 knocked out the ex-world number one and Wimbledon semi-finalist. The worst game was the quarter final matchup between Vasek Pospisil and our very own Andy Murray, where the Scot won 6-4 7-5 6-4, breaking his opponents serve exactly three times: enough to win the match and no more. So the good news is it seems to work! These games seem like reasonable candidates for the best and worst games played this year. Another notable observation from this result is that most of the best games are women’s while the worst are men’s; this could be because men’s games are longer at five sets and therefore less susceptible to upsets and sudden shifts in the odds, or simply a reflection of the fact that men’s tennis is generally more predictable than women’s. It is worth noting that while these calculations were run on data after collection, our architecture allows us to run it just as easily on live data as it is collected.

Telling a story with the data

So we’ve got some idea now of which games were the best and worst, but what actually happened in those games? What do they look like? We were able to use the data we’ve collected to distil the game down to a single number – the hotness – but we can also use this data to tell the story of those games, which is nice because I’m sure all bar the most avid tennis fanatics can’t remember them!

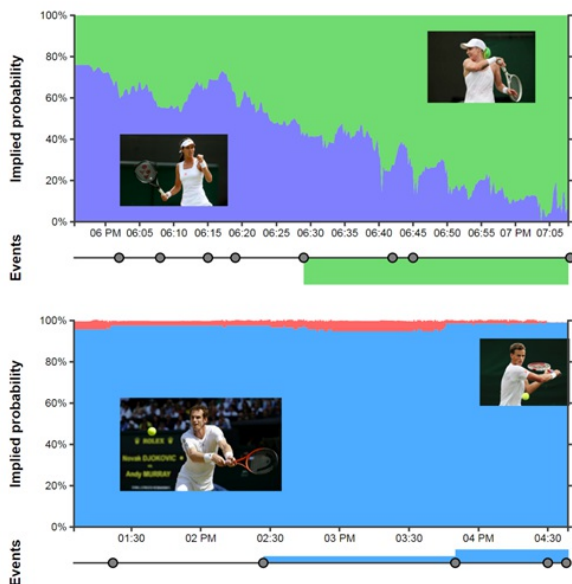


Figure 6 – The story of the

best and worst games, told using betting data.

So in Figure 6 we can see a time-series of the implied probability of each player winning throughout the game. We can see these odds move in response to various events in each game such as break points. Our boring game looks somewhat like a piece of modern art: just one big block of colour! On the other hand the game between Mattek-Sands and Ivanovic was clearly a closely fought, back and forth affair. Each shift in odds corresponds closely to an in game event: for example at around 06:15 Ivanovic breaks serve, but around four minutes later Mattek-Sands breaks straight back. Similarly we can see the three occasions Andy Murray broke serve as very small blips in the odds (if we squint and look very close).

The stack graphs plot the chance of each player winning at each time during the match. In the top plot Mattek-Sands is in green, while Ivanovic's win chance is in purple. On the bottom Murray is in blue while Posposil is in red. The events plot underneath each graph shows the set score and some key events that happened during the game (see interactive graphics [5])

If we look back to Figure 1 we can also see the story the data tells for one of the most interesting games in the championship. The nail-biter between perhaps the best ladies player of all time – Serena Williams – and the British hope Heather Watson. This game ranked 16th in hotness, but the added interest due to a home player doing so well is not really captured in the data. We can see the game start of looking a lot like Murray vs. Pospisil, but Heather starts to make herself heard in the second and third sets, setting up a

nail biting finish.

This use case of capturing betting data is a nice example of the main two steps typically involved when using `kdb+` in any business application:

- develop a data connector specific to the data source, in this case a connector for the betfair API-NG
- add business logic to our gateway to ask question of our data, in this case our hotness calculation

This standard `kdb+` framework allows us to easily take advantage of the disaster recovery, system monitoring, unified on-disk and in-memory data access via the gateway, and a range of other functionality. in our business application. If any readers are interested in reading more on the TorQ, or this framework for collecting sports exchange, both are open-source and available on [github](#)[3][4].

References

1. Schneider, Todd W., “What Real-Time Gambling Data Reveals About Sports: Introducing Gambletron 2000”
<http://toddwtschneider.com/posts/what-real-time-gambling-data-reveals-about-sports-introducing-gambletron-2000/>
2. <http://www.betfair.com>
3. <https://github.com/AquaQAnalytics/TorQ>
4. <https://github.com/picoDoc/betfair-data-capture>
5. Interactive graphics
<http://www.picodoc.org/wimbledon-2015-the-best-and-worst-games/>

A graphical sandbox for K

John Earnest (johnearnest@gmail.com)

Editors note: In the online version of this paper most of the graphic images are animated. See archive.vector.org.uk/art10501610.

This article describes *iKe*[1], a browser-based environment built on the K programming language for creating interactive, animated applications. It discusses the motivations and design considerations behind the project, and walks through the process of developing a simple program using the tool.

Background

K has a well-earned reputation for high performance data analysis among the financial community. While K 2.0 was packaged with a facility for creating graphical user interfaces (GUIs), newer releases of the language have moved away from directly supporting user-facing or interactive applications.

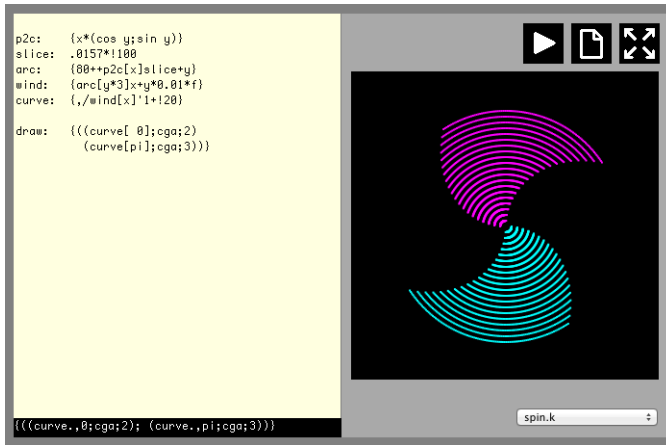
In December of 2014, I began work on an open-source implementation of a K interpreter[2] intended to be compatible with the as-of-yet unreleased K5[3]. My principal goal in this project was to attempt to make K accessible to a broader audience, lowering the barriers to entry. With this in mind, I chose to write my interpreter in JavaScript, allowing anyone with a web browser to follow a hyperlink and immediately begin experimenting with the language. I carefully separated all IO capabilities from the core interpreter, permitting the creation of multiple frontends to suit different applications, starting with a command-line REPL and followed quickly by an HTML-based user interface for the web.

K's ability to concisely express interesting programs struck me as uniquely well-suited to *Live Coding*, in which programmers interactively and improvisationally develop audiovisual programs, sometimes before an audience. I was also inspired by *Forth Haiku*[4], a site which allows users to write and share tiny programs written in a dialect of Forth augmented with the ability to render graphics and animations.

Array languages are at their best when operating on large chunks of contiguous data, and many livecoding related tasks (transforming screen coordinates, physics simulations, audio synthesis, etc.) fit well into this mold. By augmenting K with appropriate IO functions it is possible to create a fun and engaging tool for exploring topics that are far outside its usual niche.

The Tool

The iKe user interface consists of a text editor pane on the left and a preview region on the right. As programs are modified, pressing shift and the return key simultaneously will re-evaluate the contents of the text editor. If a smaller region of the program is selected, this will instead only evaluate that portion of the program. The result of any evaluated expression will be displayed in a status bar below the text editor. A programmer can flexibly use these facilities to peer into the data of a running program or redefine procedures without restarting execution.



The iKe user interface

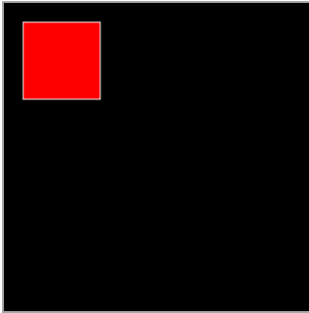
On the upper right are a series of buttons to toggle execution, share the current program or toggle fullscreen mode, respectively. Sharing a program makes use of the *Github Gist*[5] system to create a URL that can be distributed to other users which will automatically run the current program and permit modification of a copy. The ease of sharing and building on programs created in this way helps iKe serve as an ambassador for array languages.

In addition to the standard collection of K operators, iKe includes predefined procedures and constants which are useful for experimentation, including a Perlin Noise generator, a full set of trigonometric functions and their hyperbolic counterparts, a set of predefined color palettes and a built-in bitmapped font. iKe exposes a variety of callbacks for responding to keyboard and mouse input, as well as a generic animation timer. Output can come in the form of drawing filled polygons or paletted bitmaps, or playing audio samples.

Developing a program

Drawing in iKe always comes down to producing tuples. The structure of these tuples determines their interpretation. If a tuple contains two elements- a list of drawing coordinates and a list of colors- it will be drawn as a closed, filled polygon. To begin with, let's draw a red square near the top left corner of the display:

```
,((10 10;50 10;50 50;10 50);("white";"red"))
```



Drawing a fixed polygon.

As you can see, the first color is used as an outline color, and the second as a fill color. iKe permits any color format which browsers recognize, giving us a great deal of flexibility and a wide range of built-in colors to choose from. The drawing coordinate system counts left to right and top to bottom along the X and Y axes, respectively. The dimensions of the display are 160x160 pixels by default, and the constants w and h can be used to query these dimensions.

Let's make a general procedure for building regular polygons with a given number of sides. The simplest approach is to generate a range of fractions of 2π . π is another handy predefined constant:

```
{ (2*pi%x) *!x}4
0 1.5708 3.1416 4.7124
```

We can then convert these angles from normalized polar coordinates to cartesian coordinates. Pick your favorite factoring:

```
{ (cos;sin) .* \: (2*pi%x) *!x}4
(1 0 -1 0
0 1 0 -1)

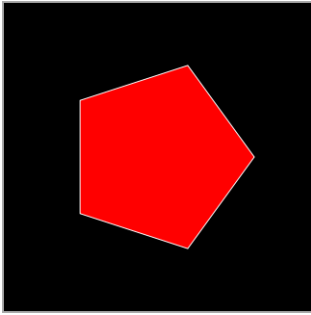
{{ (cos x;sin x) } (2*pi%x) *!x}4
(1 0 -1 0
0 1 0 -1)
```

Finally, scale these points to an appropriate radius and translate them to the center of the display. We'll also need to take the transpose of our result to get coordinate pairs in the order iKe expects:

```
{+80+50*{ (cos x;sin x) } (2*pi%x) *!x}4
(130 80
80 130
30 80
80 30)
```

Bringing everything together, a complete iKe program using this definition:

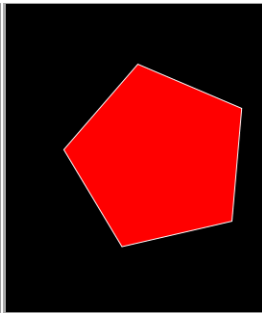
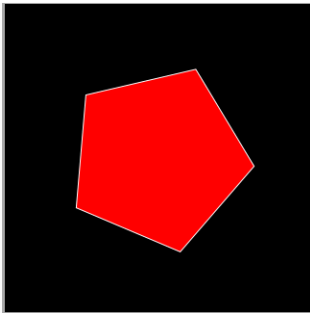
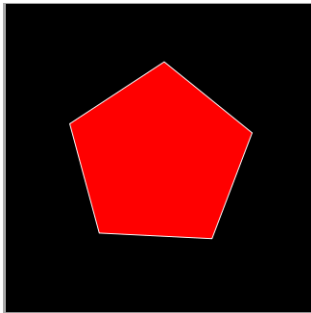
```
ngon: {+80+y*{ (cos x;sin x) } (2*pi%x) *!x}
, (ngon[5;50]; ("white"; "red"))
```



Drawing a procedurally generated pentagon.

Up to this point, we've been drawing a single static image by creating a program whose last expression returns an appropriate tuple. To create an animation, we must define a niladic procedure named `draw`. If present, this definition will be called 30 times per second and the result will be drawn. Throughout our code, we can also use the variable `f`, a frame counter which increments automatically every time the display is updated. By adding an appropriately scaled value to the radii we compute in `ngon` we can make the shape smoothly rotate:

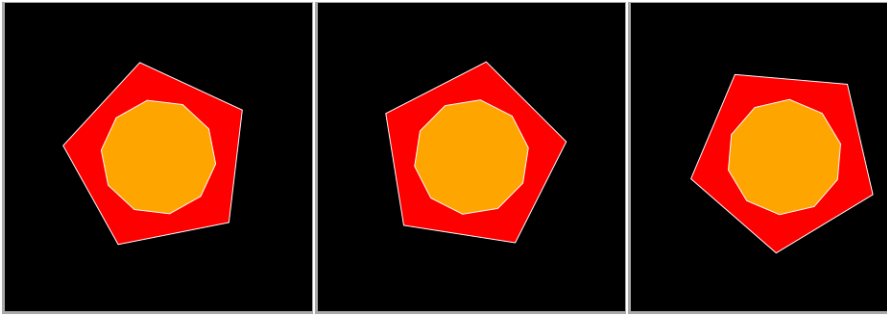
```
ngon: {+80+y*{ (cos x;sin x) } (f*.07)+(2*pi%x)*!x}
draw: {,(ngon[5;50]; ("white";"red")) }
```



A rotating pentagon.

To draw more shapes, it's simply a matter of modifying `draw` to return more tuples. Note how they're drawn back-to-front in the order they appear:

```
ngon: {+80+y*{ (cos x;sin x) } (f*.07)+(2*pi%x)*!x}
draw: {((ngon[ 5;50]; ("white";"red"      ))
      (ngon[10;30]; ("white";"orange")))}
```



Stacked polygons.

If a tuple is given a third element, it represents drawing a bitmap. The third element should be a matrix of indices to the color list. The first element gives a list of the positions where the top left corner of the bitmap will be drawn. The built-in variable `text` contains a list of 8x8 boolean matrices comprising a font:

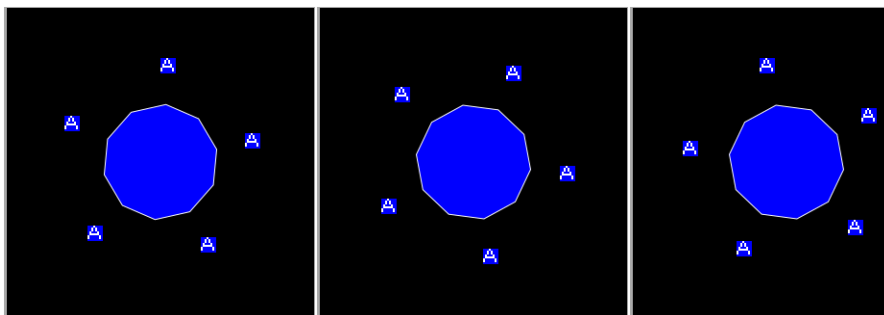
```
text@0+"A"
(1 1 1 0 1 1 1 1
 1 1 0 1 0 1 1 1
 1 1 0 1 0 1 1 1
 1 0 1 1 1 0 1 1
 1 0 0 0 0 0 1 1
 0 1 1 1 1 1 0 1
 0 1 1 1 1 1 0 1
 1 1 1 1 1 1 1 1)
```

With a small tweak to our existing program, we can draw bitmaps instead of the vertices of a polygon. It will be necessary to offset the coordinates to center each bitmap and floor the coordinates so that the bitmaps are drawn at integral positions to avoid blurring.

```
ngon: {+80+y*{(cos x;sin x)}(f*.07)+(2*pi%x)*!x}
draw: {( (-4+ngon[ 5;50]; ("white";"red");text@0+"A")
        (  ngon[10;30]; ("white";"orange")))}
```

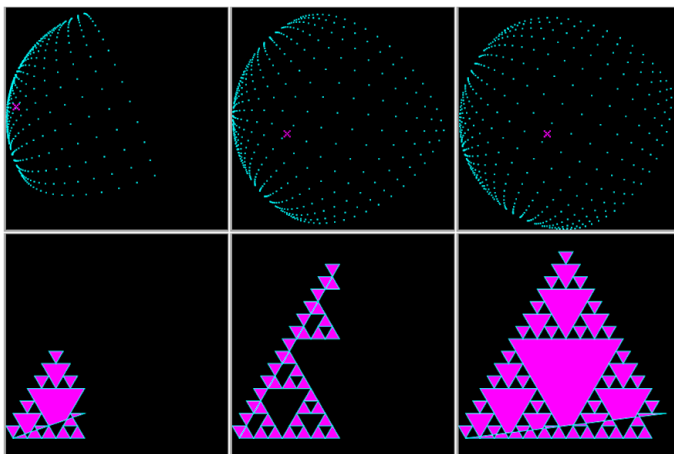
As the scene grows in complexity, we can also factor `draw` apart to keep the code easier to follow:

```
ngon: {+80+y*{(cos x;sin x)}(f*.07)+(2*pi%x)*!x}
palette: ("white";"blue")
letter: text@0+"A"
letters: {( (-4+ngon[5;50];palette;letter)}
decagon: {( ngon[10;30];palette)}
draw: {letters[],decagon[]}
```



Polygons and bitmaps.

Hopefully this has been enough to provide a taste of what working with iKe is like. The Github repository includes many more elaborate examples.



Panning around a hyperbolic projection (top) and a Lindenmayer System visualizer (bottom).

Conclusions and future work

We have seen how iKe enables the creation of graphical programs with K. Leveraging the web browser as a distribution platform lowers barriers to experimentation with the language and allows users to easily share their work and spread awareness. The ideas embodied in iKe could easily be translated to other array languages, and may help APL and J appeal to an entirely new audience.

There are still interesting IO facilities available in web browsers which could be exposed to iKe- for example, network communication via AJAX and the WebSockets protocol. The *oK* interpreter leaves substantial room for improving execution performance, and iKe may benefit from experimentation with selectively recompiling K code directly into JavaScript or exposing more pre-built library functions which can take care of heavy lifting such as image compositing operations. Alternatively, I could explore building a system similar to iKe which ran natively on desktop operating systems. While this would

require more user effort to install and set up, it could leverage the much faster official K implementations sold by Kx Systems. Finally, there is always room for improved documentation. iKe includes a manual and a collection of example programs, but guided tutorials and screencasts can improve accessibility and provide a gentler learning curve for beginners.

1. *iKe* <http://github.com/JohnEarnest/ok/tree/gh-pages/ike>
2. *oK* <http://github.com/JohnEarnest/ok>
3. *K5* <http://kparc.com>
4. *Forth Haiku* <http://forthsalon.appspot.com>
5. *Gist* <http://gist.github.com>

Subscribing to Vector

Your *Vector* subscription includes membership of the British APL Association which is open to anyone interested in APL or related languages. The membership year runs from 1 may to 30 April.

Name
.....

Address
.....

Postcode/Zip
.....

Telephone number
.....

Email address
.....

Membership	Price
UK private membership	£20
Overseas private membership	£22
+ airmail supplement outside Europe	£4
UK corporate membership	£100
Overseas corporate membership	£110
Non-voting UK member (student/OAP/unemployed)	£10

Payment methods (Sterling only)

- 1. A Sterling cheque, payable to *British APL Association*, drawn on a UK bank.
- 2. By American Express, MasterCard or Visa:

I authorise you to debit my American Express/MasterCard/Visa account

Number: Expires:/.....

.....
for the membership category indicated above.

Signature: Date:

- 3. By electronic transfer.

Our account details are: Barclay’s Bank; Cambridge; Chesterton Branch; Sort code: 20-17-35; Account number: 63955591; Account name: British APL Association; SWIFTBIC: BARCGB22; IBAN: GB86 BARC 2017 3563 9555 91.

- 4. By PayPal to credit account treasurer@vector.org.uk (no account needed)

If you pay by cheque or credit card please send completed form to:
BAA, c/o Nicholas Small, 12 Cambridge Road, Waterbeach, Cambridge CB25 9NJ